

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ДЕРЖАВНИЙ УНІВЕРСИТЕТ «ЖИТОМИРСЬКА ПОЛІТЕХНІКА»
ФАКУЛЬТЕТ ІНФОРМАЦІЙНО-КОМП'ЮТЕРНИХ ТЕХНОЛОГІЙ
КАФЕДРА ІНЖЕНЕРІЇ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

ПОЯСНЮВАЛЬНА ЗАПИСКА

до кваліфікаційної роботи освітнього ступеня «бакалавр»
за спеціальністю 121 «Інженерія програмного забезпечення»
(освітня програма «Інженерія програмного забезпечення»)

на тему:

«NoCode платформа конструювання вебресурсів»

Виконав студент групи ПЗ-22-3
ЛЕУС Вадим Олександрович

Керівник роботи:
САВІЦЬКИЙ Роман Святославович

Рецензент:
ВЛАСЕНКО Олег Васильович

Житомир – 2026

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ДЕРЖАВНИЙ УНІВЕРСИТЕТ «ЖИТОМИРСЬКА ПОЛІТЕХНІКА»
ФАКУЛЬТЕТ ІНФОРМАЦІЙНО-КОМП'ЮТЕРНИХ ТЕХНОЛОГІЙ
КАФЕДРА ІНЖЕНЕРІЇ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

«ЗАТВЕРДЖУЮ»

Зав. кафедри інженерії
програмного забезпечення
Тетяна Вакалюк
«28» лютого 2026 р.

ЗАВДАННЯ
на кваліфікаційну роботу

Здобувач вищої освіти: ЛЕУС Вадим Олександрович

Керівник роботи: САВІЦЬКИЙ Роман Святославович

Тема роботи: «NoCode платформа конструювання вебресурсів»,
затверджена Наказом закладу вищої освіти від «27» лютого 2026 р., №92.с

Вихідні дані для роботи: об'єктом дослідження є процес конструювання та динамічної генерації вебресурсів; предметом дослідження архітектурні рішення та програмні засоби розробки веборієнтованої NoCode платформи.

Розділ	Консультант	Завдання видав	Завдання прийняв
1	Савіцький Р.С.	03.03.2026	03.03.2026
2	Савіцький Р.С.	03.03.2026	03.03.2026
3	Савіцький Р.С.	03.03.2026	03.03.2026

РЕФЕРАТ

Пояснювальна записка до випускної кваліфікаційної роботи бакалавра на тему “NoCode платформа конструювання вебресурсів” містить 92 сторінок друкованого тексту. Робота ілюстрована 30 рисунками та містить 10 таблиць. Перелік використаних джерел налічує 30 найменувань і займає 3 сторінки. Пояснювальна записка також містить 6 сторінок додатків. Загальний обсяг роботи становить 98 сторінок.

Мета роботи полягає у проєктуванні та розробці NoCode платформи конструювання вебресурсів, яка забезпечує автоматизацію візуальної верстки, управління контентом та розгортання готових проєктів без необхідності написання програмного коду.

Об’єкт дослідження – процес конструювання та динамічної генерації вебресурсів.

Предмет дослідження – архітектурні рішення та програмні засоби розробки веборієнтованої NoCode платформи.

У першому розділі проведено аналіз предметної області, визначено цільову аудиторію, сформовано функціональні та нефункціональні вимоги, а також обґрунтовано вибір технологічного стеку і гібридної СУБД MySQL.

У другому розділі виконано комплексне об’єктно-орієнтоване проєктування: розроблено UML-моделі архітектури, бази даних, поведінки системи та алгоритмів взаємодії компонентів.

У третьому розділі здійснено програмну реалізацію клієнтської та серверної частин, проведено багаторівневе тестування та успішно розгорнуто платформу в розподіленій хмарній інфраструктурі.

КЛЮЧОВІ СЛОВА: NOCODE, ВЕБРЕСУРС, ВІЗУАЛЬНИЙ РЕДАКТОР, JSON, DRAG-AND-DROP.

					ІПЗ.КР.Б. – 121 – 26 – ПЗ				
Змн.	Арк.	№ докум.	Підпис	Дата					
Розроб.		Леус В.О.			NoCode платформа конструювання вебресурсів		Літ.	Арк.	Аркушів
Керівник		Савіцький Р.С.						3	98
							ФІКТ Гр. ІПЗ-22-3		
Н. контр.		Тичина О.П.							
Зав. каф.		Вакалюк Т.А.							

ABSTRACT

The explanatory note to the bachelor's thesis on the topic "NoCode platform for constructing web resources" contains 92 pages of printed text. The work is illustrated with 30 figures and contains 10 tables. The list of references includes 30 items and takes up 3 pages. The explanatory note also contains 6 pages of appendices. The total volume of the work is 98 pages.

The purpose of the work is to design and develop a NoCode platform for constructing web resources, which provides automation of visual layout, content management, and deployment of finished projects without the need to write program code.

The object of research is the process of constructing and dynamically generating web resources.

The subject of research is architectural solutions and software tools for developing a web-based No-Code platform.

In the first chapter, a domain analysis is conducted, the target audience is defined, functional and non-functional requirements are formulated, and the choice of the technology stack and the hybrid MySQL DBMS is justified.

In the second chapter, comprehensive object-oriented design is performed: UML models of the architecture, database, system behavior, and component interaction algorithms are developed.

In the third chapter, the software implementation of the client and server parts is carried out, multi-level testing is conducted, and the platform is successfully deployed in a distributed cloud infrastructure.

KEYWORDS: NOCODE, WEB RESOURCE, VISUAL EDITOR, JSON, DRAG-AND-DROP.

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		4

ЗМІСТ

РЕФЕРАТ	3
ABSTRACT	4
ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ.....	6
ВСТУП	7
РОЗДІЛ 1. ПРЕДМЕТНА ОБЛАСТЬ ТА ПОСТАНОВКА ЗАДАЧІ	10
1.1 Постановка задачі та огляд існуючих платформ-аналогів	10
1.2 Функціональні та нефункціональні вимоги до системи	21
1.3 Вибір і обґрунтування архітектурного рішення	25
1.4 Вибір інструментальних засобів для реалізації	29
Висновки до першого розділу.....	32
РОЗДІЛ 2. ПРОЄКТУВАННЯ ПРОГРАМНОЇ СИСТЕМИ	34
2.1 Моделювання варіантів використання та поведінки системи.....	34
2.2 Проєктування логічної структури бази даних	39
2.3 Проєктування компонентної структури та взаємодії модулів	45
2.4 Розробка основних алгоритмів роботи платформи	56
Висновки до другого розділу	62
РОЗДІЛ 3. РЕАЛІЗАЦІЯ ТА ТЕСТУВАННЯ ПРОЄКТУ	63
3.1 Програмна реалізація серверної частини та підсистеми безпеки	63
3.2 Розробка візуального редактора та клієнтського інтерфейсу	66
3.3 Керівництво користувача та розгортання платформи	68
3.4 Тестування програмної системи.....	81
Висновки до третього розділу	85
ВИСНОВКИ.....	87
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ.....	89
ДОДАТКИ.....	92

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ

БД – база даних

ПЗ – програмне забезпечення

СУБД – система управління базами даних

API – Application Programming Interface – прикладний програмний інтерфейс

CMS – Content Management System – система управління контентом

CSS – Cascading Style Sheets – каскадні таблиці стилів

HTML – HyperText Markup Language – мова гіпертекстової розмітки

JSON – JavaScript Object Notation – текстовий формат обміну даними

JWT – JSON Web Token – відкритий стандарт безпечної передачі інформації у вигляді JSON-об'єкта

REST – Representational State Transfer – архітектурний стиль взаємодії компонентів розподіленого застосунку в мережі

SPA – Single Page Applications – односторінковий вебзастосунок

SSL/TLS – Secure Sockets Layer / Transport Layer Security – криптографічні протоколи, що забезпечують безпечну передачу даних у мережі

UI/UX – User Interface / User Experience – інтерфейс та досвід користувача

HTML/CSS

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		6

ВСТУП

Актуальність теми. Розробка вебсайтів вимагає залучення цілої команди фахівців: дизайнерів, розробників клієнтської та серверної частин, тестувальників. Оскільки цей процес зазвичай передбачає написання коду з нуля, він залишається досить ресурсомістким, дорогим і забирає багато часу.

В умовах жорсткої конкуренції компаніям потрібні швидкі, гнучкі та дешевші способи запуску своїх вебресурсів. Актуальність впровадження NoCode-платформ полягає у їхній здатності демократизувати процес розробки програмного забезпечення. Усуваючи високий бар'єр входу, такі рішення дозволяють користувачам без глибоких технічних знань самостійно створювати повноцінні вебсторінки, використовуючи інтуїтивно зрозумілі візуальні інтерфейси та технологію Drag-and-Drop. Це значно прискорює вихід продукту на ринок та суттєво знижує поріг входження у сферу вебконструювання.

Водночас розробка самих NoCode-систем це нетривіальна архітектурна задача. Щоб перейти від жорсткої статичної розмітки до динамічної генерації інтерфейсів, потрібні нові підходи до структурування та збереження даних. Наприклад, якщо зберігати згенерований контент просто як готовий HTML-код, втрачається семантика, і його стає важко редагувати в майбутньому. Тому виникає потреба у платформах, які використовують компонентний підхід, серіалізують структуру сторінок у форматі JSON та вдало поєднують гнучкість документо-орієнтованих моделей із надійністю реляційних баз даних. Зважаючи на це, розробка власної NoCode-платформи з використанням архітектурних патернів та оптимізованих методів збереження конфігураційних даних є актуальним науково-практичним завданням.

Об'єкт дослідження – процес конструювання та динамічної генерації вебресурсів.

Предмет дослідження – архітектурні рішення та програмні засоби розробки веборієнтованої NoCode платформи.

Мета роботи полягає у проєктуванні та розробці системи для візуального конструювання вебсторінок. Ця система має автоматизувати верстку, управління

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		7

контентом та розгортання вебресурсів, позбавляючи користувача необхідності писати програмний код.

Для досягнення цієї мети було визначено такі завдання:

- проаналізувати предметну область, дослідити концепцію NoCode-розробки та зробити огляд наявних на ринку аналогів;
- обґрунтувати вибір технологічного стеку, а також визначити функціональні та нефункціональні вимоги до програмного забезпечення й апаратних ресурсів;
- спроектувати архітектуру платформи, її об'єктно-орієнтовану та компонентну структуру;
- розробити структуру бази даних на основі гібридної моделі та описати алгоритми взаємодії компонентів системи;
- реалізувати саму NoCode-платформу, серверну частину, візуальний Drag-and-Drop редактор сторінок, особистий кабінет користувача та панель адміністратора;
- протестувати систему, налаштувати механізми безпеки, а також розгорнути готовий продукт для демонстрації його роботи.

Методи дослідження. Під час роботи застосовувалися методи системного та порівняльного аналізу для огляду рішень-аналогів і формування вимог, об'єктно-орієнтованого та компонентного проєктування для побудови архітектури, а також методи моделювання баз даних.

Практичне значення одержаних результатів. Розроблена NoCode-платформа дає користувачам зручний та інтуїтивно зрозумілий інструмент для швидкого створення, налаштування і публікації вебпроектів. Це не лише знижує поріг входження у веброзробку, а й допомагає бізнесу суттєво економити час та гроші. Також сформовано архітектурну основу платформи, яка забезпечує високу масштабованість та здатність до швидкої інтеграції зі сторонніми сервісами.

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		8

Апробація результатів роботи. Основні теоретичні та практичні результати дослідження, що викладені у випускній кваліфікаційній роботі, доповідалися на двох науково-практичних конференціях:

Леус В. О., Савіцький Р. С. Особливості реалізації компонентного підходу при розробці систем динамічної генерації вебінтерфейсів // XIX Всеукраїнська науково-практична WEB конференція аспірантів, студентів та молодих вчених «Комп’ютерні інтелектуальні системи та мережі» (KICM-2026). — Кривий Ріг: Криворізький національний університет, 25-27 березня 2026 р. [1]

Леус В. О., Савіцький Р. С. Методи структуризації та збереження даних у системах візуального конструювання вебресурсів // XI Міжнародна науково-технічна конференція «Інформаційно-комп’ютерні технології». — Житомир: Державний університет «Житомирська політехніка», 02-03 квітня 2026 р. [2]

Для однозначного тлумачення специфічних термінів, інженерних понять та скорочень, що використовуються в межах даного дипломного проєкту, до роботи додається словник термінів (Додаток А).

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		9

РОЗДІЛ 1. ПРЕДМЕТНА ОБЛАСТЬ ТА ПОСТАНОВКА ЗАДАЧІ

1.1 Постановка задачі та огляд існуючих платформ-аналогів

Предметною областю даного дипломного проєкту є сфера веброзробки, а саме сегмент NoCode рішень, які надають можливість користувачам без спеціалізованої технічної підготовки створювати та розгортати повноцінні вебресурси. Аналіз предметної області та проблематики показує, що розвиток цифрової економіки диктує нові правила для сучасного бізнесу: наявність представництва в глобальній мережі Інтернет стала базовою вимогою для виживання та масштабування будь-якої комерційної діяльності.

Традиційний процес створення вебсайту є складним, багатоетапним та ресурсомістким циклом. Для реалізації навіть відносно простого корпоративного сайту чи інтернет-магазину зазвичай потрібне залучення цілої команди профільних фахівців. Спочатку UI/UX дизайнери проєктують візуальні макети та логіку взаємодії користувача з інтерфейсом. Далі Frontend-розробники перетворюють ці графічні макети у робочий код за допомогою HTML, CSS та JavaScript [3], забезпечуючи кросбраузерність та адаптивність під різні розміри екранів. Паралельно Backend-розробники проєктують архітектуру бази даних, пишуть серверну логіку та налаштовують серверну інфраструктуру. Завершується цей цикл етапом тестування, де виявляються та виправляються програмні дефекти.

Такий класичний підхід є дорогим і тривалим. Для малого бізнесу, приватних підприємців, фрілансерів або громадських ініціатив витрати на розробку вебсайту з нуля часто є невідомими. Крім того, будь-яка зміна на сайті, наприклад додавання нового розділу чи зміна розташування інформаційного блоку, вимагає повторного звернення до розробників, оскільки модифікація структури передбачає втручання у вихідний програмний код. Це створює залежність власника бізнесу від ІТ-спеціалістів та значно збільшує вартість володіння вебресурсом. Відмінності між традиційним підходом до

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		10

веброзробки та концепцією NoCode, зокрема щодо вартості, тривалості та складності створення вебресурсу, наведено на схемі (рис. 1.1).



Рис. 1.1. Порівняння традиційного циклу веброботи та концепції NoCode

Усвідомлення цього бар'єру призвело до еволюції інструментарію веброботи та появи концепції NoCode, тобто розробки без коду. Основною ідеєю цього підходу є абстрагування складності: рутинні технічні процеси перекладаються з людини на програмні алгоритми системи. Замість написання програмного коду користувач взаємодіє зі зрозумілим, переважно графічним інтерфейсом, візуально формуючи структуру сторінки. Загальний принцип функціонування такого підходу та взаємодії користувача із системою представлено у вигляді концептуальної схеми (рис. 1.2.).



Рис. 1.2. Концептуальна схема роботи розроблюваної NoCode платформи

У рамках проєктування розроблюваної системи необхідно вирішити завдання щодо автоматизації ключових етапів створення вебресурсу. Найбільш трудомісткою складовою є автоматизація процесу верстки інтерфейсу. У ручному режимі розробник повинен прописувати ієрархію тегів, налаштовувати CSS-класи, відступи та кольори. Система повинна автоматизувати цей процес через впровадження блокового підходу. Користувач має оперувати високорівневими абстракціями – готовими функціональними блоками, а платформа у фоновому режимі автоматично трансформуватиме цей вибір у семантично правильний і валідний HTML-код.

Важливою вимогою до системи є автоматичне забезпечення адаптивності, оскільки ручна оптимізація макета під мобільні пристрої, планшети та настільні монітори вимагає від користувача глибокого розуміння медіазапитів і принципів побудови гнучких сіток. Програмний комплекс повинен вирішувати цю проблему автоматично, надаючи блокам попередньо налаштовану поведінку, яка самостійно перебудовує контент залежно від роздільної здатності екрану користувача без жодного втручання з боку творця сайту.

Для автоматизації управління контентом та медіаресурсами система повинна реалізувати принцип “What You See Is What You Get”. Це означає, що зміна текстового наповнення чи заміна зображень має відбуватися безпосередньо в макеті сторінки, з миттєвим відображенням результату. Додатково система має автоматизувати базову обробку медіафайлів: кадрування та оптимізацію розміру зображень для вебсередовища, виключаючи необхідність використання сторонніх графічних редакторів користувачами, які часто завантажують неоптимізовані файли надмірних розмірів.

Крім візуальної частини, система має автоматизувати структурну побудову ресурсу та роботу з динамічними даними. Створення багатосторінкових сайтів вимагає налаштування маршрутизації, генерації унікальних URL-адрес та побудови навігаційних меню. Більш того, для реалізації процесів електронної комерції платформа повинна забезпечити механізми прив'язки даних. Зміна налаштувань одного шаблонного елемента в каталозі товарів має автоматично

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		12

застосовуватися до всіх позицій, що повністю виключає необхідність ручної верстки кожної окремої картки товару.

Опираючись на проведений аналіз, головним завданням даної роботи є проектування та розробка програмного продукту – “NoCode платформа конструювання вебресурсів”. Ця система повинна стати комплексним рішенням, де користувач без навичок програмування зможе через візуальний інтерфейс за принципом Drag-and-Drop зібрати повноцінний, адаптивний та швидкодіючий сайт. Платформа має приховати від кінцевого користувача всю технічну складність взаємодії з базами даних, серверами та кодом, надавши йому інтуїтивно зрозумілий інструментарій для реалізації його бізнес-потреб. Досягнення цієї мети вимагає глибокого дослідження архітектурних підходів до побудови візуальних редакторів [4], методів серіалізації DOM-дерева та організації ефективної клієнт-серверної взаємодії.

Дослідження концепції NoCode-розробки та архітектурних підходів. Постійне зростання вимог до швидкості розробки програмного забезпечення спричинило масове поширення концепції NoCode. У контексті веброзробки парадигма вимагає кардинальної зміни внутрішньої архітектури застосунків: процес написання вихідного програмного коду замінюється взаємодією користувача з графічним інтерфейсом. З інженерної точки зору це означає, що рутинні технічні процеси, такі як формування DOM-дерева, налаштування маршрутизації та транзакції до бази даних, мають бути глибоко інкапсульовані всередині ядра платформи, тоді як кінцевий користувач оперує виключно візуальними сутностями.

Ключовим механізмом взаємодії у системах візуального конструювання є технологія Drag-and-Drop. За класичного підходу до веброзробки зміна структури документа вимагає маніпуляцій з об'єктною моделлю через написання відповідних скриптів. Натомість Drag-and-Drop дозволяє користувачеві за допомогою вказівного пристрою захоплювати візуальні елементи інтерфейсу, переміщувати їх у робочій області та фіксувати у потрібному місці. З архітектурної точки зору, кожна така дія автоматично перехоплюється системою

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		13

та транслюється у математичний розрахунок нових координат або індексів масиву, що миттєво оновлює внутрішній стан системи та викликає перемальовування інтерфейсу без перезавантаження сторінки.

При проектуванні NoCode платформ конструювання вебресурсів фундаментальним рішенням є вибір базового архітектурного підходу до формування макета сторінки. Аналіз існуючих систем візуального редагування дозволяє виділити два основні підходи: редактори вільної форми Free-form та сіткові або блокові редактори Grid/Sequential.

Підхід вільної форми, який базується на абсолютному позиціонуванні елементів, надає користувачеві максимальну свободу дій. Інтерфейс такого редактора нагадує класичні графічні редактори. Користувач може розмістити будь-який елемент у будь-якій точці екрана, задаючи його позицію за допомогою координат X та Y відносно вікна. Незважаючи на гнучкість, цей підхід має суттєві архітектурні недоліки. Згенерований таким чином HTML-код зазвичай є надлишковим і позбавленим семантичної структури, що негативно впливає на пошукову оптимізацію та доступність. Найбільшою проблемою вільної форми є забезпечення адаптивності. Оскільки координати елементів жорстко зафіксовані, при зміні роздільної здатності екрана елементи починають накладатися один на одного або виходити за межі видимості, що вимагає від користувача ручного переналаштування координат для кожного типу пристроїв.

Альтернативним підходом є сіткова або блокова структура макета. Ця концепція базується на природному потоці документа, що використовується у вебстандартах. Замість абсолютних координат елементи позиціонуються відносно один одного, заповнюючи доступний простір контейнерів, розташованих у вигляді послідовних горизонтальних блоків або комірок гнучкої сітки. Такий підхід ідеально підходить для розробки "NoCode платформи конструювання вебресурсів". Його головною перевагою є нативна адаптивність: контент автоматично перебудовується, стискається або переноситься на нові рядки залежно від ширини екрана клієнтського пристрою. Крім того, блокова архітектура дозволяє генерувати чистий, семантично правильний HTML-код.

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		14

Реалізація такого підходу вимагає перегляду класичних методів веброзробки. За останні роки у сфері розробки інтерфейсів спостерігається стійкий перехід від простих статичних вебсторінок до складніших Single Page Applications, орієнтованих на інтерактивність та можливість гнучкого налаштування користувацького інтерфейсу. Додатковим чинником, що вплинув на актуальність розробки NoCode платформ, стала необхідність динамічного формування структури вебсторінок на основі конфігураційних даних, а не жорстко заданої розмітки.

Зазначений підхід узгоджується з концепцією побудови інтерфейсів, яка застосовується у сучасних JavaScript-бібліотеках, таких як React [5]. В її основі лежать компонентний підхід, повторне використання елементів та декларативний опис UI. На практиці це дозволяє розглядати інтерфейс платформи як набір незалежних ізольованих компонентів, що можуть комбінуватися між собою залежно від вимог конкретної сторінки, яку проєктує користувач.

Разом із тим класичні підходи до верстки, за яких структура сторінки фіксується безпосередньо в HTML або JSX-коді, мають низку обмежень, які роблять їх непридатними для систем візуального конструювання. Навіть незначні зміни, наприклад перестановка блоків або коригування їхніх параметрів, за класичної розробки потребують внесення змін у програмний код розробником та повторного розгортання застосунку. У межах довготривалої підтримки проєкту це ускладнює розвиток системи та збільшує часові витрати на модифікацію інтерфейсу.

Для вирішення цієї проблеми під час розробки архітектури платформи доцільно застосувати концепцію Schema-Driven UI – інтерфейс, керований схемою. Цей архітектурний патерн передбачає повне відокремлення опису структури інтерфейсу від логіки його відображення. Замість збереження готової HTML-розмітки, конфігурація кожної сторінки сайту зберігається у вигляді спеціалізованого формату обміну даними – JSON [6]. Уся структура сторінки

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		15

описується як ієрархічне дерево об'єктів, де кожен об'єкт містить ідентифікатор типу компонента, його унікальний ключ та набір параметрів.

Принцип роботи системи візуального конструювання, побудованої за такою концепцією, полягає у поетапній обробці конфігураційних даних. Після отримання JSON-схеми з сервера, клієнтський застосунок платформи передає її до модуля рендерингу, який рекурсивно та послідовно обробляє описані blocks. Для кожного елемента, залежно від його типу, динамічно підключається відповідний компонент системи, якому передаються параметри зі схеми. У результаті формується фінальний вигляд сторінки без необхідності жорстко задавати її структуру у вихідному коді.

Для забезпечення узгодженого зовнішнього вигляду інтерфейсу та уникнення хаосу при візуальному проектуванні, архітектура таких систем часто спирається на принципи методології Atomic Design. Відповідно до цієї концепції, компоненти організовуються у вигляді чіткої ієрархії: від найпростіших базових елементів до складніших композицій, що формують окремі готові секції сторінки. Такий підхід дає змогу розробити для NoCode платформи обмежений, але гнучкий набір стандартизованих блоків і зберігати візуальну цілісність інтерфейсів, які створюватимуть кінцеві користувачі, гарантуючи при цьому стабільність верстки при їх взаємодії через механізми Drag-and-Drop.

Аналіз існуючих платформ-аналогів. Ринок інструментів для веброзробки є насиченим і висококонкурентним. Сегмент NoCode рішень демонструє стабільне зростання, оскільки бізнес прагне зменшити залежність від дорогих команд розробників та пришвидшити вихід цифрових продуктів на ринок. Для обґрунтування доцільності розробки власної системи та формування об'єктивних вимог до неї було проведено аналіз існуючих рішень. Для аналізу було обрано платформи, які є лідерами ринку NoCode-розробки та використовують різні архітектурні підходи: Free-form та Grid/Sequential. В якості аналогів для порівняння обрано три популярні платформи: Wix [7], Tilda Publishing [8] та Webflow [9]. Детальний порівняльний аналіз цих сервісів дозволить чітко виявити їхні слабкі сторони для врахування під час розробки власного продукту.

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		16

Wix – це хмарна платформа для створення вебсайтів, яка історично є одним із лідерів масового ринку. Довгий час платформа базувалася виключно на підході вільної форми, надаючи користувачам абсолютну свободу переміщення елементів.

Принцип роботи класичного редактора полягає в тому, що елементи позиціонуються за абсолютними координатами, що дозволяє розмістити їх у будь-якій точці екрана. Головною перевагою платформи є величезна бібліотека готових шаблонів, вбудований магазин додатків для розширення функціоналу та низький поріг входження для новачків.

Однак цей архітектурний підхід зумовлює суттєві недоліки. Через абсолютне позиціонування елементів генерований HTML-код є надлишковим і складним для індексації пошуковими системами. Свобода переміщення створює критичні проблеми з адаптивністю: на мобільних пристроях елементи часто накладаються один на одного, вимагаючи кропіткої ручної правки для кожного типу екранів. Крім того, платформа характеризується жорсткою прив'язкою до постачальника – користувач не має можливості експортувати чистий код сайту для розміщення на власному хостингу, залишаючись залежним від інфраструктури Wix. Загальний вигляд візуального редактора платформи представлено на рис. 1.3.

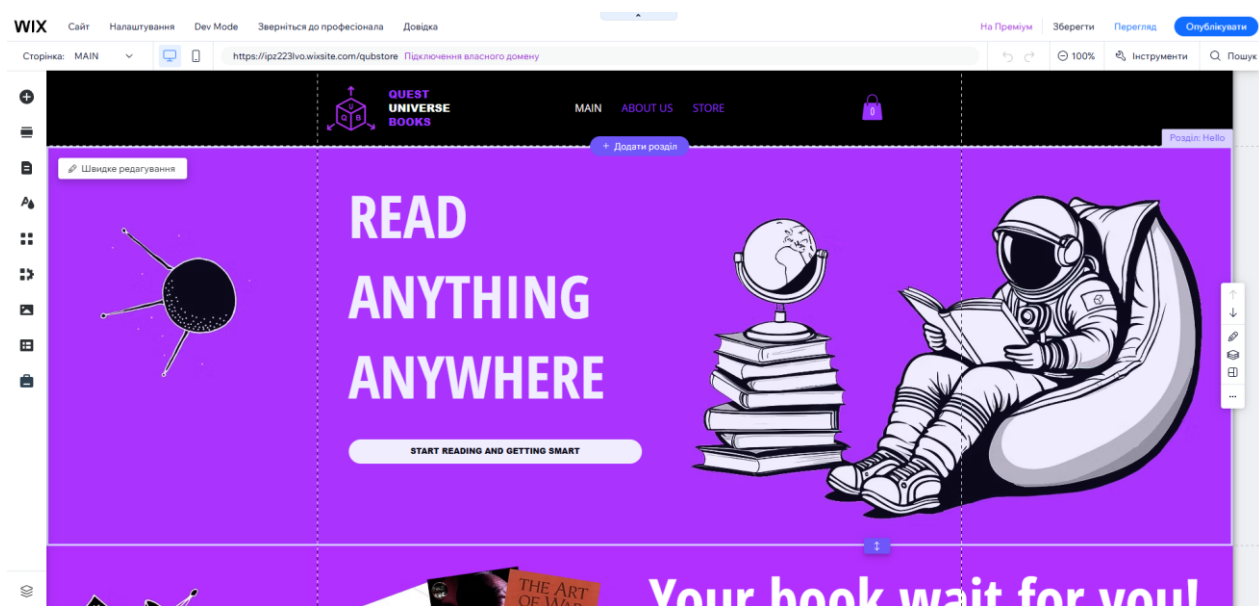


Рис. 1.3. Інтерфейс візуального редактора платформи Wix

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		17

Tilda – це популярний конструктор сайтів, який концептуально базується на блоковому підході. Основна ідеологія платформи полягає у послідовному збиранні сторінки з готових функціональних блоків, що концептуально наближено до системи, яка розробляється у даному дипломному проєкті.

Принцип роботи полягає в тому, що кожен блок має заздалегідь визначені, фіксовані налаштування дизайну та поведінки, що гарантує гармонійний вигляд кінцевого результату. Перевагами системи є висока швидкість збірки посадкових сторінок, нативна адаптивність "з коробки" та висока естетична якість базових шаблонів.

Основні недоліки системи зумовлені жорстко заданою номенклатурою доступних блоків, що обмежує гнучкість візуального наповнення сторінок. Користувач має дуже обмежені можливості щодо кастомізації стандартних блоків. Якщо виникає потреба змінити розташування певного елемента наприклад, перенести кнопку в іншу частину стандартного блоку, це неможливо зробити без переходу у спеціалізований "Zero Block" – складний внутрішній редактор вільної форми. Крім того, функціонал електронної комерції та можливість експорту коду доступні виключно на найдорожчих тарифних планах. Внутрішня серверна логіка магазину є базовою і не підходить для масштабування великих каталогів зі складною фільтрацією. Загальний вигляд інтерфейсу візуального редактора платформи Tilda представлено на рис. 1.4.

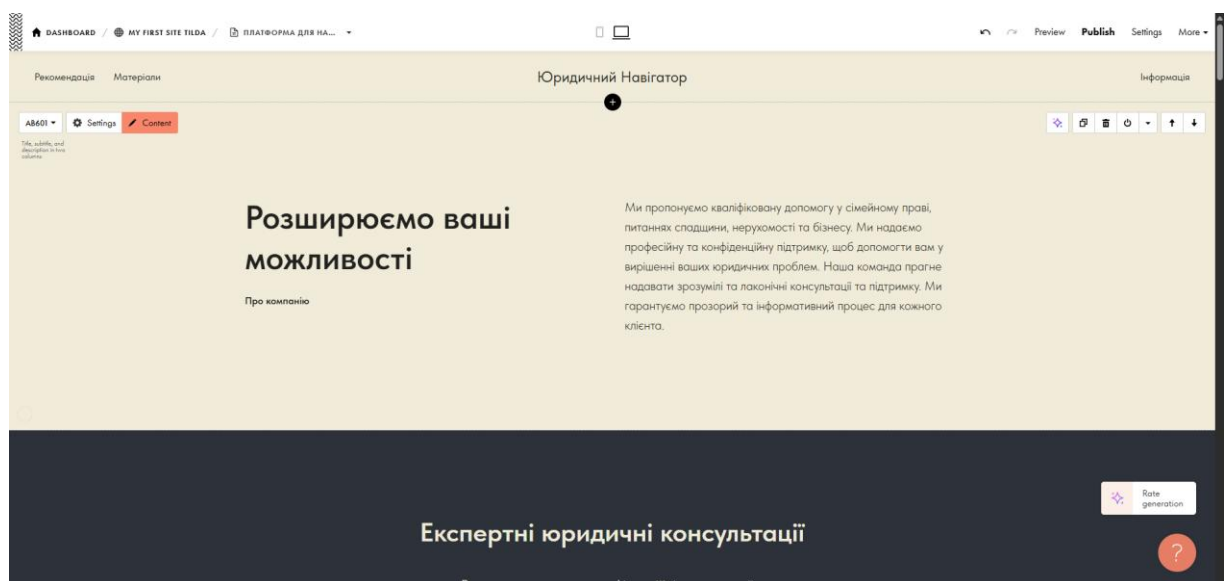


Рис. 1.4. Інтерфейс візуального редактора платформи Tilda

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		18

Webflow – це професійний інструмент веброзробки, який є еталоном сіткового підходу та позиціонується як візуальний інтерфейс для написання коду. За рівнем складності цю платформу доцільно класифікувати як Low-Code рішення, оскільки ефективна робота в ній вимагає розуміння принципів верстки, каскадних таблиць стилів та коробкової моделі.

Інтерфейс редактора візуально нагадує інтегровані середовища розробки або складні графічні редактори. Користувач безпосередньо оперує такими поняттями, як Flexbox, Grid, Padding, Margin, Z-index. Головною перевагою Webflow є повний контроль над дизайном і складними анімаціями, генерація чистого, валідного та семантичного коду, а також наявність потужної вбудованої системи управління контентом для роботи з динамічними даними.

Проте ці переваги формують і головні недоліки системи для масового користувача. Webflow має надзвичайно високий поріг входження: людина без базових знань HTML/CSS не зможе ефективно користуватися платформою. Інтерфейс є перевантаженим для звичайного власника бізнесу або контент-менеджера. Крім того, це одне з найдорожчих рішень на ринку, що робить його недоцільним для тестування простих гіпотез або невеликих проєктів. Загальний вигляд та особливості інтерфейсу візуального редактора платформи Webflow представлено на рис. 1.5.

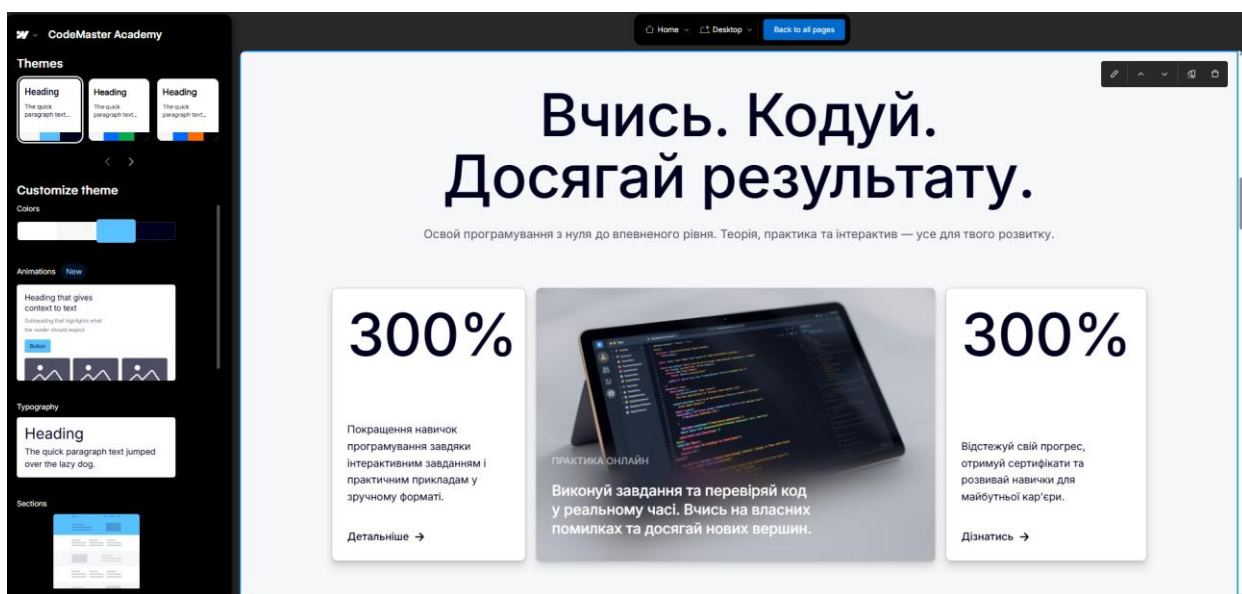


Рис. 1.5. Інтерфейс візуального редактора платформи Webflow

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				19
Змн.	Арк.	№ докум.	Підпис	Дата		

Порівняльна характеристика платформ. Для наочної систематизації результатів аналізу аналогів та обґрунтування функціональних вимог до власної розробки, основні критерії порівняння зведено у таблицю (табл. 1.1.).

Таблиця 1.1.

Порівняння систем конструювання вебресурсів

Функціональний критерій	Wix	Tilda	Webflow	Kendr
Візуальний Drag-and-Drop редактор сторінок	Так	Так	Так	Так
Наявність готових пресетів для блоків	Так	Так	Так	Так
Автоматична адаптивність макетів – Mobile-friendly	Так	Так	Так	Так
Наскрізнi глобальні компоненти – Header/Footer	Так	Так	Так	Так
Базовий модуль електронної комерції (Магазин)	Тільки Premium	Тільки Premium	Окремий тариф	Безкоштовно
Керування кошиком та оформленням замовлень	Так	Так	Так	Так
Підтримка мультикатегорійності товарного каталогу	Так	Так	Так	Так
Підтримка мультивалютності товарів	Так	Частково	Так	Так
Відстеження та управління статусами замовлень	Так	Так	Так	Так
Інтегрована панель адміністратора/модератора	Ні	Ні	Ні	Так
Динамічне керування модулями (вимкнен)	Обмежено	Ні	Ні	Так
Вбудований локальний Cookie-банер Privacy	Так	Так	Частково	Так
Наявність локальної панелі аналітики проєктів	Так	Так	Ні	Так
Генерація та валідація унікальних URL-слагів	Так	Так	Так	Так
Автоматичне фонове збереження змін Auto-save	Так	Ні	Так	Так
Вбудований антиспам-контроль неоплачених замовлень	Ні	Ні	Ні	Так
Відкритий формат збереження даних	Закритий	Закритий	Закритий	Відкритий
Відкритий REST API для інтеграцій	Частково	Обмежено	Так	Так

Проведений аналіз продемонстрував, що на ринку існуючих комерційних рішень відсутній збалансований підхід. Незважаючи на потужний функціонал існуючих рішень, усі вони мають критичні недоліки, які обмежують їх

застосування для певних категорій користувачів: повністю закритий вихідний код та прив'язка до інфраструктури постачальника, неможливість розгортання платформи на власних серверах компанії, перевантажений інтерфейс, що вимагає знань програмування, або висока вартість регулярної підписки.

Тому розробка власної платформи є доцільною та науково обґрунтованою. Створювана NoCode система має зайняти проміжну нішу: вона поєднає легкість та зрозумілість компонентного підходу для новачків із генерацією чистого коду. Зберігання структури кожної сторінки у відкритому та серіалізованому JSON-форматі дозволить уникнути проблеми експорту коду, забезпечить можливість вільного експорту даних та дозволить компаніям розгортати редактор на власних серверах, зберігаючи повний контроль над своїми даними та інфраструктурою.

1.2 Функціональні та нефункціональні вимоги до системи

Визначення цільової аудиторії та аналіз основних сценаріїв використання системи є ключовим етапом проєктування, який передувє безпосередньому формуванню вимог до програмного продукту. Розуміння потреб кінцевих користувачів дозволяє оптимізувати користувацький інтерфейс, покращити досвід взаємодії та сфокусувати зусилля на розробці найбільш затребуваного функціоналу. Аналіз ринку NoCode рішень дозволяє виділити три основні сегменти цільової аудиторії розроблюваної платформи, кожен з яких має власні болі та специфічні вимоги до системи.

Першим сегментом є власники малого бізнесу та мікропідприємці. Це користувачі без технічної освіти, яким необхідне швидке представлення свого бізнесу в мережі Інтернет. Основна проблематика полягає в тому, що замовлення розробки сайту у вебстудії вимагає значних фінансових та часових інвестицій, а будь-яка зміна контенту у майбутньому вимагатиме залучення спеціаліста. Платформа вирішує цю проблему шляхом надання інтуїтивно зрозумілого редактора, який дозволяє зібрати сторінку з готових функціональних блоків без написання жодного рядка коду. Можливість публікації проєкту дозволяє підприємцю отримати готовий до роботи вебресурс за кілька годин.

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		21

Другий сегмент аудиторії – маркетологи та спеціалісти з реклами. Це професіонали, яким необхідно постійно тестувати нові продуктові гіпотези, створювати посадкові сторінки під різні рекламні кампанії та аналізувати трафік. Їхньою головною проблемою є залежність від ІТ-відділу або розробників, через що процес від створення макета до його верстки та публікації займає занадто багато часу і знижує ефективність маркетингових кампаній. Рішенням від платформи є надання можливості дублювання існуючих проєктів для проведення А/В тестування, наявність вбудованих інструментів для базової SEO-оптимізації та висока швидкість завантаження згенерованих сторінок, що позитивно впливає на конверсію.

Третій сегмент складають незалежні вебдизайнери та фрілансери, які створюють сайти на замовлення. Вони розуміються на принципах композиції та UI/UX дизайну, але не володіють глибокими знаннями у програмуванні. Проблематика полягає в тому, що більшість існуючих NoCode платформ є закритими екосистемами. Якщо клієнт хоче перенести свій сайт на власний сервер з міркувань безпеки або корпоративних політик, популярні конструктори не дозволяють цього зробити. На відміну від жорстко обмежених аналогів, розроблювана платформа вирішує це завдання шляхом підтримки функції експорту готового проєкту. Дизайнер може спроектувати інтерфейс у візуальному редакторі, а потім вивантажити згенерований оптимізований вихідний код для незалежного хостингу. При цьому встановлено важливе архітектурне обмеження: система функціонує за принципом одностороннього потоку даних від редактора до коду, а функція зворотного імпорту стороннього коду в редактор свідомо не реалізовується для збереження цілісності JSON-схеми платформи.

На основі профілів цільової аудиторії сформовано базові сценарії взаємодії користувачів із системою, які підтверджують здатність платформи покривати різні потреби.

На основі профілів цільової аудиторії сформовано базові сценарії взаємодії користувачів із системою, які підтверджують здатність платформи покривати

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				22
Змн.	Арк.	№ докум.	Підпис	Дата		

різні потреби. Під час швидкого запуску посадкової сторінки робота з новим проєктом охоплює наповнення робочого простору візуального редактора структурними блоками. Формування макета забезпечується інструментами панелі налаштувань для адаптації кольорової палітри, типографіки та завантаження персональних медіафайлів. Оптимізація ресурсу відбувається через встановлення базових SEO-параметрів, а публікації проєкту робить сайт доступним глобально.

Інший характер взаємодії реалізується у сценарії прототипування. Спроектований у візуальному редакторі макет багатосторінкового сайту-візитки для корпоративного клієнта може обходити етап розгортання на інфраструктурі платформи. Натомість вебдизайнер застосовує процедуру експорту, яка активує серверну генерацію архіву з чистим, оптимізованим вихідним кодом. Отримані файли дозволяють вільно розгорнути створений вебресурс на будь-якому незалежному або виділеному сервері замовника. Запропонована варіативність сценаріїв доводить універсальність розробленого інструментарію та його здатність гнучко адаптуватися під бізнес-процеси кожної групи користувачів.

Функціональні вимоги до системи

Функціонал розроблюваного програмного комплексу базується на чіткому переліку завдань, виконання яких забезпечує вирішення поставлених бізнес-потреб. Архітектура платформи базується на системі управління доступом на основі ролей. У системі передбачено чотири рівні доступу: гість, авторизований користувач, модератор та адміністратор. Встановлені правила формують чітке розмежування прав і охоплюють повний життєвий цикл роботи з платформою. Детальний перелік можливостей для кожної ролі наведено у таблиці 1.2.

Таблиця 1.2.

Функціональні вимоги до системи за ролями користувачів

Роль користувача		Функціональні можливості у системі		
Гість (неавторизований користувач)		Перегляд відкритого каталогу опублікованих сайтів, створених іншими користувачами платформи. Перегляд сторінок та контенту на опублікованих сайтах, взаємодія з їх публічними елементами. Ознайомлення з правилами платформи. Реєстрація нового облікового запису в системі, автентифікація за допомогою електронної пошти та		

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		23

Продовження таблиці 1.2.

	пароля, а також ініціація процедури безпечного відновлення доступу через підтвердження електронної пошти.
Користувач (авторизований власник)	Управління власним профілем на особистими налаштуваннями. Створення, редагування, копіювання та видалення власних вебсайтів. Взаємодія з візуальним редактором: конструювання структури сторінок через додавання функціональних блоків та налаштування їх компонентів. Збереження конфігурації у форматі JSON у базі даних. Адаптивний перегляд макетів для різних пристроїв. Завантаження та управління власними медіафайлами. Використання базового модуля електронної комерції. Базове налаштування SEO. Публікація створених сайтів у відкритий доступ та збереження власних шаблонів. Створення запитів до служби підтримки та подання апеляцій щодо заблокованих ресурсів.
Модератор	Перегляд загального списку зареєстрованих користувачів, їхнього поточного статусу та переліку проєктів. Тимчасове блокування або розблокування доступу до сайтів порушників правил платформи, а також переведення сайтів у тіньовий бан. Винесення попереджень користувачам щодо порушення контентної політики. Видалення облікових записів порушників та пов'язаних із ними проєктів. Перегляд, взяття в обробку та надання відповідей на тікети служби підтримки. Створення глобальних шаблонів для всіх користувачів платформи та перегляд базової аналітики активності.
Адміністратор	Повний контроль над програмним комплексом, що включає всі права модератора. Доступ до повної адміністративної панелі керування платформою. Управління обліковими записами модераторів. Доступ до розширеної аналітики системи та моніторингу транзакцій.

Нефункціональні вимоги

На відміну від функціональних вимог, нефункціональні описують атрибути якості програмної системи, тобто те, як добре та швидко вона повинна виконувати свої функції [10]. Для забезпечення високого рівня якості розробленої NoCode платформи визначено чіткі інженерні метрики.

Вимоги до продуктивності та швидкодії визначають швидкість реакції системи на дії користувача та загальну оптимізацію ресурсів. Час відгуку серверного API на стандартні запити користувачів не повинен перевищувати 200–300 мс. Оскільки візуальний редактор, побудований на базі React, є ресурсомістким клієнтським застосунком, перемальовування інтерфейсу під час активного використання технології Drag-and-Drop має відбуватися з частотою не менше 60 кадрів на секунду для забезпечення візуальної плавності. Також згенеровані сайти користувачів повинні проходити базові метрики Google Core

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		24

Web Vitals, що досягається за рахунок автоматичної оптимізації зображень через інтегрований сервіс Cloudinary та кешування статичного контенту на рівні CDN.

Вимоги до надійності та відмовостійкості спираються на обраний стек інфраструктурних рішень. Платформа повинна бути доступною цілодобово з цільовим показником експлуатаційної доступності 99,9%, що забезпечується хмарними сервісами Vercel та Render. База даних MySQL має підтримувати автоматичне резервне копіювання щонайменше один раз на добу. У разі виникнення критичної помилки на сервері, система повинна автоматично перезапускати процес за допомогою вбудованих механізмів платформи Render для мінімізації часу простою без втрати збережених даних.

Вимоги до безпеки є критично важливими через специфіку роботи з даними користувачів [11]. Весь обмін даними між клієнтом і сервером має відбуватися виключно по захищеному протоколу HTTPS із застосуванням SSL/TLS-сертифікатів. Паролі користувачів заборонено зберігати у відкритому вигляді; вони підлягають обов'язковому хешуванню на стороні сервера. Для захисту від XSS-атак система повинна автоматично санітизувати введені користувачами дані, унеможливлючи виконання шкідливого JavaScript-коду. Усі публічні форми також повинні бути захищені від автоматизованого перебору та спам-реєстрацій за допомогою інструментів верифікації.

1.3 Вибір і обґрунтування архітектурного рішення

Ефективність роботи платформи візуального конструювання критично залежить від результатів аналізу методів зберігання динамічних структур даних та правильного вибору СУБД. Архітектурний виклик полягає у необхідності одночасно забезпечувати транзакційну цілісність для облікових даних користувачів та максимальну гнучкість для збереження контенту створених вебресурсів. Оскільки структура сторінки формується користувачем динамічно, невідома заздалегідь кількість блоків, їх тип та вкладеність, традиційні підходи до проектування баз даних потребують перегляду. Під час проектування було проаналізовано три основні патерни зберігання таких даних.

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		25

Першим методом є використання патерну Entity-Attribute-Value у реляційних базах [12]. Історично для зберігання динамічних атрибутів у SQL-базах використовувався саме він. Згідно з цим підходом, створюються три таблиці: Сутність, Атрибут та Значення. Незважаючи на гнучкість, цей підхід є антипатерном для систем рівня NoCode. Головним його недоліком є катастрофічне падіння продуктивності при читанні даних. Щоб завантажити лише одну сторінку сайту, яка містить 50 блоків, кожен з яких має по 10 налаштувань стилів, базі даних доведеться виконати сотні операцій JOIN для об'єднання таблиць. Це створює надмірне навантаження на процесор сервера баз даних і робить миттєвий рендеринг сторінки неможливим.

Другим методом виступають документо-орієнтовані бази даних **NoSQL**. Альтернативним рішенням є використання таких баз, як MongoDB. У цій парадигмі всі дані сторінки зберігаються у вигляді єдиного BSON/JSON документа. Це ідеально підходить для дерева блоків візуального редактора, де вся конфігурація сторінки завантажується за один надшвидкий запит без жодних об'єднань. Однак використання виключно NoSQL підходу створює інші проблеми для платформи. Оскільки система передбачає модуль електронної комерції, необхідна підтримка ACID-транзакцій. Забезпечення цілісності складних зв'язків у нереляційних базах вимагає написання складної програмної логіки на рівні бекенду, що підвищує ризик виникнення аномалій даних.

Третім і найбільш оптимальним варіантом став гібридний підхід – MySQL з підтримкою JSON. Для вирішення описаного архітектурного компромісу у розроблюваній системі застосовано саме його. В якості основної системи управління базами даних обрано реляційну СУБД MySQL останніх версій [13]. Вона гарантує цілісність критичних бізнес-даних через класичні таблиці зі строгими зовнішніми ключами. Водночас, завдяки нативній підтримці формату JSON, конфігурація сторінок та ієрархічне дерево візуальних блоків зберігаються у вигляді єдиного документа в межах реляційної таблиці. Візуалізацію обраного архітектурного рішення, яке поєднує надійність реляційних зв'язків та гнучкість NoSQL-документів, наведено на рисунку 1.6.

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		26

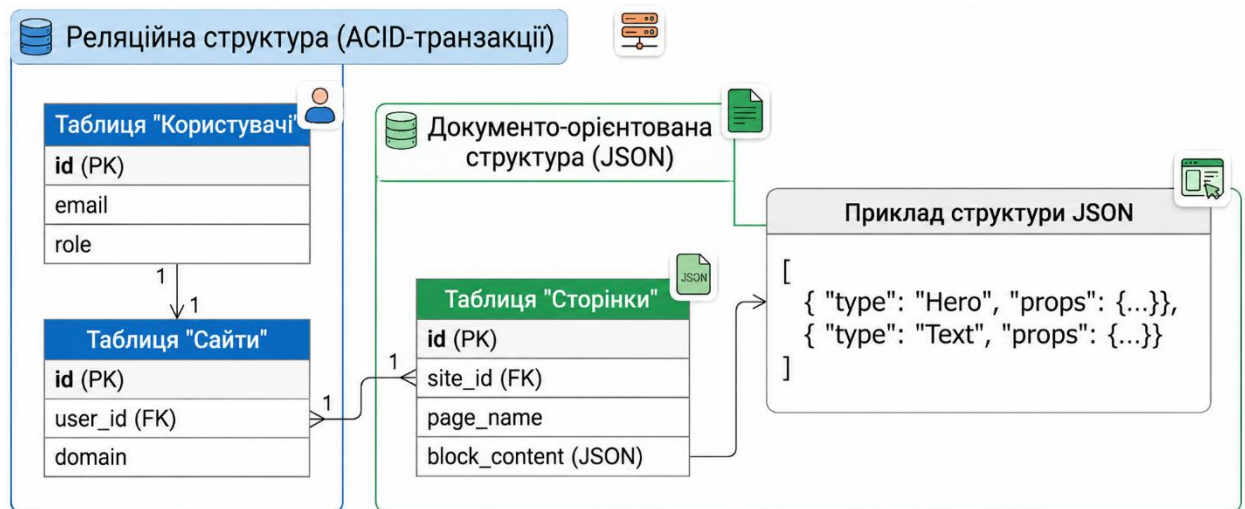


Рис. 1.6. Архітектурна діаграма гібридної моделі зберігання даних

Починаючи з версії 5.7.8, MySQL отримала нативну підтримку типу даних JSON. Це дозволяє зберігати ієрархію сторінки, дерево блоків візуального редактора, у спеціалізованій колонці `block_content` типу JSON всередині таблиці `pages`.

Запропоноване архітектурне рішення суттєво підвищує загальну швидкість платформи. Оскільки вся ієрархія сторінки зберігається в одному полі, сервер отримує готову структуру для рендерингу з усіма вкладеними блоками та їхніми налаштуваннями за допомогою лише одного простого SQL-запиту: `SELECT block_content FROM pages WHERE id = ?`. Водночас такий підхід значно спрощує подальший розвиток системи. Додавання нових функцій чи зміна параметрів блоків не вимагає міграцій та зміни структури бази даних – схема залишається стабільною, адже оновлюється лише внутрішнє наповнення JSON-документа.

Крім того, сучасні можливості рушія MySQL дозволяють індексувати віртуальні колонки, згенеровані безпосередньо з JSON-даних. Це розв'язує проблему глибокого пошуку, даючи змогу системі швидко знаходити специфічну інформацію всередині конфігурацій блоків.

У результаті, використання реляційної моделі MySQL у комбінації з нативною підтримкою JSON забезпечує платформі гнучкість документо-орієнтованих баз даних, зберігаючи при цьому всі переваги надійної ACID-

сумісної архітектури. Це робить такий гібридний підхід оптимальним вибором для систем шаблонного конструювання вебресурсів.

Інфраструктура та хмарні сервіси. Як платформу для хостингу клієнтської частини, що складається з React + Vite було обрано Vercel. Цей сервіс оптимізований для фронтенд-фреймворків. Його головною перевагою є вбудована система безперервної інтеграції та розгортання – CI/CD. При надсиланні оновлень у репозиторій, Vercel автоматично збирає проєкт і оновлює вебсайт. Крім того, платформа роздає статичні файли через власну глобальну мережу доставки контенту – CDN, що забезпечує миттєве завантаження візуального редактора у користувачів з будь-якої точки світу.

Для розгортання серверної частини, що представлена Node.js, використовується хмарна платформа Render. Вона дозволяє розгортати вебсервіси безпосередньо з репозиторію програмного коду. Render автоматично налаштовує безпечне HTTPS-з'єднання, надає зручну панель для моніторингу логів сервера в реальному часі та автоматично перезапускає сервер у разі виникнення критичних помилок, що зводить час простою до мінімуму.

В якості хмарного провайдера для СУБД MySQL обрано платформу Aiven, яка функціонує за моделлю керованої бази даних – Database-as-a-Service. Такий інженерно-прагматичний підхід дозволяє уникнути витрат часу на ручне конфігурування бази даних, налаштування фаєрволів та систем резервного копіювання на Linux-сервері. Платформа Aiven бере на себе автоматичне резервне копіювання, шифрування даних у стані спокою та забезпечення високої доступності.

Для управління медіафайлами інтегровано хмарний сервіс Cloudinary. Оскільки платформа дозволяє користувачам завантажувати власні зображення для конструювання сайтів, зберігати їх локально на серверах Render або безпосередньо в базі даних є вкрай неефективним підходом. Cloudinary бере на себе автоматичну оптимізацію завантажених зображень, їх кадрування, швидку конвертацію у вебформати та доставку через власну мережу CDN. Це критично

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		28

важливо для забезпечення швидкодії та оптимізації завантаження створених вебресурсів.

Важливою інфраструктурною вимогою для повноцінного функціонування платформи як комерційного продукту є наявність власного доменного імені. Воно необхідне не лише для ідентифікації бренду, але й для коректного налаштування політик спільного використання ресурсів між різними джерелами та роботи SSL-сертифікатів між клієнтською та серверною частинами. В якості реєстратора доменних імен обрано сервіс Namecheap через його надійність та зручні інструменти адміністрування. Визначальним фактором вибору стала безперешкодна можливість делегування управління DNS-зоною шляхом зміни NS-серверів на інфраструктуру Cloudflare.

Система мережевої безпеки та DNS-провайдер Cloudflare відіграє критичну роль у захисті застосунку. Оскільки платформа NoCode за своєю природою може бути мішенню для парсерів та зловмисних ботів, Cloudflare виступає зворотним проксі-сервером, який приховує реальні IP-адреси серверів Render та Aiven. Цей сервіс забезпечує базовий захист від DDoS-атак на мережевому L3/L4 та прикладному L7 рівнях, а також безкоштовно надає SSL/TLS сертифікати для безпечної передачі даних.

Додатково, для забезпечення безпеки підсистеми автентифікації та захисту публічних форм платформи від автоматизованих скриптів інтегровано систему CAPTCHA Cloudflare Turnstile. Будь-який відкритий вебресурс із можливістю реєстрації піддається ризикам атак типу Brute-force та масових спам-реєстрацій. Використання цього інструменту дозволяє верифікувати легітимність запитів ще на етапі взаємодії з клієнтським застосунком. Це суттєво знижує паразитне навантаження на серверну частину та базу даних, блокуючи зловмисний трафік ще до моменту обробки бізнес-логіки.

1.4 Вибір інструментальних засобів для реалізації

Вибір інструментальних засобів є одним із найважливіших етапів проєктування, який безпосередньо впливає на швидкість розробки,

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		29

продуктивність системи, її масштабованість та зручність подальшої підтримки. У цьому підрозділі послідовно розглянуто та обґрунтовано вибір технологій для клієнтської і серверної частин платформи, допоміжних середовищ розробки, а також визначено вимоги до апаратного забезпечення.

Для реалізації проєкту було обрано технологічний стек, що повністю базується на екосистемі мови JavaScript [14]. Такий підхід дозволяє використовувати єдину мову як для фронтенду, так і для бекенду, що суттєво зменшує час на розробку, спрощує інтеграцію між підсистемами та дає змогу ефективно перевикористовувати спільний програмний код.

Клієнтська частина системи є найбільш складною складовою платформи, оскільки саме вона реалізує логіку візуального редактора, що працює безпосередньо у браузері користувача. В якості основної бібліотеки для побудови користувацького інтерфейсу було обрано React. Цей вибір є ідеальним для розробки NoCode-рішень з кількох вагомих причин.

По-перше, React базується на компонентному підході. Це ідеально узгоджується з концепцією блокового редактора. Кожен елемент майбутнього сайту, від простої кнопки до повноцінної секції, інкапсулюється в окремий ізольований компонент зі своїм внутрішнім станом та стилями, що значно спрощує масштабування бібліотеки блоків платформи.

По-друге, ключовою перевагою React є оптимізація рендерингу за рахунок використання віртуальної об'єктної моделі документа (Virtual DOM). У системах візуального конструювання, де користувач постійно перетягує блоки та змінює їхні параметри, традиційні маніпуляції з реальним DOM є надзвичайно ресурсомісткими. Завдяки алгоритмам React, при переміщенні або зміні властивостей об'єкта, система швидко обчислює різницю між станами і точково перемальовує лише ті вузли, які дійсно змінилися. Це забезпечує плавність інтерфейсу навіть при роботі зі складними та об'ємними макетами сторінок. Крім того, екосистема бібліотеки дозволяє органічно інтегрувати сучасні менеджери глобального стану для ефективного управління масивами даних (JSON-конфігураціями сайтів) без складного багаторівневого передавання параметрів.

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		30

В якості інструменту збірки клієнтського застосунку замість традиційних бандлерів було обрано Vite [15]. Головним аргументом на його користь є миттєвий запуск локального сервера та надзвичайно швидка гаряча заміна модулів (Hot Module Replacement), що досягається завдяки використанню нативних ES-модулів безпосередньо у браузері.

Для реалізації складної механіки перетягування елементів у візуальному редакторі обрано спеціалізовану бібліотеку React DnD [16] у поєднанні з HTML5 Backend [17]. Вона надає гнучкий API для розробки взаємодій між елементами і забезпечує стабільну роботу при маніпуляціях із глибоко вкладеними деревами компонентів.

Додатково, для оптимізації процесу верстки та забезпечення сучасного користувацького досвіду, у проєкті інтегровано фреймворк Tailwind CSS [18] для швидкої утилітарної стилізації та бібліотеку Framer Motion [19] для реалізації плавних інтерактивних анімацій. Для формування єдиної візуальної стилістики елементів управління та оптимізації графічних ресурсів застосунку базовим набором векторних іконок було обрано бібліотеку Lucide React [20].

Серверна частина відповідає за обробку API-запитів, виконання бізнес-логіки системи та взаємодію з базою даних. В якості програмної платформи було обрано середовище Node.js [21]. Його використання є повністю виправданим для розроблюваної платформи через асинхронну, подієво-орієнтовану модель. Оскільки NoCode система передбачає одночасну роботу багатьох користувачів, які постійно надсилають на сервер великі масиви даних для збереження, бекенд повинен ефективно витримувати високе навантаження. Асинхронна природа Node.js дозволяє серверу не блокувати потік виконання під час очікування запису в базу даних. В якості вебфреймворку використовується Express.js — гнучкий інструмент, який забезпечує швидку маршрутизацію запитів, побудову архітектури REST API та легку інтеграцію проміжного програмного забезпечення.

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		31

Для забезпечення високої продуктивності процесу написання коду та проєктування архітектури системи було підібрано відповідний інструментарій розробника.

Основним інтегрованим середовищем виступає Visual Studio Code [22]. Цей вибір зумовлений його легковажністю, швидкістю роботи та широкою екосистемою розширень. Інструмент надає можливості для автодоповнення коду, інтеграцію з лінтерами для перевірки синтаксису, а також має вбудовану підтримку системи розподіленого контролю версій Git [23], що оптимізує рутинні процеси.

Для проєктування, розробки та адміністрування бази даних використовується MySQL Workbench. Цей інструмент надає зручні візуальні засоби для створення структур баз даних, генерації SQL-скриптів та побудови ER-діаграм, необхідних для документації зв'язків між сутностями.

Для моделювання логічної архітектури програмного забезпечення обрано інструмент PlantUML. Діаграми у ньому не малюються графічно, а описуються за допомогою спеціалізованої текстової мови. Це дозволяє зберігати архітектурні моделі у репозиторії Git разом із вихідним кодом проєкту, ефективно відстежувати історію їх змін та легко коригувати структуру без використання важких графічних пакетів.

Висновки до першого розділу

У першому розділі було проведено детальний аналіз предметної області та обґрунтовано актуальність створення NoCode платформи для конструювання вебресурсів. Дослідження існуючих на ринку аналогів дозволило виявити їхні сильні та слабкі сторони, а також сформувати унікальну нішу для розроблюваного продукту. Окрім цього, було визначено ключові сегменти цільової аудиторії та формалізовано базові сценарії використання системи: від швидкого запуску посадкових сторінок на піддомені платформи до прототипування з подальшим експортом чистого коду.

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		32

На основі проведеного аналізу сформовано вимоги до розроблюваної системи та обґрунтовано вибір технологічного стеку і методів зберігання даних. Доведено, що для забезпечення балансу між транзакційною цілісністю та гнучкістю контенту оптимальним є гібридний архітектурний підхід, який передбачає використання реляційної СУБД MySQL із нативною підтримкою формату JSON замість неефективних EAV-моделей чи чистих NoSQL-рішень. Також визначено єдину мову розробки JavaScript, де React застосовується для клієнтського інтерфейсу, Node.js для серверної бізнес-логіки, та обрано відповідну хмарну інфраструктуру.

Також у розділі було чітко визначено функціональні вимоги на основі моделі управління доступом, які розмежовують можливості гостей, користувачів, модераторів та адміністраторів. Сформульовано конкретні нефункціональні інженерні метрики, що стосуються швидкодії, безпеки, відмовостійкості та апаратного забезпечення. Зібрані вимоги, проаналізовані сценарії та обрані технології створюють міцну концептуальну основу для логічного та фізичного проектування архітектури програмної системи.

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		33

РОЗДІЛ 2. ПРОЄКТУВАННЯ ПРОГРАМНОЇ СИСТЕМИ

2.1 Моделювання варіантів використання та поведінки системи

Моделювання варіантів використання є одним із ключових етапів об'єктно-орієнтованого проєктування [24]. Воно дозволяє визначити межі програмного комплексу, ідентифікувати основних дійових осіб та формалізувати сценарії їхньої взаємодії з системою. Перетворення текстових функціональних вимог у візуальну архітектуру поведінки відображено на розширеній діаграмі варіантів використання платформи (рис. 2.1.).

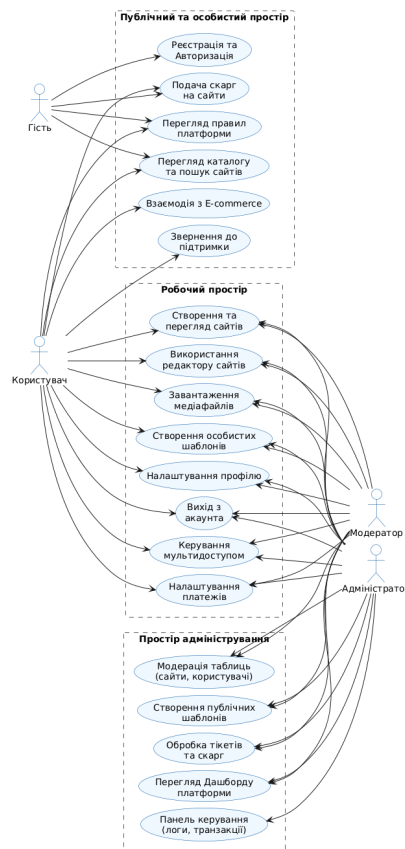


Рис. 2.1. Діаграма варіантів використання системи

На основі представленої моделі, яка використовує управління доступом на основі ролей, в архітектурі платформи виділено чотирьох незалежних акторів. Їхня детальна ідентифікація та зони відповідальності наведені у таблиці 2.1.

Таблиця 2.1.

Ідентифікація акторів системи

Актор	Опис ролі та зона відповідальності
Гість	Неавторизований відвідувач. Має базовий доступ до публічної частини платформи, може переглядати створені сайти та ініціювати реєстрацію.

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		34

Продовження таблиці 2.1.

Користувач	Авторизований користувач. Має доступ до робочого простору, конструктора сайтів, управління медіафайлами та модулем електронної комерції.
Модератор	Співробітник служби підтримки. Працює в адміністративному просторі, обробляє запити користувачів, слідкує за дотриманням правил платформи та модерює контент.
Адміністратор	Технічний керівник платформи. Має найвищий рівень доступу до всіх підсистем, включаючи панель керування платформою, логування дій та фінансові транзакції.

Зважаючи на масштаб платформи, усі варіанти використання системи логічно розділені на три функціональні зони: публічний простір, спільний робочий простір та простір адміністрування.

Функціонал публічного та особистого простору охоплює ключові процеси ідентифікації та базової взаємодії з платформою. Гості системи можуть переглядати загальні правила, ознайомлюватися з відкритим каталогом опублікованих сайтів та ініціювати процес реєстрації для створення власного облікового запису. Після авторизації Користувачам додатково надається можливість подавати скарги на розміщені ресурси у разі виявлення порушень, звертатися до служби підтримки шляхом створення тікетів або подачі апеляцій, а також повноцінно взаємодіяти з модулем електронної комерції, що включає використання кошика та оформлення замовлень на сайтах-магазинах.

Спільний робочий простір містить прецеденти, пов'язані з безпосереднім створенням та управлінням контентом. Цей функціонал доступний Користувачам для ведення їхніх проєктів, а також Модераторам і Адміністраторам для створення службових ресурсів чи макетів. Основними сценаріями тут є ініціалізація нових сайтів, зміна їхніх статусів та безпосередня взаємодія з візуальним Drag-and-Drop редактором. Крім того, актори мають змогу завантажувати медіафайли до персональної галереї, зберігати власні напрацювання у вигляді особистих шаблонів, налаштовувати параметри свого профілю і безпечно завершувати сесію шляхом виходу з акаунта.

Простір адміністрування призначений виключно для управління платформою та підтримки її стабільної роботи. Модератори та Адміністратори

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		35

здійснюють контроль за таблицями баз даних, керуючи статусами користувачів та їхніх вебсайтів. До їхньої відповідальності входить створення публічних шаблонів, обробка вхідних тікетів та скарг для розв'язання конфліктних ситуацій, а також перегляд аналітичного дашборду активності платформи. Ексклюзивний доступ до панелі керування має лише Адміністратор, який додатково контролює системні логи, управляє фінансовими транзакціями та має повноваження призначати або призупиняти права модераторів.

Візуалізація чітко розмежовує зони відповідальності: актори базового рівня взаємодіють переважно з публічною частиною та робочим простором, тоді як службові ролі мають доступ до адміністративних інструментів модерації та контролю. Розроблена модель слугує надійним фундаментом для подальшого проєктування об'єктної структури класів та бази даних.

Для більш детального розуміння логіки роботи системи та підготовки до проєктування алгоритмів, було розроблено специфікації для ключових прецедентів платформи. Специфікація деталізує кроки взаємодії актора з системою, визначає передумови та можливі альтернативні сценарії з обробкою помилок. Це дозволяє чітко сформулювати контракт між клієнтською частиною та сервером.

У таблиці 2.2. наведено специфікацію базового варіанта використання "Реєстрація та Авторизація", яка забезпечує безпечний доступ до робочого простору та управління сесіями користувачів.

Таблиця 2.2.

Специфікація прецеденту "Реєстрація та Авторизація"

Атрибут	Опис
Назва прецеденту	Реєстрація та Авторизація
Актор	Гість
Мета	Отримати доступ до закритого функціоналу створення, редагування сайтів та управління профілем.
Передумови	Гість знаходиться на публічній сторінці платформи, має стабільне інтернет-з'єднання.
Основний сценарій (Успішний)	1. Гість обирає опцію "Увійти" на навігаційній панелі. 2. Система відображає форму для вводу електронної пошти та пароля. 3. Гість вводить свої облікові дані та натискає "Увійти". 4. Клієнтський застосунок валідує формат введених даних.

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		36

Продовження таблиці 2.2.

Альтернативний сценарій (Помилки)	5. Сервер перевіряє наявність користувача в БД або створює нового. 6. Сервер генерує JWT-токен доступу та повертає його клієнту. 7. Система зберігає токен і перенаправляє користувача до ізольованого Робочого простору. 4а. Введено некоректний email або слабкий пароль: система блокує відправку запиту, виводить повідомлення про помилку валідації та просить повторити ввід. 5а. При реєстрації – Користувач з таким email вже існує: сервер повертає статус 409 Conflict, виводить повідомлення про конфлікт. 5б. При вході – Невірний пароль або email: сервер повертає статус 401 Unauthorized, виводиться повідомлення про помилку без уточнення на конкретне поле.
Постумови	Гість набуває ролі "Користувач". У системі створюється активна авторизована сесія.

Основним інноваційним функціоналом платформи є маніпуляція DOM-деревом через візуальний інтерфейс. У таблиці 2.3. описано інженерну специфікацію прецеденту "Використання редактору сайтів", який є найскладнішим компонентом клієнтської частини.

Таблиця 2.3.

Специфікація прецеденту "Використання редактору сайтів"

Атрибут	Опис
Назва прецеденту	Використання редактору сайтів
Актор	Користувач
Мета	Створити або змінити структуру та зовнішній вигляд власного вебсайту без безпосереднього написання вихідного коду.
Передумови	Користувач авторизований у системі та відкрив існуючий або ініціалізував новий проєкт.
Основний сценарій (Успішний)	1. Користувач відкриває маршрут візуального редактора. 2. Система виконує HTTP GET запит, завантажує поточну JSON-конфігурацію сайту з бази даних та рендерить ієрархію компонентів на екрані. 3. Користувач обирає необхідний функціональний блок з бічної панелі інструментів. 4. За допомогою Drag-and-Drop користувач перетягує блок у робочу область; система розраховує місце вставки. 5. Користувач виділяє доданий блок та змінює його параметри через панель налаштувань. 6. Користувач натискає кнопку "Зберегти". 7. Система серіалізує поточний стан віртуального DOM у формат JSON і відправляє PUT запит на Backend API. 8. Сервер валідує дані і успішно оновлює відповідний запис у базі даних.
Альтернативний сценарій (Помилки)	7а. Втрата з'єднання з інтернетом: система перехоплює помилку мережі, сповіщає користувача про неможливість синхронізації та тимчасово кешує зміни локально у LocalStorage браузера.

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		37

Продовження таблиці 2.3.

	8а. Помилка валідації JSON на сервері: сервер відхиляє запит, система повідомляє про критичну помилку стану та пропонує перезавантажити сторінку до останньої стабільної версії.
Постумови	Оновлена JSON-структура ресурсу надійно збережена в базі даних та готова до компіляції або публікації.

Окрім клієнтського функціоналу, стабільність платформи підтримується адміністративними процесами. Специфікація прецеденту модерації (табл.2.4.) демонструє захисні механізми системи та зміну глобальних станів об'єктів.

Таблиця 2.4.

Специфікація прецеденту "Модерація таблиць"

Атрибут	Опис
Назва прецеденту	Модерація таблиць (Блокування сайту)
Актор	Модератор, Адміністратор
Мета	Обмежити публічний доступ до вебресурсу, що порушує контентну політику платформи (спам, шахрайство).
Передумови	Модератор авторизований у системі, знаходиться в адміністративній панелі керування та ідентифікував сайт-порушник.
Основний сценарій (Успішний)	1. Модератор знаходить необхідний сайт через систему фільтрації у таблиці всіх проєктів. 2. Модератор відкриває картку проєкту і натискає дію "Призупинити сайт". 3. Система вимагає обов'язкового введення причини блокування. 4. Модератор вводить причину та підтверджує дію. 5. Сервер виконує оновлення запису в базі даних, змінюючи статус сайту з published на suspended.
Альтернативний сценарій (Помилки)	6. Сервер автоматично генерує об'єкт "Попередження", прив'язуючи його до власника сайту. 7. Система відправляє автоматичний електронний лист власнику сайту із зазначенням причини блокування та інструкцією для подачі апеляції. 5а. Конкурентний доступ (Сайт вже видалено власником): сервер не знаходить активного запису в базі, система виводить помилку модератору та оновлює таблицю. 5б. Спроба модератора заблокувати сайт іншого модератора: система перевіряє рівні доступу та відхиляє операцію.
Постумови	Маршрутизатор платформи починає віддавати HTTP-статус 403 (доступ заборонено) при спробі модератора видалити ресурс іншого модератора або адміністратора.

Розроблені специфікації прецедентів визначають архітектурні контракти системи. Вони демонструють, що кожна дія актора супроводжується серверною логікою: управлінням транзакціями, валідацією прав доступу, обробкою помилок мережі та синхронізацією розподілених баз даних. Ця деталізація є

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		38

вхідними даними для проєктування об'єктно-орієнтованої структури класів та маршрутів API.

2.2 Проєктування логічної структури бази даних

Вибір оптимальної системи управління базами даних є критичним етапом проєктування, оскільки архітектура платформи поєднує вимоги до жорсткої транзакційної цілісності з необхідністю зберігання динамічних, неструктурованих даних. Під час аналізу розглядалися альтернативні класи баз даних, зокрема документо-орієнтовані та класичні реляційні СУБД. Хоча NoSQL-рішення ідеально підходять для збереження деревоподібних структур конфігурації вебсторінок, вони значно ускладнюють реалізацію чітких зв'язків для модулів електронної комерції, білінгу та управління правами доступу, де критично важливою є безперебійна підтримка властивостей ACID. Серед реляційних систем порівнювалися PostgreSQL та MySQL. Незважаючи на потужні можливості PostgreSQL, остаточний вибір було зроблено на користь MySQL версії 8.0. Ця СУБД демонструє високу продуктивність при операціях читання, що є типовим патерном навантаження для платформ конструкторів сайтів, де частота перегляду згенерованих сторінок значно перевищує частоту їх редагування. Крім того, восьма версія MySQL пропонує розширений та оптимізований інструментарій для роботи з типом даних JSON, який необхідний для реалізації ключового функціоналу.

Враховуючи специфіку NoCode платформи, де структура згенерованого сайту є динамічною і не має фіксованого набору полів, використання нативного типу даних JSON стало фундаментальним архітектурним рішенням. Такий гібридний підхід дозволяє ідеально поєднати надійність та передбачуваність строгих SQL-зв'язків для системних сутностей із гнучкістю NoSQL-документів для збереження користувацького контенту. Це усуває необхідність створення надлишкових зв'язаних таблиць для кожного окремого типу візуального блоку, що значно прискорює процеси вибірки даних з бази та подальшого рендерингу інтерфейсу у браузері.

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		39

Безпосередньо логічна структура бази даних платформи складається з понад 20 взаємопов'язаних таблиць, які комплексно покривають процеси управління користувачами, створення та налаштування сайтів, функціонування системи електронної комерції, роботу медіатеки та адміністративні функції. Для уникнення надлишковості інформації та надійної підтримки цілісності даних на рівні сховища, більшість таблиць реляційної моделі було нормалізовано до третьої нормальної форми. Загальну структуру основних сутностей бази даних, їхні атрибути, первинні та зовнішні ключі, а також логічні зв'язки між ними представлено у вигляді діаграми “сутність-зв'язок” на рис. 2.2., що формує єдиний та узгоджений інформаційний простір платформи.

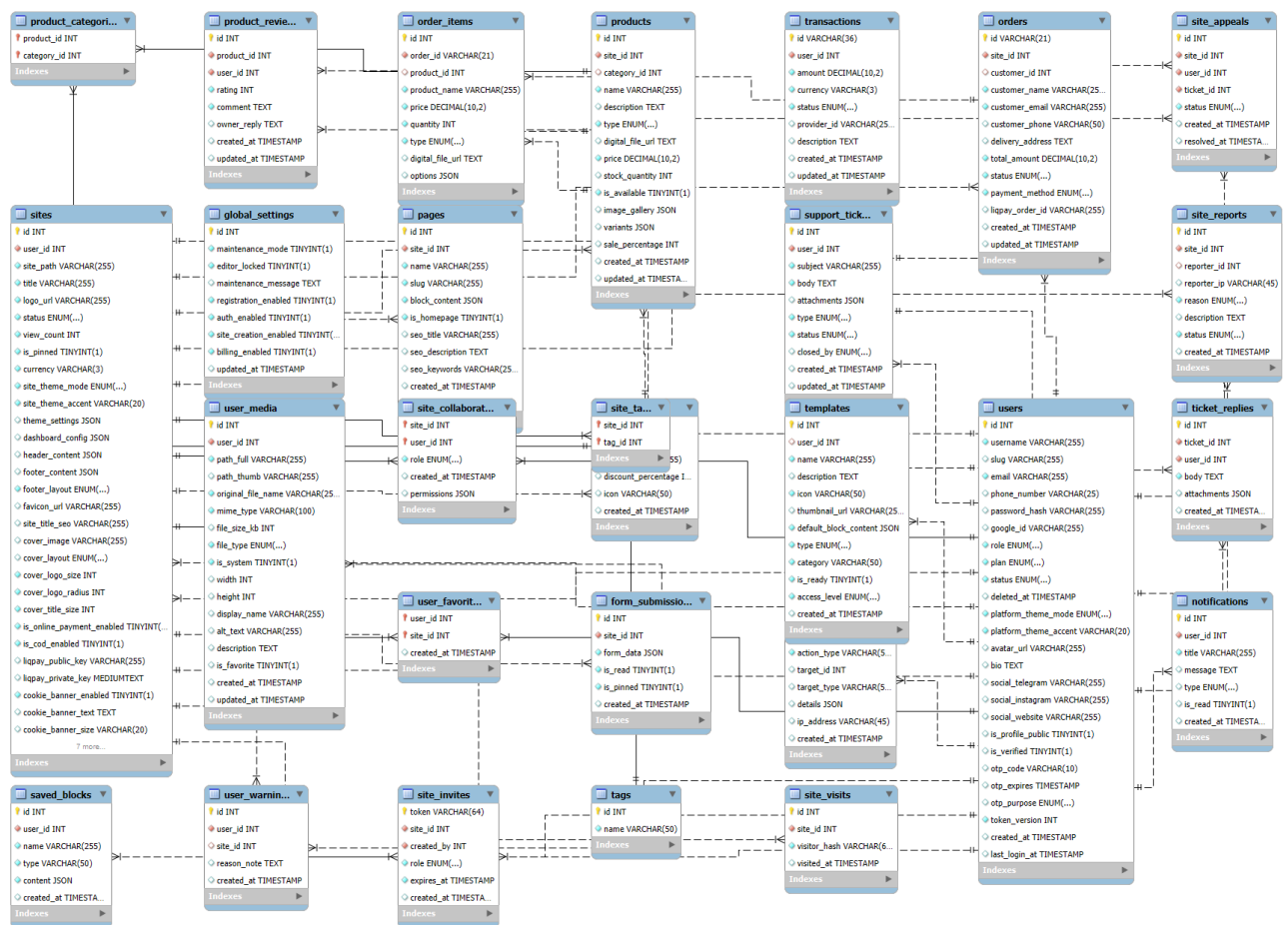


Рис. 2.2. ER-діаграма реляційної бази даних KendrDB

Архітектура бази даних базується на чітких правилах взаємодії між ключовими сутностями системи, що забезпечує структурну цілісність інформації. Базовим є зв'язок між користувачами та їхніми проєктами, який реалізовано як відношення один-до-багатьох. Один користувач має можливість

створювати безліч проєктів, проте кожен сайт прив'язаний лише до одного власника. Для підтримки чистоти даних на рівні бази налаштовано каскадне видалення: знищення облікового запису користувача ініціює автоматичне видалення всіх пов'язаних із ним сайтів. Своєю чергою, кожен проєкт має ієрархічну структуру сторінок, що реалізується зв'язком один-до-багатьох між сутностями sites та pages. При цьому маршрутизація всередині проєкту захищена від дублювання – комбінація ідентифікатора сайту та шляху сторінки є строго унікальною.

Для забезпечення додаткового функціоналу платформи реалізовано складніші типи зв'язків. Зокрема, організація публічного каталогу проєктів потребує гнучкої системи фільтрації, тому між таблицями sites та tags встановлено зв'язок багато-до-багатьох через проміжну таблицю site_tags, що дозволяє призначати сайтам кілька тегів одночасно. Важливим елементом структури також є модуль електронної комерції. У ньому замовлення та товари поєднані відношенням багато-до-багатьох за допомогою транзитної таблиці order_items. Таке архітектурне рішення не лише пов'язує замовлення з конкретними позиціями, але й виконує критично важливу бізнес-функцію, фіксує та зберігає історичну вартість товару на момент здійснення покупки, захищаючи історію транзакцій від майбутніх змін цін у каталозі.

Фізична структура БД. Для реалізації логічної моделі було створено фізичні таблиці. У таблицях 2.5. – 2.8. наведено специфікації найважливіших сутностей системи, що формують ядро платформи.

Таблиця 2.5.

Структура таблиці users

Назва поля	Тип даних	Обмеження	Опис
id	INT	PK, AUTO_INCREMENT	Ідентифікатор користувача.
username	VARCHAR(255)	UNIQUE, NOT NULL	Ім'я користувача в системі.
email	VARCHAR(255)	UNIQUE, NOT NULL	Електронна пошта.
password_hash	VARCHAR(255)	NULL	Хешований пароль.

Продовження таблиці 2.5

google_id	VARCHAR(255)	UNIQUE, NULL	Ідентифікатор для авторизації.
role	ENUM	NOT NULL, DEFAULT 'user'	Роль у системі
plan	ENUM	NOT NULL, DEFAULT 'FREE'	Тарифний план
status	ENUM	NOT NULL, DEFAULT 'active'	Стан акаунту
is_verified	BOOLEAN	DEFAULT FALSE	Підтвердження електронної пошти.
is_profile_public	BOOLEAN	DEFAULT TRUE	Прапорець публічності профілю користувача.
platform_theme_mode	ENUM	NOT NULL, DEFAULT 'system'	Налаштування теми інтерфейсу платформи.
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Дата та час реєстрації облікового запису.

Основною сутністю генератора є вебсайт. Таблиця sites (табл. 2.6.) має загальні налаштування проєкту, параметри SEO та ключі для прийому платежів.

Таблиця 2.6.

Структура таблиці sites

Назва поля	Тип даних	Обмеження	Опис
id	INT	PK, AUTO_INCREMENT	Ідентифікатор вебсайту
user_id	INT	FK, NOT NULL	Посилання на власника сайту
site_path	VARCHAR(255)	UNIQUE, NOT NULL	Унікальний субдомен/шлях сайту
title	VARCHAR(255)	NOT NULL	Назва проєкту
status	ENUM	NOT NULL, DEFAULT 'private'	Поточний статус сайту
theme_settings	JSON	NULL	Налаштування теми сайт
currency	VARCHAR(3)	NOT NULL, DEFAULT 'UAH'	Валюта магазину
is_online_payment_enabled	BOOLEAN	NOT NULL, DEFAULT 1	Активації системи онлайн-оплат на сайті
liqpay_public_key	VARCHAR(255)	NULL	Публічний ключ LiqPay
liqpay_private_key	MEDIUMTEXT	NULL	Приватний ключ LiqPay

Для збереження безпосереднього контенту, який користувач створює через Drag-and-Drop редактор, використовується таблиця pages (табл. 2.7.).

Таблиця 2.7.

Структура таблиці pages

Назва поля	Тип даних	Обмеження	Опис
id	INT	PK, AUTO_INCREMENT	Ідентифікатор сторінки
site_id	INT	FK, NOT NULL	Посилання на сайт

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		42

Продовження таблиці 2.7

name	VARCHAR(255)	NOT NULL	Системна назва сторінки
slug	VARCHAR(255)	NOT NULL	URL-шлях сторінки
block_content	JSON	NOT NULL	Масив компонентів сторінки, згенерованих у редакторі
is_homepage	BOOLEAN	NOT NULL, DEFAULT 0	Прапорець, що визначає сторінку як головну
seo_title	VARCHAR(255)	NULL	Заголовок для пошукової оптимізації
seo_description	TEXT	NULL	Опис сторінки для пошукової оптимізації
seo_description	TEXT	NULL	Опис сторінки для пошукової оптимізації
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Дата та час створення сторінки
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Дата та час створення сторінки
updated_at	TIMESTAMP	ON UPDATE CURRENT_TIMESTAMP	Дата та час останнього редагування контенту

Для забезпечення якісного обслуговування користувачів та модерації реалізовано підсистему тікетів. Структуру таблиці support_tickets наведено у таблиці support_tickets (табл. 2.8.).

Таблиця 2.8.

Структура таблиці support_tickets

Назва поля	Тип даних	Обмеження	Опис
id	INT	PK, AUTO_INCREMENT	Унікальний ідентифікатор звернення
user_id	INT	FK, NOT NULL	Ідентифікатор користувача, який створив тікет
subject	VARCHAR(255)	NOT NULL	Тема звернення
body	TEXT	NOT NULL	Текст проблеми або питання
attachments	JSON	NULL	Масив посилань на прикріплені файли
type	ENUM	NOT NULL, DEFAULT 'general'	Тип звернення
status	ENUM	NOT NULL, DEFAULT 'open'	Стан тікета.
closed_by	ENUM	NULL	Вказує, хто заклав тікет
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Дата та час створення звернення

Опис архітектури JSON-схеми сайту. Ключовим інноваційним та архітектурним рішенням платформи є оптимізований підхід до збереження

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		43

контенту користувацьких сторінок. Замість традиційного створення десятків відокремлених реляційних таблиць для кожного можливого елемента інтерфейсу, розроблена система використовує поле `block_content` у таблиці `pages`, яке зберігає структуру макета у нативному форматі JSON. Фізично ці дані представляють собою впорядкований масив об'єктів, де кожен окремий елемент описує конкретний функціональний блок на сторінці. Незважаючи на загальну гнучкість NoSQL-підходу до зберігання, кожен такий інформаційний вузол підпорядковується чіткій внутрішній схемі та обов'язково містить регламентований набір атрибутів.

Основою вузла є унікальний ідентифікатор `id`, який є алгоритмічно необхідним для коректної роботи механізмів узгодження у бібліотеці `React`, а також для точного відстеження переміщення елементів під час взаємодії з `Drag-and-Drop` інтерфейсом. Для визначення логіки відображення елемента використовується атрибут `type` – строкова константа, яка передається до фабрики рендерингу для динамічного виклику відповідного `React`-компонента. Зі свого боку, уся детальна конфігурація конкретного блоку інкапсулюється у вкладеному об'єкті `props`, який зберігає користувацьке текстове наповнення, URL-посилання на інтегровані медіафайли та індивідуальні параметри візуальної стилізації. Приклад фрагмента JSON-схеми, що зберігається в базі даних:

```
[
  {
    "id": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
    "type": "Hero",
    "props": {
      "title": "Ласкаво просимо до мого портфоліо",
      "subtitle": "Вебдизайнер та розробник",
      "layout": "centered",
      "backgroundColor": "#1a1a1a"
    }
  },
  {
    "id": "c9bf9e57-1685-4c89-bafb-ff5af830be8a",
    "type": "Text",
    "props": {
      "content": "Тут ви знайдете мої найкращі роботи...",
      "fontSize": 16,

```

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		44

```

    "alignment": "left"
  }
}
]

```

Такий підхід забезпечує максимальну продуктивність: під час завантаження сайту Backend виконує лише один SQL-запит до таблиці pages, отримує готовий JSON-масив і відправляє його на клієнт, де React миттєво відмальовує весь інтерфейс без необхідності додаткових обчислень на сервері – реалізація архітектури One-way data flow.

2.3 Проєктування компонентної структури та взаємодії модулів

Діаграма розгортання призначена для моделювання фізичної та інфраструктурної архітектури системи. За допомогою цієї діаграми можна оцінити конфігурацію серверних потужностей та хмарних сервісів, на яких працюватимуть окремі компоненти платформи, а також описати топологію їхньої взаємодії.

Оскільки система проєктується як сучасне SaaS-рішення, її архітектура відходить від традиційного монолітного розміщення на одному фізичному сервері на користь розподіленої хмарної інфраструктури. На рис. 2.3. представлена розроблена діаграма розгортання програмного комплексу.

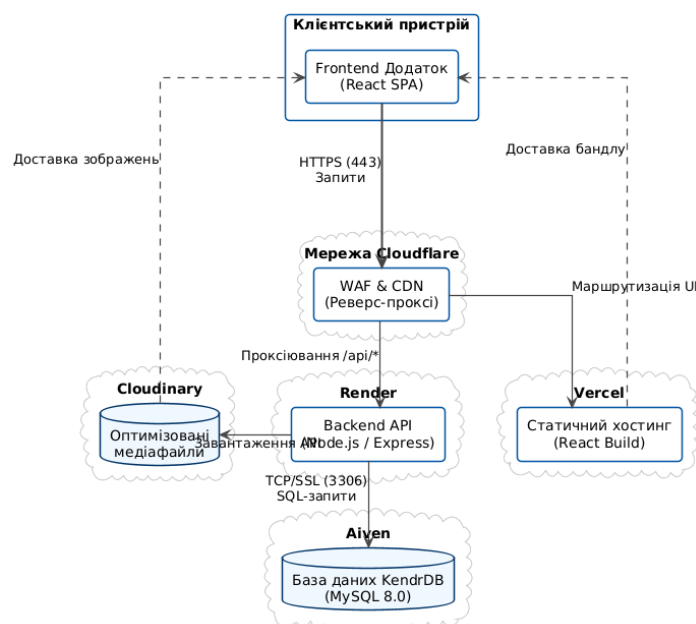


Рис. 2.3. Діаграма розгортання системи

Архітектура платформи побудована як сучасна розподілена хмарна система, що охоплює кілька взаємопов'язаних інфраструктурних вузлів. Взаємодія з системою починається з фізичного пристрою кінцевого користувача, де у веббраузері завантажується та виконується клієнтський застосунок. Цей вузол відповідає за безпосередній рендеринг візуального редактора, управління локальним станом та обробку всіх інтерактивних дій користувача.

Єдиною точкою входу для всього зовнішнього трафіку виступає мережа Cloudflare, яка працює як глобальний реверс-проксі сервер. Вона забезпечує базовий рівень безпеки, відповідаючи за управління SSL-сертифікатами, фільтрацію зловмисних запитів та захист від DDoS-атак. Окрім безпеки, Cloudflare виконує ключову роль у маршрутизації: він аналізує вхідний трафік і спрямовує запити на отримання інтерфейсу до хмарної платформи Vercel, а всі API-запити перенаправляє на сервер застосунків Render.

Платформа Vercel оптимізована для хостингу фронтенд-частини та виступає як сервер доставки статичної інформації. На ній зберігається скомпільований React-застосунок, який миттєво передається на клієнтські пристрої завдяки власній глобальній CDN. Своєю чергою, на хмарній платформі Render розгорнуто серверну частину системи, де виконується основна бізнес-логіка. У цьому середовищі працює Backend API на базі Node.js та Express.js. Цей вузол, здатний до автоматичного масштабування при зростанні навантаження, обробляє авторизацію, валідацію даних, модерацію та генерацію конфігурацій для створених сайтів.

Збереження даних розділено на два ізольовані потоки. Для структурованої інформації використовується керована база даних від провайдера Aiven, що працює за моделлю DBaaS. На цьому вузлі розгорнуто MySQL 8.0 Server, який обслуговує базу KendrDB. З міркувань безпеки доступ до неї строго обмежений: підключатися може виключно бекенд-сервер на Render. Водночас для роботи з медіафайлами замість традиційної локальної файлової системи інтегровано сервіс Cloudinary. Це спеціалізоване хмарне сховище повністю бере на себе

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		46

завантаження, оптимізацію та зберігання користувацьких аватарів, логотипів і графічного контенту проєктів.

Сценарій взаємодії в системі ініціюється Клієнтським пристроєм. Коли користувач вводить адресу платформи, запит проходить через фільтри безпеки Cloudflare і маршрутизується до Vercel, який повертає клієнту зібраний React-додаток.

Після завантаження інтерфейсу у браузері, React-додаток починає взаємодіяти із сервером для отримання або збереження даних. Він надсилає HTTP-запити за маршрутом /api/.... Cloudflare розпізнає ці API-запити і безпечно проксіює їх на Render.

Node.js-сервер приймає запит, виконує необхідну бізнес-логіку і надсилає SQL-запит до бази даних Aiven. Після отримання результатів від БД, сервер формує JSON-відповідь і повертає її клієнту.

Робота з файлами має окремий потік. Під час завантаження користувачем зображення, Backend відправляє файл до Cloudinary через захищене API, отримуючи натомість оптимізоване посилання. Надалі, під час перегляду створених сайтів, клієнтські браузери завантажують зображення напряму з глобальної CDN Cloudinary, повністю минаючи основний сервер платформи.

Така мікросервісна та хмарна архітектура гарантує високу відмовостійкість системи, швидку доставку контенту користувачам у будь-якій точці світу та безпеку зберігання даних.

Програмна реалізація системи базується на принципах модульності, слабкої зв'язності та розділення відповідальності. Для проєктування структури програмного коду було використано методології об'єктно-орієнтованого та компонентного програмування, а також класичні патерни проєктування.

Доменна модель описує ключові сутності бізнес-логіки та зв'язки між ними, абстрагуючись від конкретної реалізації у програмному коді. Діаграму класів предметної області без урахування архітектурних патернів подано на рис.2.4.

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		47

Основною сутністю предметної області є клас User, який представляє обліковий запис. Він має відношення композиції до класів Site, Media та SupportTicket, оскільки ці об'єкти фізично не можуть існувати без власника-користувача. Клас Site є агрегатором для контенту. Він пов'язаний композицією з класом Page, де ключовим атрибутом є block_content типу JSON – саме в ньому зберігається вся структура візуального редактора. Додатково до сайту можуть належати об'єкти класів Product та Category, які формують підсистему електронної комерції.

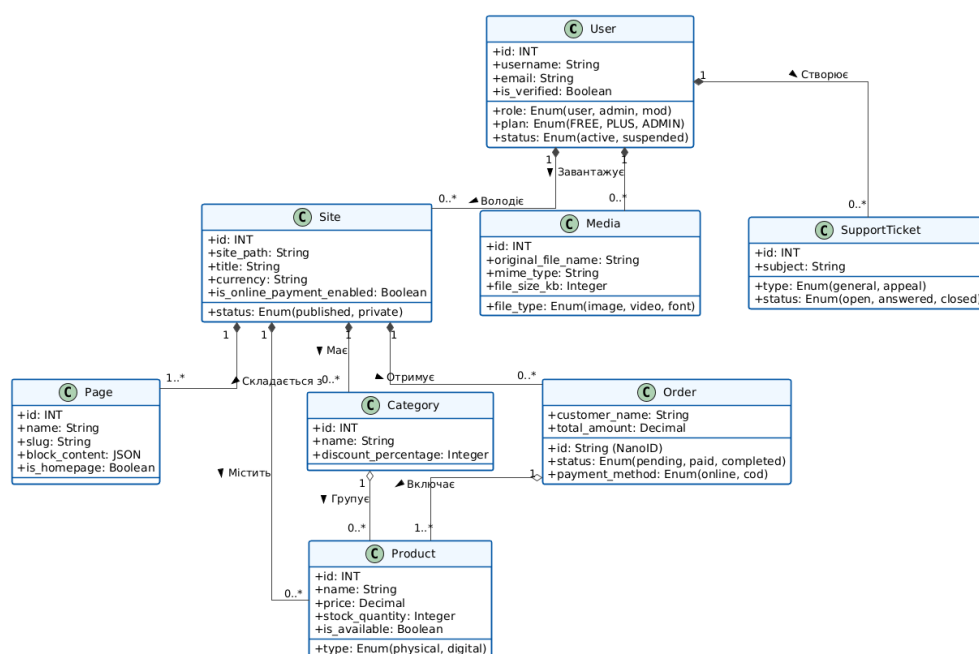


Рис. 2.4. Діаграма класів предметної області

Для забезпечення масштабованості коду, уникнення його дублювання та спрощення подальшої підтримки, розробка програмного комплексу спирається на комплексне використання сталих патернів проектування як на клієнтському, так і на серверному рівнях. Зокрема, на стороні фронтенд-застосунку імплементовано структурний патерн Decorator для ізоляції логіки захисту маршрутів, Factory Method для динамічної генерації React-компонентів у візуальному редакторі, а також Singleton для централізованого управління мережевими запитами. Серверна частина логічно розділена за архітектурою MVC, де потік управління проходить через проміжні шари перехоплювачів Middleware для верифікації JWT-токенів, перш ніж потрапити до контролерів і

моделей, при цьому пул з'єднань із базою даних контролюється єдиним Singleton-класом. Детальну структурну взаємодію зазначених компонентів та наочне проходження потоку даних від інтерфейсу користувача до кінцевого сховища відображає діаграма класів реалізації (рис. 2.5).

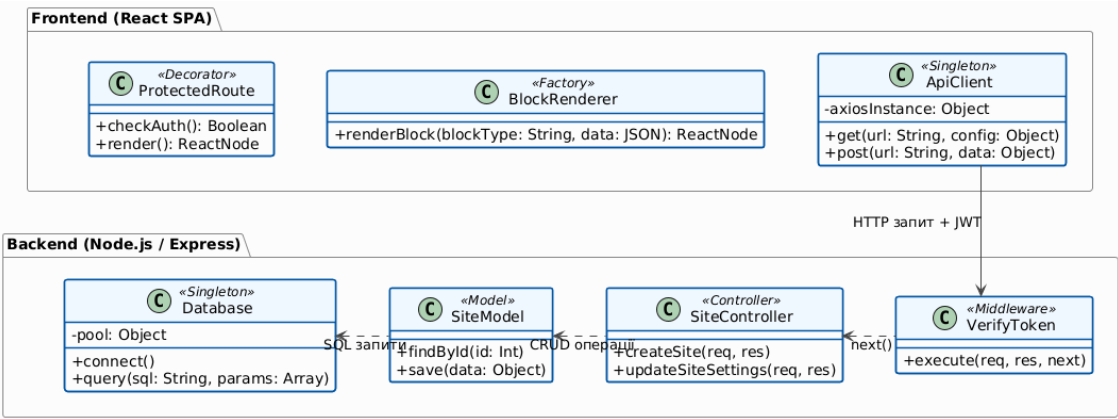


Рис. 2.5. Діаграма класів реалізації з використанням патернів

Програмна архітектура платформи базується на комплексному застосуванні перевірених шаблонів проєктування, що забезпечують її гнучкість, масштабованість та зручність підтримки. Фундаментальним підходом для організації серверної частини виступає класичний патерн Model-View-Controller. У межах цієї парадигми маршрутизатор делегує вхідні запити до контролера, який інкапсулює основну бізнес-логіку та звертається до моделі для виконання операцій з базою даних. При цьому роль відокремленого представлення у системі бере на себе клієнтський React-застосунок.

Для оптимізації роботи з глобальними конфігураціями та ресурсами широко застосовується патерн Singleton. На стороні клієнта цей підхід реалізовано через єдиний екземпляр конфігурації ApiClient для мережевих запитів. На сервері патерн використано для створення класу Database, який керує єдиним пулом з'єднань із MySQL на рівні файлу db.js. Це критично важливе рішення, яке ефективно запобігає створенню надлишкових підключень до бази даних при високому навантаженні.

Основою механіки візуального редактора є використання патерна Factory Method. Цю логіку реалізовано через спеціальний компонент BlockRenderer, який під час рендерингу аналізує тип блоку безпосередньо з конфігураційної

JSON-схеми та динамічно генерує відповідний React-компонент для його коректного відмальовування на сторінці.

Окрім структурних патернів, у системі імплементовано концепції Proxy та Decorator для забезпечення контролю доступу. На бекенді ця поведінка реалізована через проміжні обробники, які діють як перехоплювачі: вони зупиняють виконання запиту для верифікації JWT-токена перед тим, як дозволити доступ до захищеного контролера. На рівні клієнтського інтерфейсу аналогічну захисну функцію виконують компоненти вищого порядку, зокрема компонент ProtectedRoute. Він обгортає цільові компоненти та працює як проксі-об'єкт, автоматично блокуючи відображення закритих маршрутів системи для неавторизованих користувачів.

Компонентна структура клієнтської та серверної частин

Фізична структура програмного коду поділена на два ізольовані застосунки, взаємодія між якими відбувається виключно через REST API. Діаграму компонентів, що відображає файлову та модульну організацію проєкту, наведено нижче (рис. 2.6.).

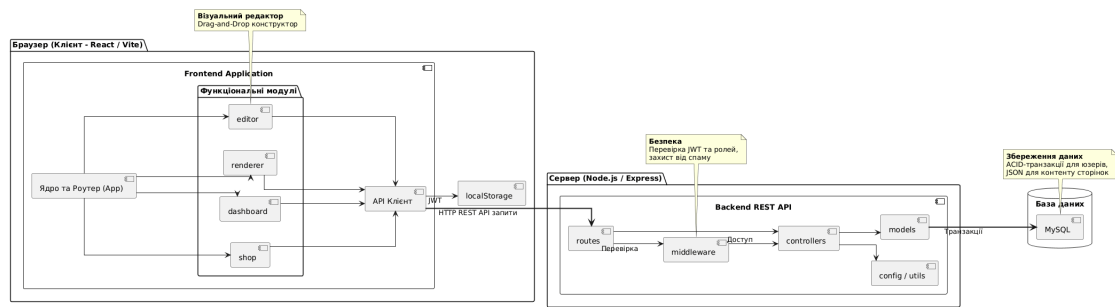


Рис. 2.6. Діаграма компонентів програмного комплексу

Серверна частина Node.js / Express побудована за модульним принципом. Папка routes відповідає за декларацію кінцевих точок API, направляючи трафік до controllers. Безпека забезпечується шаром middleware, який містить захисні екрани. Зв'язок із зовнішніми сервісами винесено у папку config.

Клієнтська частина платформи, побудована з використанням екосистеми React та інструменту збірки Vite, спроектована за принципом строгої модульної архітектури. Вся кодова база логічно розділена на три великі функціональні блоки, що забезпечує чистоту та ізольованість коду. Фундаментом виступає

директорія app, яка діє як ядро застосунку. Саме тут зосереджено ініціалізацію глобальних провайдерів контексту та базові налаштування маршрутизації системи [25].

Основна бізнес-логіка розподілена між незалежними фіче-модулями у директорії modules. Найскладнішим та найоб'ємнішим серед них є модуль editor, який повністю інкапсулює складну механіку Drag-and-Drop, логіку панелі налаштувань та роботу з бібліотекою візуальних блоків конструктора. Окрім безпосередньо редактора, цей розділ включає ізольовані модулі для панелі користувача, адміністративної частини та електронного магазину.

Третім критично важливим компонентом є директорія shared, що функціонує як спільний простір для всієї кодової бази. У ній акумульовано перевикористовувані компоненти: універсальні UI-елементи, допоміжні утиліти для генерації коду та кастомні React-хуки, які можуть вільно підключатися будь-яким іншим модулем. Така глибока та чітка декомпозиція проєкту гарантує високу підтримуваність коду, адже вона дозволяє розробникам незалежно тестувати, оновлювати та масштабувати окремі частини платформи, зводячи до мінімуму ризики пошкодити загальну бізнес-логіку системи під час рефакторингу чи додавання нового функціоналу.

Проектування взаємодії компонентів та програмних інтерфейсів

Взаємодія між клієнтським застосунком та серверною частиною платформи базується на архітектурному стилі REST API. Обмін інформацією відбувається через протокол HTTP з використанням легкого формату даних JSON, що забезпечує швидку серіалізацію та десеріалізацію об'єктів. Клієнтська частина ініціює асинхронні запити за допомогою ізольованих мережевих сервісів[26], які абстрагують логіку підключення та автоматично додають необхідні заголовки, зокрема JWT-токени для підтвердження авторизації. Серверна частина обробляє ці запити без збереження стану сесії, покладаючись на інформацію, передану в самому запиті.

Життєвий цикл обробки запиту на сервері проходить через чітко визначені архітектурні шари. Після маршрутизації трафіку вхідні дані проходять через

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		51

ланцюжок проміжних обробників, які виконують функції захисного екрана: валідують корисне навантаження, перевіряють права доступу та відхиляють некоректні запити. Тільки після успішної перевірки управління передається відповідному контролеру. Контролер виконує закладену бізнес-логіку, формує SQL-запити до бази даних через абстракцію моделі, обробляє результат і загортає його у стандартизовану HTTP-відповідь із відповідним статус-кодом.

Для деталізації та візуалізації процесів обміну даними між клієнтською та серверною частинами платформи розроблено відповідні UML-діаграми. Найкращим інструментом для ілюстрації життєвого циклу запитів та передачі управління є діаграми послідовності. Вони дозволяють чітко відобразити хронологічний порядок виклику методів та обміну повідомленнями між об'єктами системи під час виконання конкретного сценарію. Наприклад, на рис.2.7. зображено діаграму послідовності процесу створення нового вебсайту, що ілюструє повний шлях даних від клієнтського інтерфейсу до збереження запису в базі даних та повернення підтвердження користувачу.

Цей вид візуальних моделей дозволяє відобразити хронологічний порядок виклику методів та передачі повідомлень між екземплярами класів під час виконання конкретного сценарію.

На рис. 2.7. зображено діаграму послідовності процесу створення нового вебсайту, який є базовим функціоналом платформи.

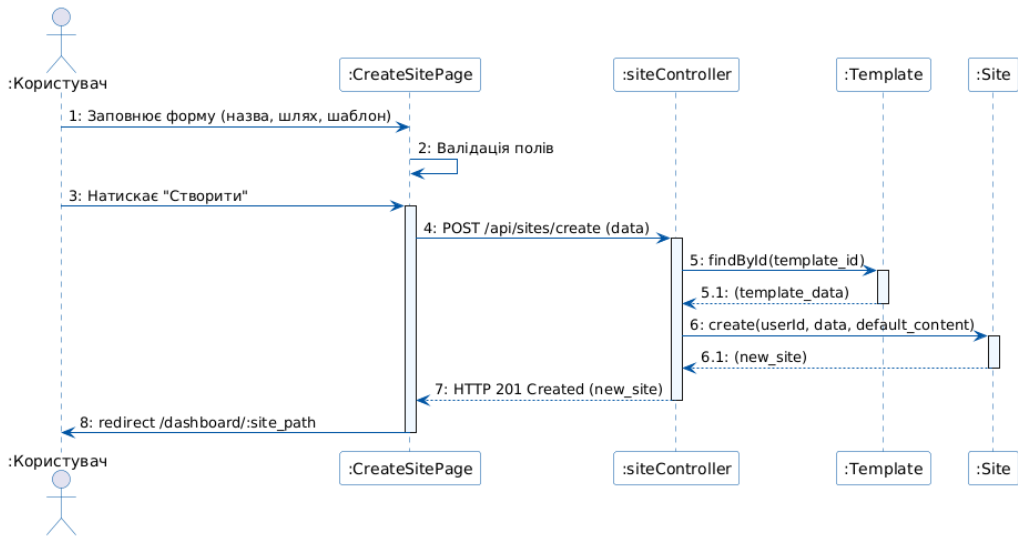


Рис. 2.7. Діаграма процесу створення вебсайту

Взаємодія між об'єктами під час створення проєкту розпочинається з того, що користувач заповнює форму базовими даними через клієнтський інтерфейс CreateSitePage. Одразу після цього компонент виконує синхронну валідацію введених полів. Підтвердивши коректність інформації, ініціатор натискає кнопку створення, що запускає процес відправки даних. На цьому етапі клієнт формує асинхронний HTTP POST запит на маршрут /api/sites/create та передає JSON-пакет з інформацією.

Маршрутизатор обробляє вхідний запит і делегує його до siteController. Далі контролер звертається до моделі TemplateModel для отримання базового JSON-контенту обраного шаблону. Після завантаження контенту шаблону викликається метод create() моделі SiteModel, куди передаються ідентифікатор користувача, дані форми та дефолтна структура. Успішне виконання відповідного SQL-запиту супроводжується формуванням серверної відповіді зі статусом 201 Created, і об'єкт нового ресурсу повертається клієнту. На завершальному етапі клієнтський застосунок успішно завершує транзакцію та виконує автоматичне перенаправлення користувача до робочого простору створеного сайту.

На рис. 2.8. подано діаграму послідовності створення звернення до служби підтримки.

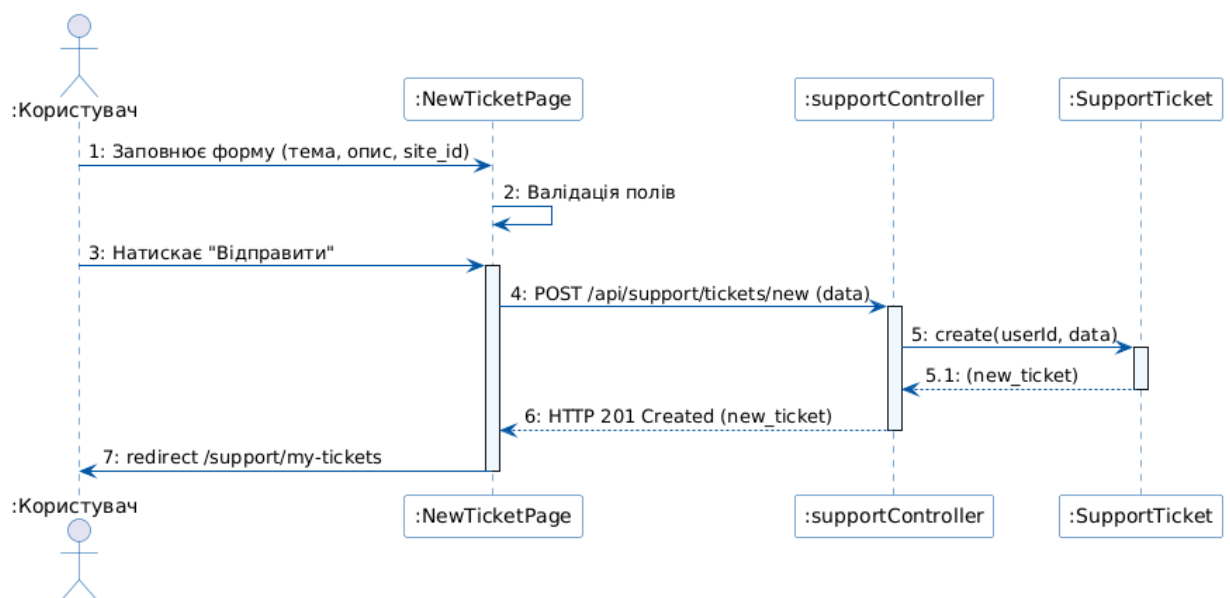


Рис. 2.8. Діаграма процесу створення звернення до служби підтримки

Логіка обміну даними при формуванні тікету підтримки розпочинається з того, що користувач вводить деталі проблеми у компоненті NewTicketPage. Одразу після цього інтерфейс здійснює локальну перевірку наявності обов'язкових полів. Коли користувач підтверджує відправку форми, клієнт ініціює HTTP POST запит на маршрут /api/support/tickets/new, передаючи корисне навантаження до supportController. Далі контролер звертається до моделі SupportTicket, викликаючи метод create(). Після успішного збереження запису в базу даних контролер повертає статус 201 Created. На завершальному етапі клієнтський маршрутизатор автоматично перенаправляє користувача на сторінку зі списком активних тікетів.

На рис. 2.9. ілюструється бізнес-транзакція – процес призупинення сайту адміністратором платформи, який містить розгалуження та умовні виклики.

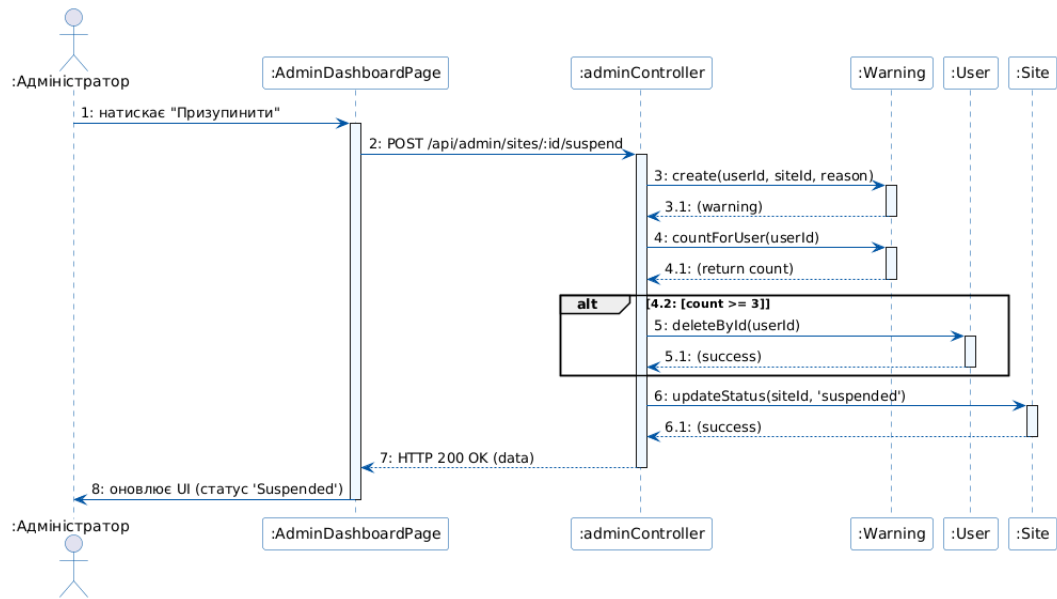


Рис. 2.9. Діаграма процесу призупинення сайту адміністратором

Хронологія адміністративної модерації починається з ініціації адміністратором блокування сайту-порушника через компонент AdminDashboardPage. Після цього відправляється HTTP POST запит на захищений маршрут /api/admin/sites/:id/suspend до adminController. Контролер першочергово звертається до WarningModel, використовуючи метод create(), для фіксації попередження в профілі порушника. Далі виконується повторний запит до цієї ж моделі через countForUser(), щоб отримати загальну кількість активних

попереджень користувача. На цьому етапі відбувається умовне розгалуження (alt): якщо кількість попереджень досягає критичної межі ($\text{count} \geq 3$), контролер звертається до `UserModel` та запускає каскадне видалення облікового запису за допомогою `deleteById()`. Незалежно від результату попереднього кроку, контролер взаємодіє зі `SiteModel` та виконує оновлення статусу ресурсу через `updateStatus()`. Завершивши ці операції, він повертає клієнту успішний статус 200 OK. У фіналі інтерфейс адміністратора синхронізується з сервером, візуально позначаючи відповідний сайт статусом "Suspended".

Для глибшого розуміння структурних зв'язків між об'єктами під час складних транзакцій використовується діаграма кооперації. Вона відображає ті самі повідомлення, що й діаграма послідовності, проте демонструє не часовий порядок взаємодій, а архітектуру та топологію зв'язків між компонентами. Просторову взаємодію базових компонентів під час виконання адміністративної процедури блокування ресурсу наведено на рис. 2.10.

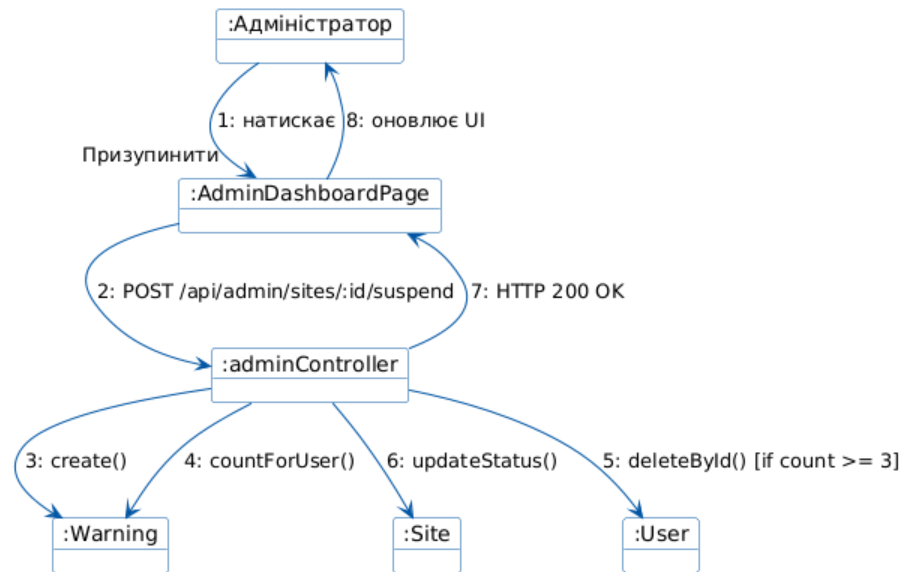


Рис. 2.10. Діаграма процесу призупинення сайту

Діаграма демонструє радіальну топологію управління: `adminController` виступає центральним вузлом, який ізолює клієнтський інтерфейс від безпосередньої взаємодії з шаром даних. Контролер бере на себе відповідальність за координацію трьох різних моделей, забезпечуючи виконання принципу єдиної відповідальності та транзакційної цілісності операції модерації.

2.4 Розробка основних алгоритмів роботи платформи

Для детального розуміння внутрішньої логіки платформи розроблено алгоритми роботи ключових бізнес-процесів. Візуалізація алгоритмів здійснена за допомогою діаграм активності. Ці діаграми відображають динаміку поведінки системи, показуючи паралельні процеси, розгалуження та чіткий розподіл зон відповідальності між Клієнтом та Сервером за допомогою доріжок.

Алгоритм процесу "Створення сайту користувачем" описує один із найважливіших процесів платформи – ініціалізацію нового вебресурсу на базі готового шаблону. Динаміку взаємодії користувача з системою, паралельну обробку даних та перевірку унікальності під час цього процесу наочно відображено на рис. 2.11.

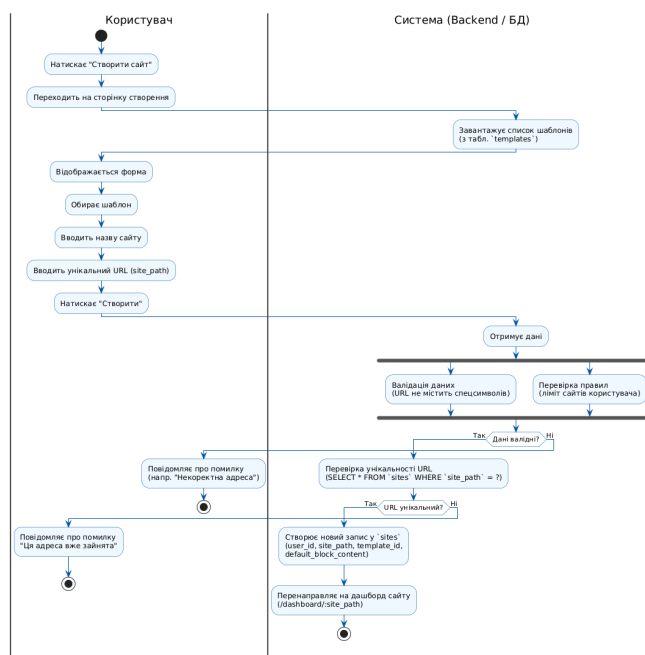


Рис. 2.11. Діаграма процесу створення сайту

Процес створення нового вебсайту в системі формалізовано у вигляді чіткого алгоритму, що охоплює кілька послідовних етапів взаємодії між клієнтським інтерфейсом та сервером. Життєвий цикл процесу розпочинається з етапу ініціалізації: коли користувач ініціює створення проєкту, система автоматично звертається до бази даних та завантажує масив доступних публічних і системних шаблонів. Після цього відбувається етап введення даних, під час якого користувач обирає бажаний макет та заповнює конфігураційну

форму. Щойно дані відправляються на сервер, бекенд розпочинає паралельну валідацію, одночасно перевіряючи синтаксичну правильність введеної інформації та її відповідність встановленим бізнес-правилам платформи.

Якщо корисне навантаження успішно проходить первинну перевірку, алгоритм переходить до етапу верифікації унікальності. На цьому кроці система виконує SQL-запит до таблиці sites, перевіряючи, чи є вільним обраний користувачем ідентифікатор маршруту. У разі успішного проходження всіх етапів настає завершальна фаза: у базі даних генерується новий запис проєкту, до якого автоматично копіюється базова структура сторінок з обраного шаблону, після чого відбувається маршрутизація користувача до його нового робочого простору. Водночас алгоритм передбачає обробку альтернативних гілок: якщо на будь-якому з етапів валідації виявляються некоректні дані або запитаний URL вже зайнятий іншим ресурсом, процес збереження миттєво переривається, а клієнтський застосунок виводить відповідне повідомлення про помилку для інформування користувача.

Алгоритм процесу "Редагування контенту - Редактор" оптимізовану роботу з візуальним конструктором, де основне навантаження лягає на клієнтську частину, а звернення до сервера мінімізовані. Послідовність виконання операцій при взаємодії користувача з віртуальним DOM та механізм синхронізації даних із сервером відображено на рис. 2.12.

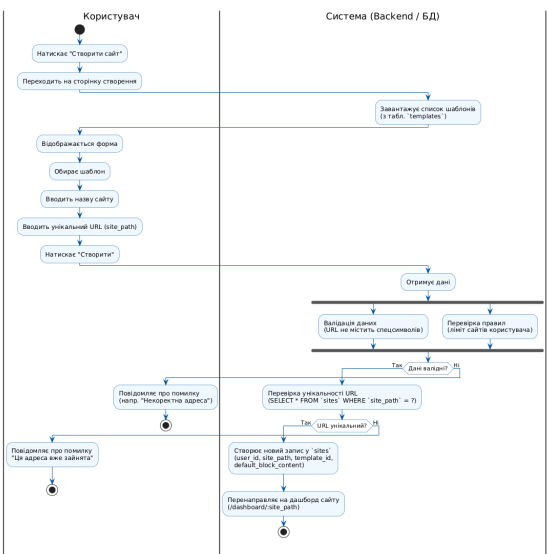


Рис. 2.12. Діаграма процесу візуального редагування контенту

Процес взаємодії користувача з візуальним редактором побудований за принципом оптимізації продуктивності та мінімізації надлишкових мережових запитів. Життєвий цикл сесії редагування розпочинається з етапу ініціалізації стану: під час відкриття робочого простору серверна частина вилучає поточний масив контентних блоків із таблиці pages та передає його на клієнт. На основі отриманих даних фронтенд-застосунок виконує первинний рендеринг інтерфейсу з використанням віртуальної об'єктної моделі документа.

Уся подальша взаємодія відбувається ізольовано, виключно в межах локальної пам'яті браузера. Користувач має змогу здійснювати комплексні маніпуляції з макетом сторінки паралельно додавати нові компоненти, редагувати їхні параметри, видаляти непотрібні елементи або динамічно змінювати їхній порядок розташування за допомогою механіки Drag-and-Drop. Ключовою архітектурною особливістю цього процесу є те, що під час активного редагування жодні проміжні запити до бази даних не ініціюються, що гарантує плавність роботи та миттєвий відгук інтерфейсу.

Фіксація змін відбувається централізовано на етапі синхронізації. Після натискання кнопки збереження клієнтський застосунок агрегує весь локальний стан, компілює його у фінальний оновлений JSON-об'єкт та відправляє на бекенд. Сервер приймає цю сформовану структуру, виконує цільову SQL-операцію UPDATE для оновлення запису в базі даних та повертає HTTP-статус успішного виконання. Отримавши підтвердження від сервера, клієнтський інтерфейс генерує відповідне візуальне сповіщення, інформуючи користувача про успішне збереження макета.

Алгоритм процесу "Взаємодія клієнта з сайтом-магазином" описує логіку додавання товару до кошика на згенерованому сайті-магазині, враховуючи перевірку залишків та авторизації. Детальну блок-схему поведінки системи під час клієнтської взаємодії з модулем комерції наведено на рис. 2.13.

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		58

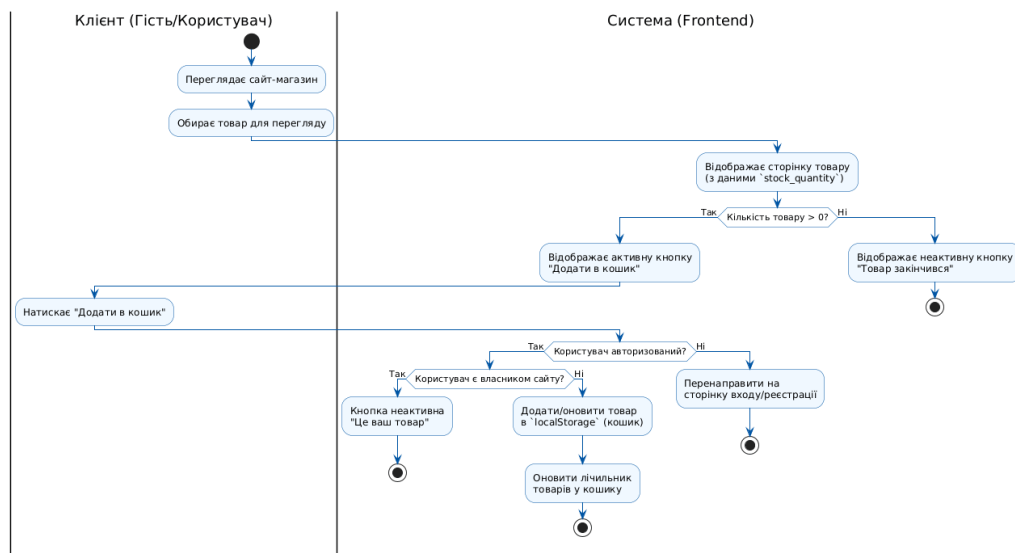


Рис. 2.13. Діаграма клієнтської взаємодії з модулем комерції

Процес взаємодії користувача з модулем електронної комерції під час додавання товару до кошика формалізовано у вигляді багатокрокового алгоритму з вбудованими механізмами валідації. Життєвий цикл операції розпочинається з етапу перегляду: коли клієнт відкриває сторінку конкретної позиції, фронтенд-застосунок превентивно аналізує значення поля `stock_quantity`. На основі цих даних реалізується умова наявності якщо залишок дорівнює нулю, інтерфейсна кнопка покупки автоматично блокується для запобігання хибним замовленням, тоді як за фактичної наявності товару вона залишається активною.

Після безпосередньої спроби користувача додати обраний товар система переходить до етапу контролю доступу, перевіряючи наявність активної авторизаційної сесії. Якщо запит ініційовано неавторизованим клієнтом, алгоритм призупиняється, а користувача маршрутизують на сторінку реєстрації. Окрім базової автентифікації, система виконує специфічну перевірку для захисту від штучної накрутки показників магазину. На цьому кроці верифікуються ідентифікатори: бекенд перевіряє, чи не збігається поточний клієнт із власником даного електронного магазину, блокуючи операцію у разі збігу. За умови успішного проходження всіх етапів перевірки настає оновлення кошика. Валідний товар інтегрується до локального стану клієнтського застосунку, що

ініціює реактивне оновлення лічильника в інтерфейсі та сигналізує користувачу про успішне завершення дії.

Алгоритм процесу "Призупинення сайту Адміністратором" важливий для системної модерації, який демонструє автоматичне застосування санкцій до порушників. Логіку розгалужень, прийняття рішень та зміну станів об'єктів у ході виконання адміністративного блокування проілюстровано на рис.2.14.

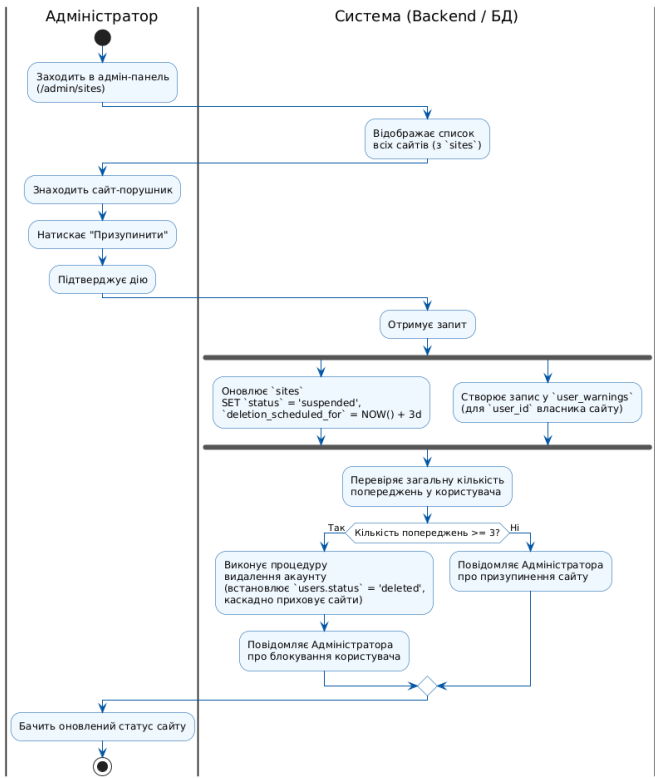


Рис. 2.14. Діаграма процесу призупинення сайту адміністратором

Процес модерації контенту розпочинається з ідентифікації адміністратором сайту-порушника та ініціації його призупинення. Після цього бекенд паралельно виконує дві дії: змінює статус сайту в базі даних на призупинений та створює відповідний запис у таблиці user_warnings. Далі сервер підраховує загальну кількість попереджень власника ресурсу для перевірки на рецидив. На етапі прийняття рішення система аналізує цю кількість: якщо вона досягає критичної межі у три або більше порушень, обліковий запис користувача автоматично блокується разом із каскадним приховуванням усіх його проєктів. Якщо ж ліміт не перевищено, санкції застосовуються виключно до конкретного сайту-порушника, залишаючи інші ресурси користувача активними.

Алгоритм процесу "Обробка тикету підтримки" ілюструє циклічний процес вирішення проблеми клієнта за допомогою системи тикетів. Послідовність кроків від створення запиту до його остаточного закриття, включаючи можливі ітерації уточнення інформації між користувачем та адміністратором, наочно відображено на рис. 2.15.

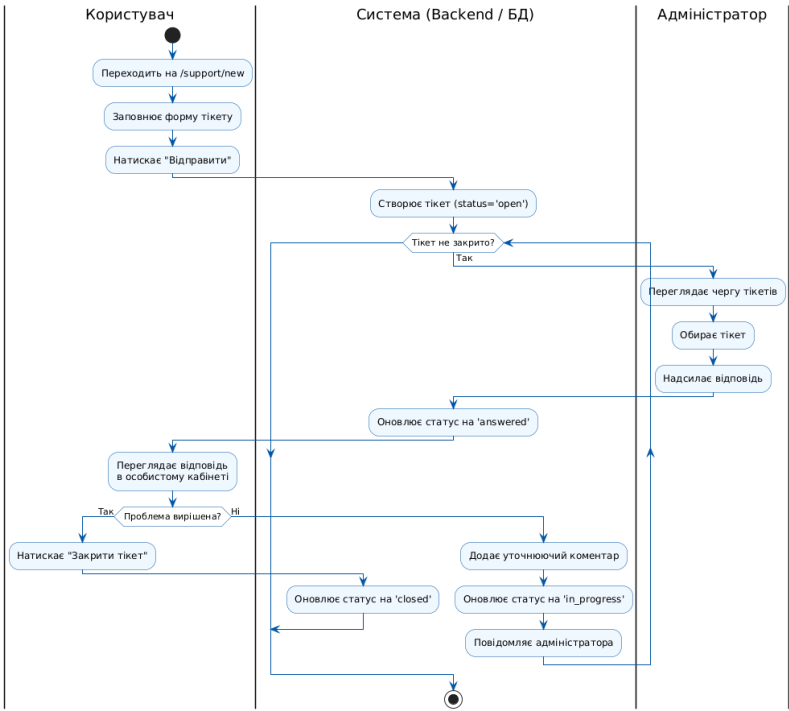


Рис. 2.15. Діаграма процесу обробки тикету підтримки

Алгоритм опрацювання звернень до служби підтримки розпочинається з ініціації тикета, під час якої користувач надсилає форму, а бекенд створює відповідний запис у таблиці support_tickets зі статусом open. Після цього активується логічний цикл обробки звернення, в межах якого адміністратор надає відповідь, що автоматично змінює статус тикета на answered.

На наступному етапі користувач перевіряє отриману інформацію: якщо проблема вважається вирішеною, він підтверджує закриття звернення, що призводить до завершення циклу. У випадку, якщо відповідь не задовольняє користувача, він додає уточнення до тикета, статус якого змінюється на in_progress. Система надсилає сповіщення адміністратору, а алгоритм повертається до початку циклу для повторної обробки звернення.

Висновки до другого розділу

У другому розділі було виконано комплексне проєктування програмної архітектури платформи, яке охоплює всі етапи від розробки високорівневої моделі інфраструктури до деталізації алгоритмів взаємодії компонентів.

Побудовано діаграму розгортання, яка ілюструє мікросервісний підхід до побудови системи з використанням сучасних хмарних рішень. Такий розподіл гарантує відмовостійкість, глобальну доступність та незалежне масштабування клієнтської і серверної частин.

Проведено детальне моделювання варіантів використання системи, де визначено чотирьох незалежних акторів та формалізовано їхні права доступу. Для ключових прецедентів розроблено розширені специфікації, що слугують основою для контрактного програмування.

Розроблено об'єктно-орієнтовану структуру системи. Побудовані діаграми класів предметної області та класів реалізації ілюструють застосування класичних патернів проєктування. Діаграма компонентів відображає фізичну організацію коду за принципом Feature-Sliced Design для клієнтської частини та модульного поділу для серверної.

Спроектовано структуру гібридної бази даних MySQL, нормалізованої до третьої нормальної форми, де реляційні зв'язки між системними сутностями поєднуються зі зберіганням користувацького контенту у форматі JSON.

Описано динамічну поведінку системи за допомогою діаграм активності, що деталізують бізнес-логіку ключових процесів. Крім того, побудовано діаграми послідовності та кооперації, які візуалізують хронологію та топологію передачі повідомлень між клієнтом, контролерами та шаром даних.

Розроблені логічні та структурні моделі повністю задовольняють вимоги технічного завдання і є вичерпною документацією для переходу до етапу безпосередньої фізичної реалізації програмного продукту, описаної у наступному розділі.

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		62

РОЗДІЛ 3. РЕАЛІЗАЦІЯ ТА ТЕСТУВАННЯ ПРОЄКТУ

3.1 Програмна реалізація серверної частини та підсистеми безпеки

Розробка серверної частини платформи базується на використанні середовища виконання Node.js та фреймворку Express, що забезпечує асинхронну обробку запитів і високу продуктивність. Архітектура побудована за принципами REST API із застосуванням патерну MVC. Логіка розділена на ізольовані модулі: маршрутизатори, контролери, моделі бази даних та проміжне програмне забезпечення, що надає високий рівень підтримуваності та масштабованості коду.

Для захисту інфраструктури від типових вебвразливостей на рівні сервера впроваджено модуль helmet, який автоматично встановлює безпечні HTTP-заголовки. Доступ до ресурсів API контролюється політикою Cross-Origin Resource Sharing, яка динамічно перевіряє джерело запиту та дозволяє взаємодію лише із завчасно визначеними клієнтськими доменами. Для захисту публічних форм від автоматизованого спаму та Brute-force атак інтегровано систему Cloudflare Turnstile, перевірка якої винесена в окремий middleware-модуль.

Взаємодія з реляційною базою даних MySQL реалізована через механізм пулу з'єднань з використанням обгортки на основі промісів. Це дозволяє управляти паралельними запитами без перевантаження сервера баз даних. Конфігурація підключення є динамічною: параметри зчитуються із захищених змінних оточення, а при розгортанні у хмарному середовищі автоматично активується SSL-шифрування каналу передачі даних.

База даних платформи спроектована з урахуванням вимог третьої нормальної форми. Для забезпечення цілісності даних активно використовуються зовнішні ключі з механізмом каскадного видалення. Наприклад, при видаленні акаунту користувача система автоматично видаляє всі пов'язані з ним сайти, сторінки, товари та медіафайли. Для оптимізації виконання складних вибірок створено спеціалізовані індекси. Крім того, критичні операції

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		63

виконуються в рамках SQL-транзакцій, що гарантує атомарність операцій та запобігає частковому запису даних у разі програмних збоїв.

Значна частина функціоналу пов'язана із системою безпеки, авторизації та керування доступом. Процес реєстрації нових користувачів супроводжується перевіркою глобальних налаштувань системи, валідацією надійності пароля за допомогою регулярних виразів та його подальшим хешуванням алгоритмом bcrypt із використанням криптографічної солі. Для підтвердження електронної пошти та відновлення доступу розроблено механізм генерації одноразових паролів, які зберігаються в базі даних та обмежені коротким терміном дії. Додатково впроваджено механізм OAuth 2.0 для швидкої та безпечної авторизації через акаунти Google.

Авторизація в системі побудована на основі стандарту JWT. Для захисту приватних маршрутів розроблено багаторівневе проміжне програмне забезпечення. При кожному зверненні до закритого ресурсу система виконує такі етапи перевірки:

- a. Вилучення та криптографічна валідація підпису токена.
- b. Перевірка поточного статусу облікового запису в базі даних.
- c. Синхронізація версії токена. У базі даних зберігається ідентифікатор версії сесії користувача. При зміні даних ця версія оновлюється, що робить усі попередньо видані JWT-токени недійсними.

Для розмежування прав доступу реалізовано рольову модель доступу. Маршрути API захищені відповідними гвардами, які перевіряють, чи має користувач достатні привілеї для виконання певної дії. Для адміністративного персоналу та модераторів платформи впроваджено обов'язкову двофакторну автентифікацію. Навіть при введенні коректних облікових даних система генерує тимчасовий код, який надсилається на корпоративну пошту співробітника, і лише після його підтвердження генерується токен доступу до панелі керування (деталі реалізації middleware перевірки токена наведено в Додатку Б, лістинг Б.3).

Для підвищення безпеки та контролю за діями персоналу розроблено систему внутрішнього аудиту. Усі критичні дії адміністраторів, такі як зміна

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		64

статусів користувачів, видалення сайтів, управління шаблонами, що автоматично фіксуються в окремій таблиці бази даних із зазначенням типу дії, ідентифікатора адміністратора, IP-адреси та часової мітки. Це забезпечує прозорість управління платформою та полегшує розслідування інцидентів.

Логіка підсистеми безпеки також враховує процеси Soft Delete: якщо користувач ініціює видалення акаунту, система надає 14-денний період для можливого відновлення, ізолюючи при цьому доступ до публічних ресурсів. Після завершення цього терміну спеціально налаштований планувальник завдань бази даних ініціює фонове безповоротне очищення інформації. Аналогічні фонові завдання використовуються для автоматичного видалення застарілих попереджень користувачів.

Оскільки платформа дозволяє створення інтернет-магазинів, на серверній частині реалізовано захищену підсистему електронної комерції та інтеграцію з платіжним шлюзом LiqPay. Сервер відповідає за формування захищених платіжних запитів, зберігання публічних та зашифрованих приватних ключів доступу користувачів, а також за обробку callback-запитів від платіжної системи. Усі фінансові транзакції проходять перевірку криптографічного підпису перед зміною статусу замовлення в базі даних.

Для забезпечення роботи функціоналу конструктора та модулів електронної комерції з графічним контентом, на сервері налаштовано інтеграцію з хмарним провайдером Cloudinary. Процес завантаження розділений на етапи: спочатку файл приймається сервером через middleware multer, перевіряється його MIME-тип та розмір, після чого дані через офіційний SDK з використанням API-ключів передаються у хмару. Це дозволяє делегувати важкі завдання з обробки, стиснення та збереження медіафайлів на спеціалізовану зовнішню інфраструктуру, залишаючи у власній базі даних лише оптимізовані URL-посилання.

Стабільність роботи бекенду забезпечується централізованим обробником помилок. Усі винятки, що виникають у контролерах, перехоплюються на верхньому рівні, логуються для подальшого аналізу та повертаються клієнту у

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		65

вигляді стандартизованих JSON-відповідей без розкриття чутливої інформації або стек-трейсів сервера.

Завдяки описаним рішенням серверна частина забезпечує високий рівень безпеки даних, цілісність транзакцій, відмовостійкість та готовність до масштабування при збільшенні кількості користувачів.

3.2 Розробка візуального редактора та клієнтського інтерфейсу

Клієнтська частина платформи побудована на базі бібліотеки React та зібрана за допомогою сучасного інструментарію Vite, що забезпечує швидке гаряче перезавантаження під час розробки та високооптимізований білд для робочого середовища. Використання React забезпечує компонентний підхід до розробки та ефективно оновлення об'єктної моделі документа завдяки механізму віртуального DOM. Архітектура фронтенду розділена на логічні модулі: ядро відмальовування сторінок, візуальний редактор, панель керування, систему маршрутизації та модулі електронної комерції.

Управління глобальним станом додатку реалізовано за допомогою React Context API, що дозволило уникнути проблеми надмірного прокидання властивостей через багаторівневе дерево компонентів. Створено низку ізольованих провайдерів стану: AuthContext для управління даними сесії користувача, ThemeContext для глобальної темізації інтерфейсу, а також CartContext та FavoritesContext для управління кошиком і збереженими товарами у модулі магазину.

Основою захисту клієнтських маршрутів є спеціалізований компонент-обгортка ProtectedRoute, який виконує роль перехоплювача навігації. Він інтегрований у систему маршрутизації та перевіряє поточний стан авторизації користувача, його роль та статус облікового запису. Залежно від рівня доступу система динамічно дозволяє перехід на захищені сторінки або перенаправляє на форму авторизації, запобігаючи несанкціонованому доступу до панелі керування чи редактора без необхідності зайвих звернень до сервера.

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		66

Центром платформи є підсистема динамічного рендерингу сторінок. Збережена в базі даних структура сайту представляє собою масив конфігураційних об'єктів у форматі JSON. Ядро рендерингу ітерує цей масив і динамічно співставляє тип кожного об'єкта з відповідним React-компонентом (структуру ядра рендерингу наведено в Додатку Б, лістинг Б.2). Для оптимізації продуктивності та зменшення розміру початкового пакета застосовано технологію Lazy Loading з використанням механізму Suspense. Компоненти складних блоків завантажуються в пам'ять клієнтського пристрою асинхронно, тільки тоді, коли вони безпосередньо необхідні для відмальовування.

Логіка візуального конструктора базується на інтерактивній взаємодії за допомогою технології Drag-and-Drop, реалізованої через кастомний хук `useBlockDrop`. Механізм перетягування обробляє дві основні події: додавання нового блоку з бібліотеки елементів та зміну порядку вже існуючих блоків на сторінці. При наведенні курсора на робочу область система обчислює координати вказівника відносно меж цільового елемента. Завдяки математичному порівнянню координат обчислюється точна позиція верхньої або нижньої половини цільового блоку, що супроводжується візуальною індикацією для користувача. Додатково реалізовано алгоритм автоматичного прокручування екрана, який плавно зміщує область видимості, коли користувач підтягує елемент до країв екрана при утриманні блоку.

Кожен структурний елемент візуального редактора концептуально розділений на дві частини: компонент відображення та компонент налаштувань. Компонент відображення є ізольованим `dumb component`, який лише приймає вхідні параметри та рендерить їх. Компонент налаштувань, у свою чергу, генерує панель інструментів із полями вводу, перемикачами, селекторами кольору та інструментами вибору шрифтів. Зміна будь-якого параметра в панелі налаштувань миттєво змінює локальний стан конфігурації, забезпечуючи користувачу досвід редагування у режимі реального часу.

Для реалізації складних елементів інтерфейсу, таких як випадаючі списки, модальні вікна збереження шаблонів або інструменти кадрування зображень[27],

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		67

активно використовується механізм React Portals. Це дозволяє рендерити ці елементи безпосередньо в корінь DOM-дерева, повністю ізолюючи їх від контексту накладання батьківських компонентів. Такий підхід гарантує, що модальні вікна та випадаючі меню завжди відображатимуться поверх інших елементів інтерфейсу, незалежно від їхньої вкладеності.

Оформлення клієнтського інтерфейсу побудовано з використанням CSS-фреймворку Tailwind CSS. Для забезпечення гнучкої тимізації платформи класи Tailwind скомбіновані з динамічними CSS-змінними. При обробці блоків система динамічно під'єднує індивідуальні стилі, запобігаючи конфліктам між глобальними стилями платформи та локальними стилями згенерованих користувачами вебресурсів.

Для забезпечення безперервності робочого процесу та захисту від втрати даних впроваджено систему автозбереження через кастомний хук useAutoSave. Оскільки відправка мережевого запиту при кожному натисканні клавіші створювала б надмірне навантаження на сервер та затримки в інтерфейсі, алгоритм автозбереження використовує патерн програмування “debounce”. Система акумулює всі зміни локально і відправляє REST-запит на збереження JSON-структури лише після того, як користувач припиняє взаємодію з редактором на визначений проміжок часу. (приклад реалізації логіки debounce наведено в Додатку Б, лістинг Б.1).

Такий підхід до реалізації клієнтської частини дозволив створити реактивний, оптимізований, стійкий до помилок та інтуїтивно зрозумілий інтерфейс, який абстрагує складність написання коду і надає користувачам професійний інструмент для візуального конструювання вебресурсів.

3.3 Керівництво користувача та розгортання платформи

Розгортання платформи базується на хмарній інфраструктурі, що використовує модель розподілених сервісів для забезпечення максимальної доступності та швидкодії. Архітектура передбачає поділ системи на три основні сегменти: клієнтську частину, серверну частину та керовану базу даних.

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		68

Для розміщення клієнтського додатка обрано платформу Vercel. Процес розгортання реалізовано через механізм безперервної інтеграції шляхом синхронізації з репозиторієм на платформі GitHub. При кожному оновленні гілки проєкту система автоматично ініціює збірку оптимізованого виробничого білда за допомогою Vite. Для коректної роботи маршрутизації односторінкового додатка та запобігання помилкам при перезавантаженні сторінок налаштовано правила перенаправлення запитів у файлі конфігурації, що спрямовують усі вхідні маршрути на головний файл index.html.

Серверна частина функціонує в ізольованому контейнері на платформі Render. Це середовище забезпечує стабільне виконання Node.js коду, автоматичне керування залежностями та моніторинг стану сервера. Зв'язок між клієнтом та сервером здійснюється за захищеним протоколом HTTPS. Платформа підключена до власного доменного імені kendr.online, придбаного через реєстратора Namecheap, що забезпечує впізнаваність ресурсу та професійний рівень доступу для кінцевих користувачів.

Збереження даних реалізовано за допомогою хмарного сервісу Aiven, де розгорнуто керований екземпляр бази даних MySQL. Таке рішення дозволяє відокремити обчислювальні потужності сервера від сховища даних, забезпечуючи високу швидкість обробки запитів та безпеку збереження інформації за межами локальних серверів. Для потреб розробки та локального тестування передбачено можливість запуску системи в ізольованому середовищі з використанням окремої локальної бази даних, що керується через MySQL Workbench.

Конфігурація доступу до всіх сервісів винесена у змінні оточення, що гарантує безпеку та легкість перемикання між локальним та виробничим середовищами без зміни програмного коду.

Інтерфейс авторизації функціонує як єдиний динамічний компонент, який перемикається між режимами входу, реєстрації, відновлення пароля та перевірки одноразових кодів без перезавантаження сторінки. Керування поточним станом вікна прив'язане до параметрів URL-адреси, що зберігає обраний режим навіть

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		69

після оновлення сторінки у браузері. Взаємодія з користувачем починається з універсальної панелі, де доступна класична автентифікація через електронну пошту та пароль, а також інтеграція з провайдером Google OAuth. Процес створення нового облікового запису розширено вбудованим інструментом для миттєвого завантаження та попереднього перегляду фотографії профілю безпосередньо всередині форми.

Безпека клієнтської частини інтегрована на рівні кожного етапу введення даних. Усі запити до сервера проходять перевірку через прихований віджет Cloudflare Turnstile для блокування автоматизованих ботів. Під час створення акаунту інтерфейс у реальному часі аналізує пароль, відображаючи візуальний індикатор його надійності та відхиляючи прості комбінації. Процедури верифікації електронної пошти або зміни пароля працюють через 6-значні ОТР-коди з таймером блокування повторної відправки для запобігання спаму. Кожен крок супроводжується спливаючими сповіщеннями [28], які інформують про успіх операції або точну причину відхилення запити. Наочний вигляд головного екрана входу та реєстрації представлено на рисунку 3.1.

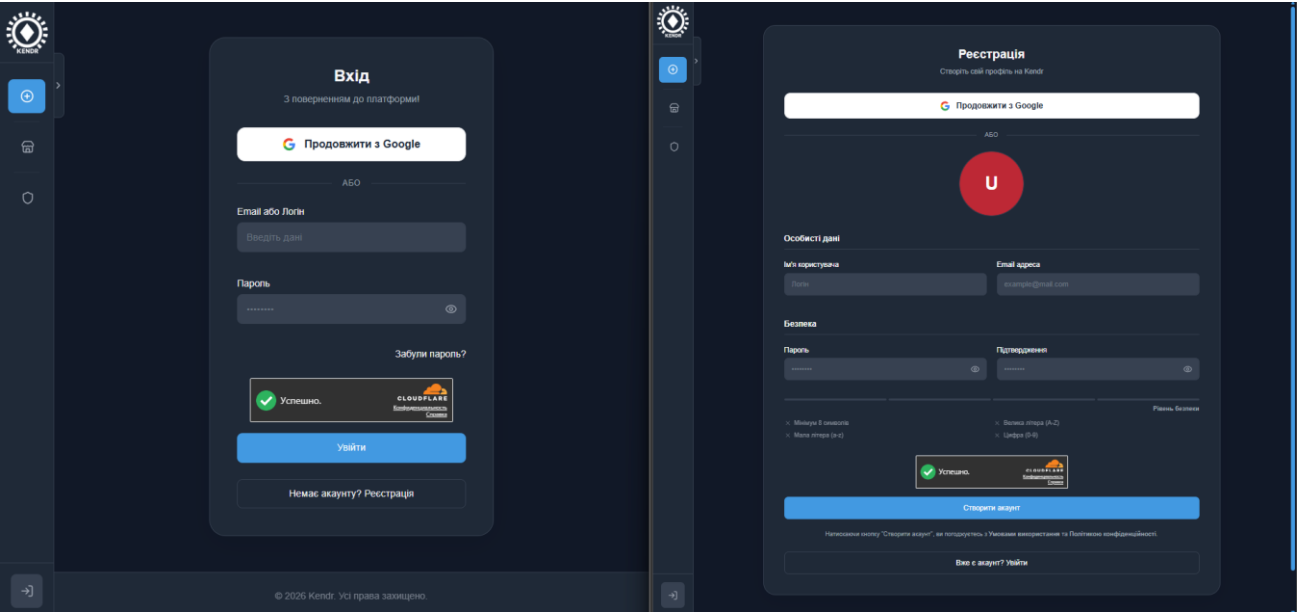


Рис. 3.1. Інтерфейс форми авторизації та реєстрації користувачів

Головна панель керування функціонує як центральний хаб для роботи з користувацькими вебресурсами. Проекти структуровано у вигляді адаптивної сітки карток, кожна з яких містить обкладинку, аналітичні метрики та поточний

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		70

стан сайту. Інтерфейс автоматично розрізняє права доступу, пропонуючи відповідний набір інструментів залежно від того, чи є користувач повноправним власником ресурсу, чи запрошеним співавтором. Прямо з картки проєкту доступні швидкі дії: перемикання статусу публікації, закріплення пріоритетних сайтів у верхній частині списку та повне видалення проєкту.

Верхня частина екрана містить закріплену панель інструментів із розширеною системою пошуку та фільтрації. Робота з великим масивом даних оптимізована завдяки миттєвому текстовому пошуку, сортуванню за датою створення, назвою чи переглядами, а також групуванню за тематичними тегами. Програмна логіка сторінки включає механізм динамічного підвантаження карток порціями для мінімізації навантаження на браузер. Компонент управління повністю синхронізовано з підсистемою білінгу, яка відстежує ліміти доступних проєктів відповідно до тарифного плану, автоматично деактивуючи кнопку ініціалізації нового ресурсу при досягненні квоти. Візуальне представлення робочого простору панелі управління наведено на рисунку 3.2.

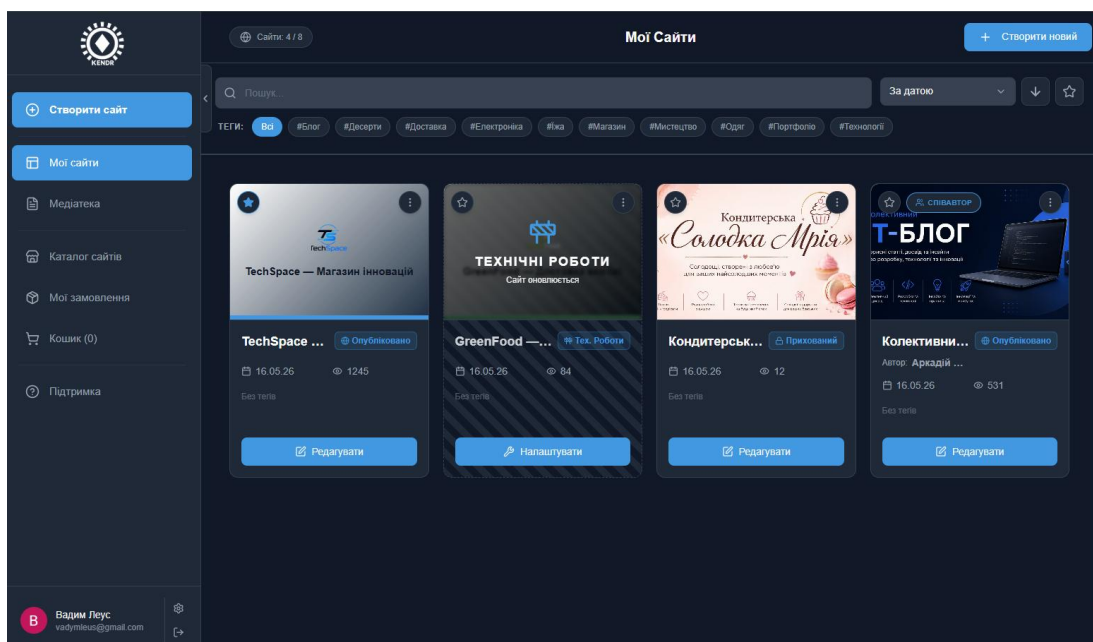


Рис. 3.2. Головна панель керування проєктами користувача

Інтерфейс процесу створення нового ресурсу (рис. 3.3.) функціонує за принципом Split-View, що об'єднує панель конфігурації та зону інтерактивного попереднього перегляду. Панель налаштувань складається з двох логічних

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		71

вкладок: “Інформація” та “Дизайн”. На першому етапі система запитує базові ідентифікатори проєкту, застосовуючи багаторівневу валідацію. Введення назви сайту супроводжується миттєвою локальною перевіркою на допустиму довжину та наявність заборонених символів. Водночас генерація унікального ідентифікатора маршруту запускає асинхронний запит до серверного API з використанням техніки *debounce*, що дозволяє перевіряти доступність адреси у фоновому режимі без перевантаження бази даних зайвими викликами під час набору тексту.

Другим етапом ініціалізації є вибір візуальної основи проєкту через вкладку “Дизайн”. Система пропонує каталог готових шаблонів, розділених на системні та персональні, а також опцію старту з «чистого аркуша». Обрання будь-якого макета негайно синхронізується із зоною попереднього перегляду, де користувач може взаємодіяти зі згенерованим інтерфейсом, перемикаючи роздільні здатності та оцінювати структуру блоків до моменту фізичного створення запису в базі даних. Фінальна ініціація збереження відбувається лише за умови повної консистентності даних: успішної валідації адреси, наявності назви та відсутності перевищення лімітів тарифного плану користувача.

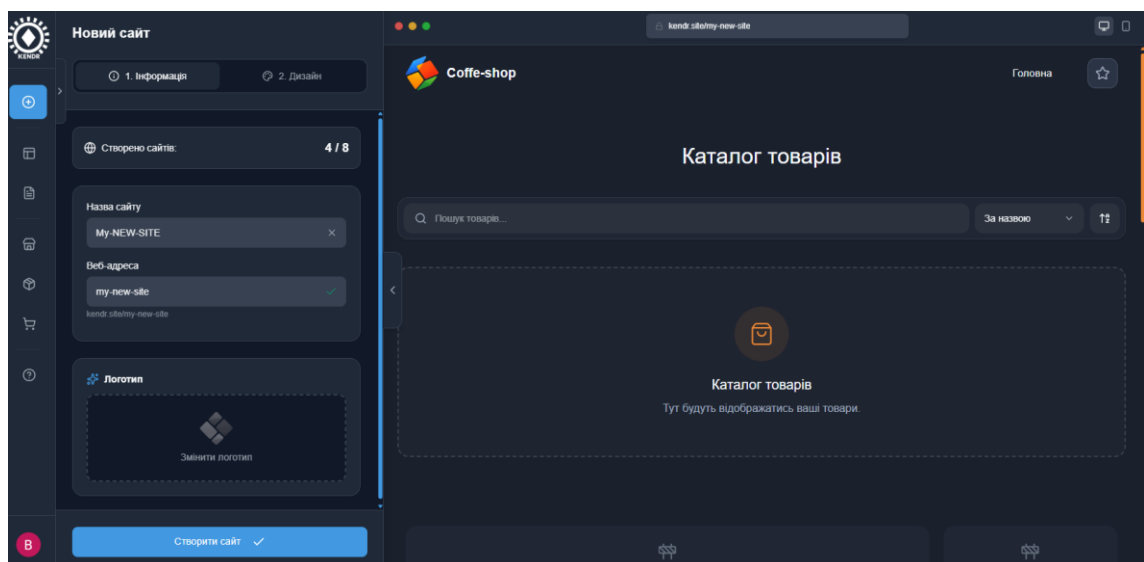


Рис. 3.3. Процес створення нового вебресурсу

Архітектура візуального редактора побудована за модульним принципом, де робочий простір логічно поділено на інтерактивну область рендерингу та багатофункціональну бічну панель інструментів. Навігаційна панель містить

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		72

систему вкладок, яка забезпечує швидкий доступ до бібліотеки готових компонентів, ієрархічного дерева шарів для управління вкладеностями та глобальних конфігурацій сторінки. Конструювання макета відбувається через інтегрований механізм Drag-and-Drop, який обробляє координати вказівника та динамічно обчислює зони прилипання. Це дозволяє безперешкодно перетягувати структурні одиниці від базових текстових полів чи медіафайлів до складних E-commerce каталогів та інтерактивних карт безпосередньо у віртуальний DOM робочої області.

Взаємодія з доданими компонентами реалізована через систему фокусних обгорт: при виділенні будь-якого елемента на екрані миттєво активується контекстне меню, а бічна панель перемикається в режим глибокої стилізації. Користувач отримує доступ до візуальних контролів для зміни типографіки, тонкого налаштування внутрішніх та зовнішніх відступів, керування кольоровою палітрою і застосування CSS-анімацій у режимі реального часу. Синхронізація змін керується фоновим алгоритмом автозбереження, який відстежує мутації стану контенту, серіалізує оновлену структуру в оптимізований JSON-формат та асинхронно відправляє пакети даних на сервер бекенду. Застосована технологія гарантує збереження всіх проміжних результатів верстки без ручного втручання та захищає від втрати інформації при розриві інтернет-з'єднання. Компонування описаних елементів інтерфейсу наведено на рисунку 3.4.

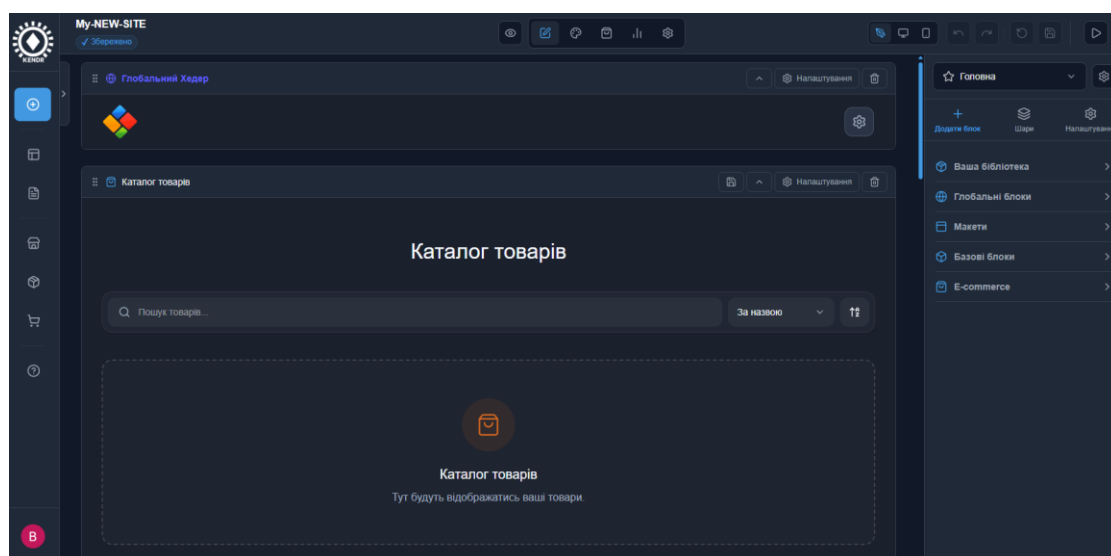


Рис. 3.4. Робоча область візуального конструктора сторінок

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		73

Механізм конфігурації компонентів функціонує на базі динамічного рендерингу панелі налаштувань. При виділенні певного елемента в робочій області система ідентифікує його тип та через внутрішній маршрутизатор автоматично підвантажує відповідний інтерфейсний модуль. Структурно панель розділена на дві ізольовані вкладки: управління контентом та розширені візуальні параметри. Такий піділ чітко розмежовує процес введення безпосередніх даних від алгоритмів їх стилізації. Передача нових значень до головного компонента сторінки відбувається через єдиний синхронний обробник подій, який оновлює віртуальне дерево блоків та миттєво відображає результати в редакторі.

Набір доступних інструментів кастомізації формується індивідуально для кожної структури залежно від її специфіки. На прикладі конфігуратора головної обкладинки, інтерфейс пропонує інтерактивні перемикачі для керування видимістю дочірніх вузлів заголовка, підзаголовка чи кнопок дії. Логіка роботи з фоном підтримує як статичні зображення, так і інтеграцію відеороликів із можливістю накладання напівпрозорих кольорових фільтрів для покращення читабельності. Усі контролери тісно пов'язані з глобальною системою стилів сайту, що оберігає загальний дизайн-код проєкту від випадкових порушень під час ручного редагування. Детальний інтерфейс меню параметрів наведено на рисунку 3.5.

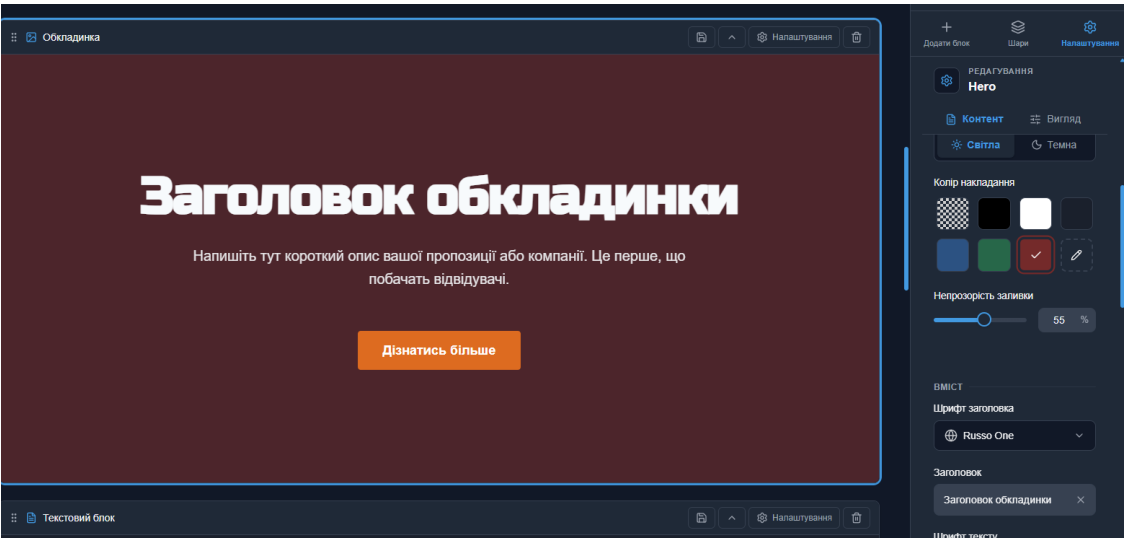


Рис. 3.5. Панель динамічного налаштування параметрів блоку

Підсистема управління медіафайлами реалізована у вигляді єдиного централізованого сховища, яке інкапсулює логіку імпорту, зберігання та базової обробки статичних ресурсів. Процес завантаження підтримує механізм DnD та асинхронне фонове відправлення масивів файлів. Завантажувач інтегровано з модулем квотування, який на льоту перевіряє розмір кожного об'єкта та загальну заповненість хмарного диска відповідно до лімітів тарифного плану користувача, відхиляючи невалідні запити з виведенням деталізованих сповіщень.

Організація робочого простору медіатеки базується на адаптивній сітці з підтримкою прогресивного підвантаження контенту порціями для оптимізації споживання оперативної пам'яті браузеру. Інтерфейс містить інструментарій для багатофакторної фільтрації: миттєвий текстовий пошук за назвою, сортування за розміром чи датою, а також глибоку фільтрацію за MIME-типами та конкретними форматами. Робота з файлами підтримує клавіатурні модифікатори для масового виділення об'єктів. Виділений масив даних можна миттєво видалити або завантажити на локальний пристрій у такому разі система автоматично формує єдиний ZIP-архів за допомогою клієнтської генерації через бібліотеку JSZip [29].

Для детального управління конкретним ресурсом передбачено бічну панель інспектора властивостей. У цьому модулі користувач може змінювати системну назву файлу, додавати внутрішні нотатки та редагувати SEO-атрибути (Alt-текст) для покращення індексації сайту пошуковими системами. Інспектор динамічно адаптує свій інтерфейс під тип контенту: для відео автоматично вираховується та виводиться тривалість, а при виділенні файлу шрифту система застосовує API FontFace, підвантажуючи його в браузер для відображення тестового типографічного рядка у реальному часі. Крім того, при виборі зображення для специфічного блоку, підсистема автоматично ініціює модуль кропінгу [30], дозволяючи кадрувати фотографію до необхідного співвідношення сторін перед її інтеграцією у макет. Візуальне представлення розробленого модуля наведено на рисунку 3.6.

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		75

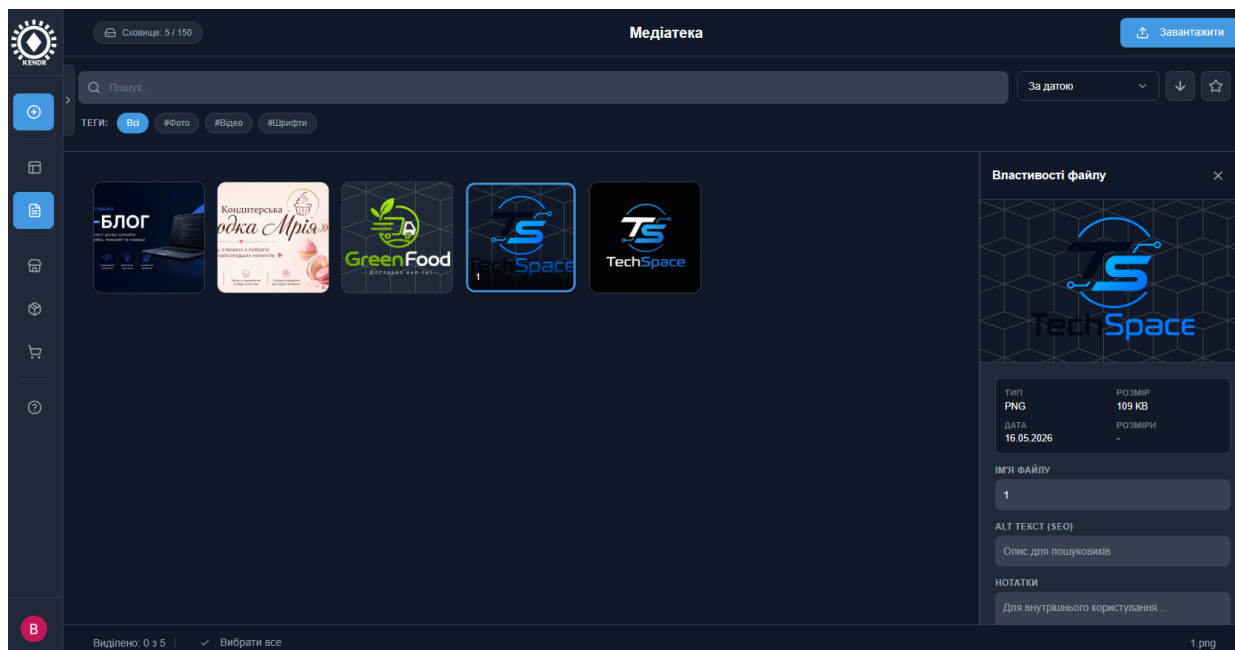


Рис. 3.6. Інтерфейс централізованої медіатеки

Архітектура системи передбачає наявність багатофункціонального модуля керування обліковим записом, який логічно розділено на публічну сторінку-портфолію та приватну панель конфігурації. Сторінка профілю динамічно генерується на основі унікального текстового ідентифікатора і слугує агрегатором активності користувача на платформі. Інтерфейс автоматично адаптує своє візуальне оформлення, використовуючи заданий користувачем персональний акцентний колір для генерації фонових градієнтів. Одночасно підвантажується агрегована аналітика: загальна кількість опублікованих ресурсів та сумарна кількість їх переглядів. Сторінка інтегрує підключені соціальні мережі та генерує інтерактивну сітку створених сайтів із вбудованими алгоритмами пошуку і сортування.

Важливою складовою підсистеми є механізми керування приватністю та прозора взаємодія з інструментами модерації. Користувач може налаштовувати рівень доступу до свого портфолію, до повного приховування сторінки від сторонніх відвідувачів платформи. У разі примусового блокування акаунта за порушення правил, алгоритм автоматично переводить колірну гаму аватара в монохромний режим і виводить глобальне попередження.

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		76

Процес безпосередньої конфігурації параметрів (рис. 3.7.) винесено в окремий маршрут налаштувань, побудований на базі модульної системи вкладок. Простір розділено на чотири основні категорії: редагування загальних даних, керування політиками безпеки, налаштування публічності та вибір глобальної теми оформлення інтерфейсу. Для оптимізації клієнтського досвіду механізм навігації використовує локальне сховище браузера для кешування стану останньої відкритої вкладки. Логіка рендерингу жорстко прив'язана до рольової моделі, система приховує або показує певні конфігураційні розділи залежно від того, чи належить акаунт звичайному користувачу, чи адміністратору платформи. Організація цього розділу відображена на рисунку 3.7.

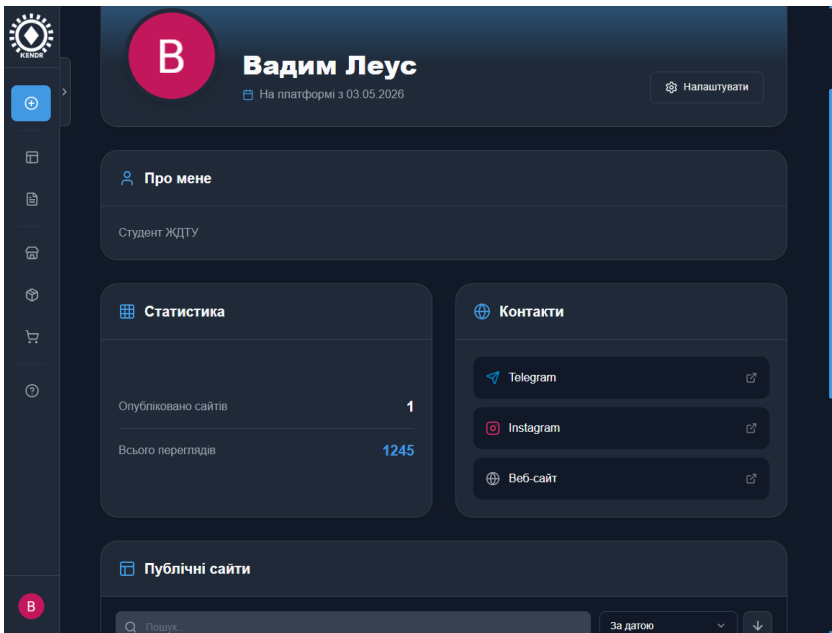


Рис. 3.7. Розділ налаштувань профілю користувача

Підсистема клієнтської підтримки є комплексним інструментом для забезпечення структурованої двосторонньої комунікації між користувачами та адміністрацією платформи. Вхідною точкою модуля слугує інтегрований центр підтримки, який містить базу знань із найпоширенішими запитаннями та гнучку маршрутизацію до ключових функціональних блоків: створення нових запитів, управління активними діалогами та спеціалізований розділ оскарження модераторських рішень.

Процес формування звернення реалізовано через адаптивну форму з підтримкою технології DnD для зручного додавання медіафайлів. Модуль

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				77
Змн.	Арк.	№ докум.	Підпис	Дата		

автоматично валідує прикріплені вкладення на клієнтському рівні за MIME-типом та розміром, жорстко обмежуючи їх кількість згідно з глобальною конфігурацією лімітів системи. До складу функціоналу також входить інтегрована підсистема апеляцій: у разі блокування сайту за порушення правил користувач отримує доступ до панелі, де відображається деталізована причина санкцій та точний таймер до остаточного видалення ресурсу. При ініціації оскарження алгоритм автоматично генерує контекстний тікет, успадковуючи ідентифікатор заблокованого сайту та створюючи шаблонний текст пояснення. Статус такої апеляції синхронізується з тікетом і відстежується за допомогою динамічних візуальних індикаторів.

Керування запитами здійснюється через табличний інтерфейс користувача, який підтримує багатофакторну фільтрацію за станом та категорією, а також наскрізний текстовий пошук за ID чи темою звернення. Детальний розгляд тікета імplementовано у вигляді сучасного чат-додатка з підтримкою асинхронного фонового поллінгу. Вікно діалогу оснащено смарт-скролінгом: алгоритм обчислює поточне положення прокрутки та або автоматично опускає вид до нових повідомлень, або виводить плаваючу кнопку з лічильником непрочитаних сповіщень, якщо користувач в цей момент аналізує історію переписки. Усі прикріплені зображення миттєво генерують локальні прев'ю в пам'яті браузерa із можливістю масштабування.

Вбудована система безпеки та захисту від спаму (рис. 3.8.) контролює життєвий цикл звернення через статусну модель. Інтерфейс автоматично блокує поле вводу, якщо тікет закритий адміністратором або самим ініціатором. На рівні клієнтської логіки реалізовано автоматичний антиспам-фільтр: якщо користувач відправляє кілька повідомлень поспіль без отримання відповіді від модератора, механізм тимчасово обмежує можливість подальшої відправки повідомлень. У такому випадку інтерфейс виводить попередження про блокування флуду, оптимізуючи навантаження на службу підтримки та запобігаючи зловживанням. Зовнішній вигляд робочого середовища системи внутрішньої технічної підтримки зображено на рисунку 3.8.

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		78

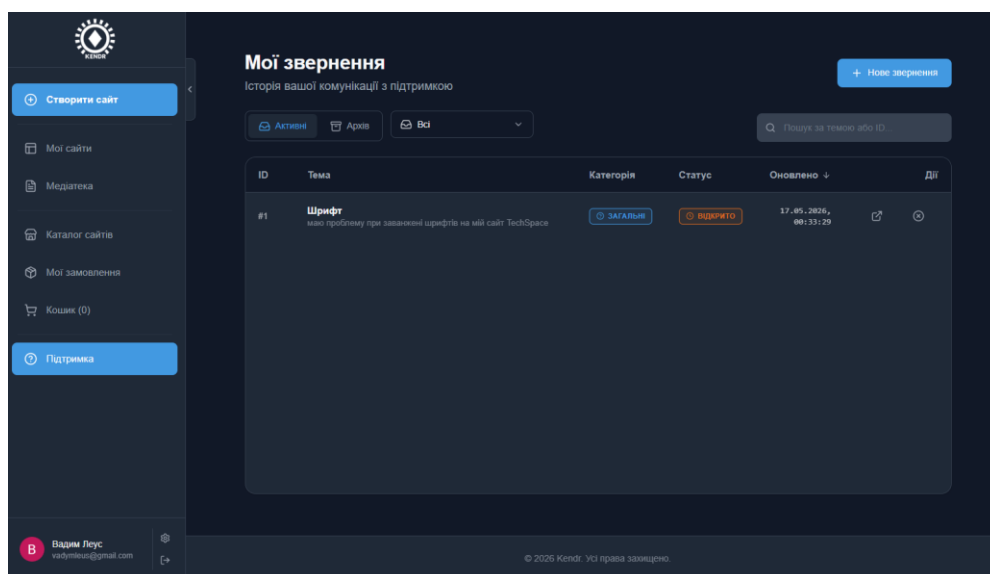


Рис. 3.8. Система внутрішньої технічної підтримки

Окремим модулем функціонує панель адміністратора (рис. 3.9.), доступ до якої жорстко обмежений рольовою моделлю та захищений двофакторною автентифікацією. Інтерфейс адміністратора надає зведену статистику роботи платформи, інструменти для модерації користувацьких сайтів, управління шаблонами та аналізу системних логів. Центральним елементом аналітики виступає інтерактивний дашборд, який розділяє потоки даних на загальний огляд та деталізований журнал подій. Візуалізація ключових метрик відбувається через систему віджетів, що транслюють динаміку приросту користувацької бази, кількість активних ресурсів, відкриті тікети та отримані скарги. Інтерфейс відображає тренди з автоматичним обчисленням змін за обраний період. Графічна репрезентація активності базується на адаптивних компонентах із підтримкою гнучкої фільтрації за типами сутностей і часовими проміжками, де інтерактивні стовпчики генерують контекстні підказки при наведенні курсора.

Підсистема керування екосистемою спирається на розширене табличне подання інформації зі складними механізмами фільтрації за ієрархією ролей, тарифними планами та поточним станом акаунтів. Модераторський інтерфейс містить набір інструментів глибокого впливу: від м'якого обмеження доступу шляхом призначення випробувального терміну для створених ресурсів до повного блокування чи незворотного видалення профілів разом із пов'язаними

медіафайлами. Задля запобігання випадковим деструктивним діям інтерфейсний алгоритм вимагає ручного введення підтверджувальних текстових маркерів. Усі табличні масиви даних синхронізовані з утилітою експорту, яка компілює структуровані CSV-файли для локального збереження та подальшого зовнішнього аудиту.

Центр управління платформою концентрує глобальні конфігурації безпеки та налаштування аварійних режимів. Модуль містить перемикачі для миттєвого переведення системи у стан технічного обслуговування, жорсткого блокування інструментів редагування сайтів та тимчасового призупинення процесів реєстрації, авторизації або приймання транзакцій. Інтерфейс підтримує конфігурацію інформаційного банера з інтеграцією динамічного зворотного відліку до завершення регламентних робіт. Фінансовий моніторинг винесено в ізольований робочий простір, де акумулюється статистика за прямими платежами платформи та внутрішніми замовленнями користувачів. Таблиця транзакцій підтримує багатокритеріальне сортування фінансових показників, наскрізний пошук за ідентифікаційними даними клієнтів та відстеження статусу операцій через багаторівневу модель кольорових візуальних індикаторів. Структурне компонування розглянутих елементів панелі адміністратора відображено на рисунку 3.9.

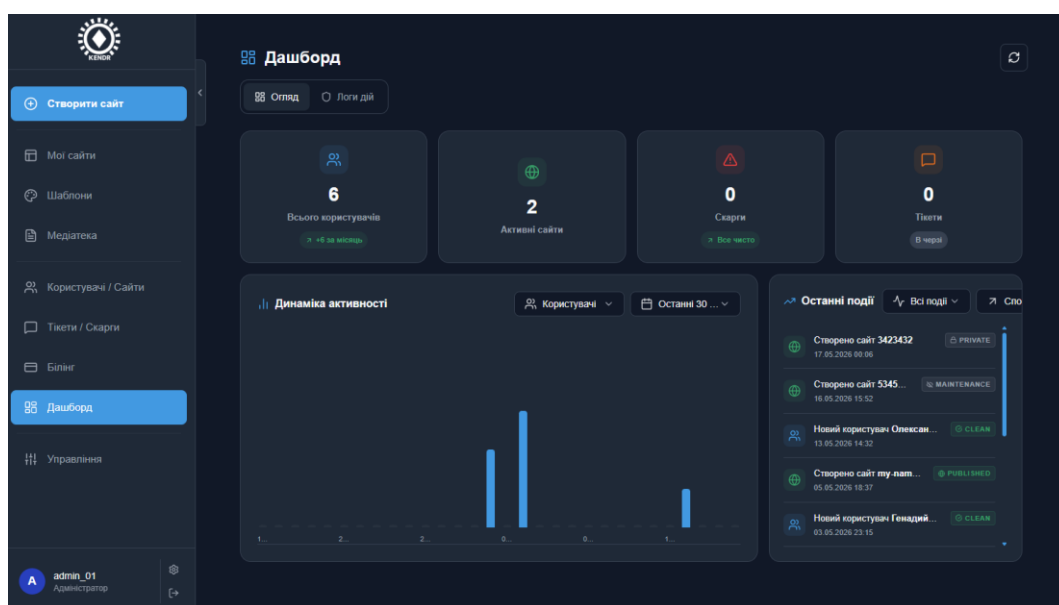


Рис. 3.9. Інтерфейс панелі адміністратора платформи

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		80

Описана архітектура розгортання та структура клієнтських інтерфейсів у сукупності утворюють цілісну екосистему для конструювання та адміністрування вебресурсів. Синхронізація хмарних сервісів із оптимізованим фронтенд-додатком гарантує стабільну обробку даних навіть за умов пікових навантажень. Жорстке розмежування функціоналу між публічною, приватною та модераторською зонами чітко структурує всі етапи взаємодії з платформою, від первинної авторизації до управління глобальними налаштуваннями сервера. Обраний стек технологій та модульна побудова компонентів створюють надійний фундамент для подальшого масштабування системи, додавання нових інструментів візуального редактора та розширення інтеграційних можливостей без переривання безперервного циклу надання послуг кінцевим споживачам.

3.4 Тестування програмної системи

Для забезпечення надійності, відмовостійкості та високої якості програмного коду платформи було впроваджено комплексну багаторівневу стратегію тестування. Вона охоплює модульне, інтеграційне, наскрізне тестування, перевірку API, а також тестування продуктивності. Такий підхід дозволяє виявляти дефекти як на рівні ізолюваних алгоритмів, так і на рівні бізнес-процесів усієї системи під навантаженням.

Етап модульного тестування був спрямований на перевірку ізолюваних функцій та утиліт, які не мають зовнішніх залежностей і можуть тестуватися незалежно від інших компонентів системи. Для реалізації цього рівня перевірок на клієнтській частині застосунку використовувався сучасний фреймворк Vitest, що забезпечує швидке виконання тестів та зручні механізми верифікації логіки.

Важливим напрямом тестування стали алгоритми валідації вхідних даних, оскільки вони є критично важливими для забезпечення безпеки та стабільності платформи. Зокрема, було розроблено набір тест-кейсів для функції `analyzePassword`, яка оцінює криптографічну стійкість пароля, перевіряючи наявність великих літер, чисел та мінімальної допустимої довжини. Також протестовано функцію `validateSiteSlug`, призначену для перевірки унікальних

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		81

ідентифікаторів сайтів на відповідність правилам маршрутизації та допустимому формату URL.

Окремо виконувалася перевірка алгоритму санітизації `validateSiteTitle`, основним завданням якого є виявлення та нейтралізація потенційно небезпечних HTML-конструкцій. Тестування підтвердило, що функція коректно обробляє спроби XSS-ін'єкцій шляхом видалення або екранування небезпечних тегів і скриптів у назвах проєктів.

Для забезпечення ізолюваності тестованого коду активно застосовувалися механізми мокінгу, які дозволяли імітувати поведінку браузерного API, локального сховища та окремих зовнішніх модулів без необхідності запуску повноцінного браузерного середовища. Такий підхід значно спростив тестування окремих функцій та підвищив стабільність і передбачуваність результатів виконання тестів.

Етап інтеграційного тестування передбачав перевірку коректної взаємодії між окремими компонентами клієнтської частини застосунку, побудованої на базі React. Основний акцент було зроблено на тестуванні взаємодії React-компонентів із віртуальним DOM-деревом, перевірці обробки подій користувача та коректності оновлення стану інтерфейсу після виконання певних дій. Для реалізації інтеграційних тестів використовувався інструментарій `@testing-library/react`, який дозволяє моделювати поведінку реального користувача та оцінювати роботу інтерфейсу з точки зору кінцевого використання.

Для верифікації серверної логіки та перевірки коректності функціонування REST API проводилося тестування програмного інтерфейсу з використанням інструментарію Postman. Цей етап охоплював перевірку ключових ендпойнтів серверної частини, маршрутів авторизації, управління користувачами, генерації контенту та роботи з базою даних. Під час тестування перевірялися не лише успішні сценарії взаємодії, але й обробка помилкових або некоректних запитів. Окремо аналізувалися HTTP-коди відповіді, зокрема повернення статусу 401 Unauthorized при відсутності або недійсності JWT-токена, а також 403 Forbidden при спробі доступу до ресурсів без відповідних прав доступу. Додатково

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		82

перевірено коректність обробки статусів 400 Bad Request та 500 Internal Server Error у випадках передачі некоректних параметрів або виникнення внутрішніх помилок сервера.

Крім цього, тестування охоплювало перевірку передачі властивостей між батьківськими та дочірніми компонентами, правильність виклику callback-функцій, а також коректність повторного рендерингу компонентів при зміні стану застосунку. Такий підхід дозволив переконатися у стабільності роботи користувацького інтерфейсу та відсутності помилок у логіці взаємодії між окремими модулями фронтенд-системи.

Тестування програмного інтерфейсу. Для верифікації серверної логіки та перевірки коректності функціонування REST API використовувався інструментарій Postman. Тестування охоплювало перевірку ключових ендпойнтів серверної частини, маршрути авторизації, управління користувачами, генерації контенту та роботи з базою даних.

Під час тестування перевірялися не лише успішні сценарії взаємодії, але й обробка помилкових або некоректних запитів. Окремо аналізувалися HTTP-коди відповіді, зокрема повернення статусу 401 Unauthorized при відсутності або недійсності JWT-токена, а також 403 Forbidden при спробі доступу до ресурсів без відповідних прав доступу. Додатково перевірено коректність обробки статусів 400 Bad Request та 500 Internal Server Error у випадках передачі некоректних параметрів або виникнення внутрішніх помилок сервера.

Створені колекції запитів у Postman дозволили автоматизувати процес тестування та забезпечити повторюваність перевірок після внесення змін до серверної логіки. Окремо виконувалася перевірка структури JSON-відповідей, правильності роботи механізмів пагінації, сортування та фільтрації даних. Також тестувалася швидкість відповіді API та стабільність роботи серверної частини при послідовному виконанні великої кількості запитів.

Комплексне тестування програмного інтерфейсу гарантує коректний обмін даними між клієнтською та серверною частинами, а також забезпечити надійність і передбачуваність роботи в умовах реальної експлуатації.

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		83

Для перевірки глобальних бізнес-сценаріїв з точки зору кінцевого користувача в умовах, максимально наближених до реальних, використано фреймворк Playwright. E2E-тести автоматизують проходження повного життєвого циклу користувача на платформі в середовищі реального браузера.

Один із розроблених сценаріїв виконує таку послідовність дій:

- a. Імітація авторизації тестового користувача через форму входу з перевіркою збереження сесійних cookie.
- b. Перехід до модуля створення проєкту та заповнення форми новою назвою і згенерованим унікальним маршрутом.
- c. Навігація до панелі керування створеним сайтом та активація функціоналу згоди на обробку даних.
- d. Перехід на сторінку створеного ресурсу для верифікації появи банера та перевірки логіки його закриття після натискання кнопки згоди.
- e. Повернення до панелі керування та виконання процедури повного видалення проєкту з підтвердженням слова "DELETE". Цей крок супроводжується перехопленням мережевого трафіку для верифікації статусу HTTP-запиту до REST API.

Тестування продуктивності та навантаження

Для перевірки стійкості серверної інфраструктури до високого трафіку було проведено тестування продуктивності за допомогою Apache JMeter.

Процес тестування включав декілька взаємопов'язаних етапів, спрямованих на перевірку продуктивності, стабільності та відмовостійкості системи. Зокрема, було проведено навантажувальне тестування, у межах якого виконувалася симуляція одночасної роботи великої кількості користувачів для оцінки пропускної здатності API під час генерації контенту вебсайтів.

Стрес-тестування передбачало штучне перевищення допустимих меж навантаження з метою виявлення вузьких місць системи, а також оцінювання ефективності механізмів балансування та відновлення після можливих збоїв. Крім цього, проводилося тестування стабільності, яке базувалося на тривалій

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		84

симуляції помірного навантаження для виявлення потенційних витоків пам'яті в процесах Node.js та поступової деградації продуктивності бази даних.

Конфігурація тестування була наближена до реальних сценаріїв взаємодії користувачів із платформою, що дозволило уникнути критичного впливу на робоче середовище та водночас виконати оптимізацію пулу з'єднань із базою даних і параметрів кешування. Такий комплексний підхід забезпечив перевірку коректної взаємодії всіх рівнів архітектури від клієнтського інтерфейсу до серверного маршрутизатора та системи зберігання даних під час виконання критично важливих операцій.

Успішне проходження всіх етапів тестування підтвердило готовність платформи до розгортання та подальшої безпечної експлуатації кінцевими користувачами.

Висновки до третього розділу

У третьому розділі було успішно виконано практичну реалізацію, комплексне тестування та хмарне розгортання розробленої програмної платформи.

Програмна реалізація серверної частини на базі Node.js та фреймворку Express дозволила створити масштабований REST API з архітектурою MVC. Завдяки використанню реляційної бази даних MySQL із застосуванням SQL-транзакцій та механізмів каскадного видалення забезпечено надійність і цілісність збережених даних. Важливим результатом стало впровадження комплексної підсистеми безпеки, яка включає JWT-авторизацію, рольову модель доступу, захист від спаму, двофакторну автентифікацію для персоналу та систему внутрішнього аудиту. Інтеграція із зовнішніми сервісами, зокрема платіжним шлюзом LiqPay та хмарним медіасховищем Cloudinary, дозволила реалізувати повноцінний функціонал електронної комерції.

Клієнтську частину системи реалізовано за допомогою бібліотеки React. Створено реактивний та стійкий до помилок користувацький інтерфейс із застосуванням технологій динамічного рендерингу та ізольованого управління

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		85

станом. Ключовим елементом фронтенду став візуальний редактор сторінок, який завдяки технології Drag-and-Drop, розділенню компонентів на логіку відображення та налаштувань, а також системі автозбереження, дозволяє користувачам інтуїтивно конструювати та кастомізувати вебресурси без навичок програмування.

Для підтвердження надійності програмного продукту було успішно застосовано багаторівневу стратегію тестування. Проведення модульного, інтеграційного, наскрізного E2E-тестування, перевірка API-ендпойнтів та тестування продуктивності під навантаженням довели високу якість коду, безпеку обробки даних та здатність архітектури стабільно функціонувати в умовах реальної експлуатації.

Завершальним етапом стало розгортання платформи на розподіленій хмарній інфраструктурі із налаштуванням безперервної інтеграції. Огляд розробленого графічного інтерфейсу та складене керівництво користувача підтверджують, що готовий програмний продукт має ергономічний дизайн, повністю задовольняє поставлені функціональні вимоги та є готовим до використання кінцевими споживачами.

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		86

ВИСНОВКИ

У кваліфікаційній роботі успішно досягнуто поставленої мети – спроектовано та розроблено повноцінну NoCode платформу для візуального конструювання вебресурсів. Для реалізації цієї мети було виконано комплекс визначених завдань: проаналізовано предметну область, досліджено концепцію NoCode-розробки та здійснено огляд наявних на ринку аналогів. За результатами дослідження обґрунтовано вибір технологічного стеку, а також визначено функціональні та нефункціональні вимоги до програмного забезпечення й апаратних ресурсів. У процесі інженерної роботи спроектовано архітектуру платформи, її об'єктно-орієнтовану та компонентну структуру, розроблено базу даних на основі гібридної моделі та описано алгоритми взаємодії компонентів системи. Практичним підсумком стало те, що безпосередньо реалізовано саму NoCode-платформу: серверну частину, візуальний Drag-and-Drop редактор сторінок, особистий кабінет користувача та панель адміністратора. На завершальному етапі систему успішно протестувано, налаштовано механізми безпеки та розгорнуто готовий продукт для демонстрації його роботи. Створений програмний комплекс надає користувачам можливість швидко розгортати посадкові сторінки, інтернет-магазини та прототипи без необхідності написання вихідного коду.

Виконанню практичної частини передував глибокий аналіз предметної області, який дозволив чітко окреслити цільову аудиторію, виявити недоліки існуючих рішень та сформувати унікальну нішу для продукту. На основі цих даних було обґрунтовано вибір сучасного технологічного стеку – екосистеми JavaScript у поєднанні з гібридною моделлю бази даних MySQL, що підтримує як реляційні зв'язки, так і гнучкий формат JSON. Детальне проектування архітектури з використанням стандарту UML забезпечило надійний фундамент для розробки, дозволивши заздалегідь продумати логіку взаємодії компонентів, мікросервісне розгортання та структуру бази даних, нормалізованої до третьої нормальної форми.

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		87

Процес програмної реалізації завершився створенням стійкої та масштабованої серверної частини, що відповідає за складну бізнес-логіку, фінансові транзакції через платіжний шлюз LiqPay, безпечну роботу з медіафайлами у хмарі та контроль доступу на основі JWT і двофакторної автентифікації. Водночас клієнтська частина платформи забезпечила кінцевих користувачів інтуїтивно зрозумілим візуальним редактором. Інтеграція механізмів Drag-and-Drop, оптимізованої системи автозбереження та динамічного рендерингу компонентів забезпечила швидку реакцію інтерфейсу та комфортний користувацький досвід.

Висока надійність та відмовостійкість розробленої системи були підтверджені шляхом проведення комплексного тестування. Багаторівнева стратегія, що охопила модульні, інтеграційні, наскрізні E2E-тести, детальну перевірку API та симуляцію навантаження, довела здатність платформи стабільно працювати та обробляти дані навіть в умовах інтенсивного трафіку. Фінальне розгортання проєкту в розподіленій хмарній інфраструктурі з налаштованими процесами безперервної інтеграції забезпечило глобальну доступність системи.

Підсумовуючи результати роботи, можна стверджувати, що розроблена платформа є технологічно завершеним, готовим до експлуатації програмним продуктом. Вона повністю задовольняє встановлені функціональні та нефункціональні вимоги, має безперечну практичну цінність для цільової аудиторії та володіє високим потенціалом для подальшого комерційного розвитку.

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		88

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Леус В. О., Савіцький Р.С.. ОСОБЛИВОСТІ РЕАЛІЗАЦІЇ КОМПОНЕНТНОГО ПІДХОДУ ПРИ РОЗРОБЦІ СИСТЕМ ДИНАМІЧНОЇ ГЕНЕРАЦІЇ ВЕБІНТЕРФЕЙСІВ. Тези ХІХ Всеукраїнської науково-практичної web конференції аспірантів, студентів та молодих учених «КОМП'ЮТЕРНІ ІНТЕЛЕКТУАЛЬНІ СИСТЕМИ ТА МЕРЕЖІ», 25-27 березня 2026 року. Кривий Ріг : «Криворізький національний університет», 2026. С.101.
2. Леус В. О., Савіцький Р. С. МЕТОДИ СТРУКТУРИЗАЦІЇ ТА ЗБЕРЕЖЕННЯ ДАНИХ У СИСТЕМАХ ВІЗУАЛЬНОГО КОНСТРУЮВАННЯ ВЕБРЕСУРСІВ. Матеріали ХІ Міжнародної науково-технічної конференції «Інформаційно-комп'ютерні технології», 02–03 квітня 2026 року. Житомир : Державний університет «Житомирська політехніка», 2026.
3. Фленаган Д. JavaScript. Повне керівництво. Київ : Жива мова, 2019. 1080 с.
4. Мартін Р. Чиста архітектура. Мистецтво розробки програмного забезпечення. Київ : Фабула, 2019. 368 с.
5. React – The library for web and native user interfaces. React Dev. URL: <https://react.dev/> (дата звернення: 14.02.2026).
6. JSON Data Interchange Format (ECMA-404). JSON.org. URL: <https://www.json.org/> (дата звернення: 25.03.2026).
7. Wix: Офіційний сайт платформи. Wix. URL: <https://uk.wix.com/>(дата звернення: 05.04.2026).
8. Tilda Publishing: Конструктор сайтів. Tilda Publishing. URL: <https://tilda.cc/uk/> (дата звернення: 12.05.2026).
9. Webflow: The modern web development platform. Webflow. URL: <https://webflow.com/> (дата звернення: 18.02.2026).
10. ДСТУ ISO/IEC 25010:2016. Інженерія систем і програмного забезпечення. Вимоги до якості програмного продукту та оцінювання (SQuaRE). Моделі

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		89

- якості системи та програмного продукту. Київ : ДП «УкрНДНЦ», 2018. 34с.
- 11.Про захист інформації в інформаційно-телекомунікаційних системах : Закон України від 05.07.1994, № 80/94-ВР. Відомості Верховної Ради України. С. 286.
 - 12.Фаулер М. Архітектура корпоративних програмних застосунків. Київ : Вільямс, 2018. 544 с.
 - 13.MySQL 8.0 Reference Manual. MySQL Developer Zone. URL: <https://dev.mysql.com/doc/refman/8.0/en/> (дата звернення: 07.03.2026).
 - 14.Фрімен Е., Робсон Е. Вивчаємо програмування на JavaScript. Київ : Фабула, 2020. 640 с.
 - 15.Vite: Next Generation Frontend Tooling. Vite. URL: <https://vitejs.dev/> (дата звернення: 22.04.2026).
 - 16.React DnD: Drag and Drop for React. React DnD. URL: <https://react-dnd.github.io/react-dnd/> (дата звернення: 10.05.2026).
 - 17.FileSaver.js: An HTML5 saveAs() implementation. GitHub. URL: <https://github.com/eligrey/FileSaver.js> (дата звернення: 01.03.2026).
 - 18.Tailwind CSS - Rapidly build modern websites. Tailwind CSS. URL: <https://tailwindcss.com/> (дата звернення: 29.04.2026).
 - 19.Framer Motion: A production-ready motion library. Framer. URL: <https://www.framer.com/motion/> (дата звернення: 15.02.2026).
 - 20.Lucide React: Beautiful & consistent icons. Lucide. URL: <https://lucide.dev/guide/packages/lucide-react> (дата звернення: 09.05.2026).
 - 21.Node.js Documentation. Node.js. URL: <https://nodejs.org/en/docs/> (дата звернення: 17.03.2026).
 - 22.Visual Studio Code Documentation. Visual Studio Code. URL: <https://code.visualstudio.com/docs> (дата звернення: 24.04.2026).
 - 23.Git: Distributed Version Control System. Git SCM. URL: <https://git-scm.com/doc> (дата звернення: 03.05.2026).

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		90

- 24.ДСТУ ISO/IEC 12207:2016. Інженерія систем і програмного забезпечення. Процеси життєвого циклу програмного забезпечення (ISO/IEC 12207:2008, IDT). Київ : ДП «УкрНДНЦ», 2018. 120 с.
- 25.React Router: Declarative Routing for React.js. React Router. URL: <https://reactrouter.com/> (дата звернення: 11.02.2026).
- 26.Axios HTTP Client Documentation. Axios Docs. URL: <https://axios-http.com/> (дата звернення: 28.03.2026).
- 27.Lodash Documentation. Lodash. URL: <https://lodash.com/> (дата звернення: 19.04.2026).
- 28.React Toastify: React notifications made easy. GitHub. URL: <https://fkhadra.github.io/react-toastify/> (дата звернення: 06.05.2026).
- 29.JSZip: Create, read and edit .zip files. JSZip. URL: <https://stuk.github.io/jszip/> (дата звернення: 21.02.2026).
- 30.React Image Crop: A responsive image cropping tool. GitHub. URL: <https://github.com/DominicTobias/react-image-crop> (дата звернення: 16.03.2026).

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		91

ДОДАТКИ

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		92

Глосарій

Адміністратор/Admin – користувач із найвищим рівнем привілеїв. Має повний контроль над інфраструктурою платформи, управлінням користувачами, глобальними налаштуваннями системи, фінансовою аналітикою та базою системних шаблонів. Може застосовувати санкції до будь-яких користувачів платформи. Доступ до профілю адміністратора додатково захищений обов'язковою двофакторною автентифікацією.

Блок/Block – основна структурна одиниця контенту на сторінці. Дані та налаштування блоку зберігаються у базі даних у форматі JSON і динамічно рендеряться на клієнті за допомогою ізольованих React-компонентів.

Звернення до підтримки/Support Ticket – механізм зворотного зв'язку, що дозволяє авторизованому користувачу створити запит до персоналу платформи для вирішення технічних проблем, отримання консультацій або подання офіційної апеляції на блокування власного вебресурсу.

Категорія/Category – сутність модуля електронної комерції, що належить конкретному сайту та використовується власником для логічного групування товарів і полегшення навігації для відвідувачів магазину.

Користувач/User – базовий обліковий запис, зареєстрований у системі. Характеризується унікальним ідентифікатором, електронною поштою, рівнем підписки та визначеною роллю. Звичайний користувач має право створювати власні сайти, керувати ними та запрошувати інших користувачів до спільної роботи.

Медіатека/Media Library – централізоване сховище графічних файлів, інтегроване з хмарним сервісом. Відповідає за збереження, оптимізацію, організацію та кадрування завантажених користувачем зображень для їх подальшого використання у блоках сайту або картках товарів.

Модератор/Moderator – представник персоналу платформи, наділений розширеними правами для контролю за дотриманням правил. Має доступ до

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		93

адміністративної панелі для обробки звернень користувачів, перевірки контенту та призупинення сайтів порушників. На відміну від Адміністратора, Модератор не має доступу до критичних системних налаштувань, бази даних та не може блокувати чи застосовувати санкції щодо адміністраторів або інших модераторів.

Панель керування сайтом/Site Dashboard – робочий простір та графічний інтерфейс власника вебресурсу. Надає комплекс інструментів для створення нових сторінок, керування електронною комерцією, перегляду базової аналітики та зміни глобальних налаштувань сайту.

Попередження/Warning – офіційний санкційний захід, що застосовується модератором або адміністратором до користувача у разі порушення правил платформи. Систематичне накопичення попереджень ініціює автоматичні обмеження, аж до повного видалення облікового запису користувача та всіх його даних.

Призупинений сайт/Suspended Site – вебресурс, статус якого було примусово змінено персоналом платформи через порушення умов використання. Публічний доступ до такого ресурсу блокується, а власник отримує сповіщення з можливістю подати апеляцію через систему підтримки.

Редактор візуальний/Block Editor – ключовий інструмент платформи, побудований на базі технології Drag-and-Drop. Дозволяє користувачу конструювати структуру сторінок, переміщувати блоки та детально кастомізувати їхній зовнішній вигляд у режимі реального часу без необхідності написання програмного коду.

Сайт/Site – ключова сутність платформи. Являє собою сукупність вебсторінок, об'єднаних єдиним унікальним ідентифікатором маршруту. Кожен сайт є незалежним проєктом із власним дизайном, налаштуваннями платіжних шлюзів та базою клієнтів.

Статус сайту/Site Status – поточний стан ресурсу, що визначає рівень його доступності в мережі. Залежно від етапу розробки або рішень адміністрації, сайт може мати статус: published, private, maintenance або suspended.

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		94

Сторінка/Page – окремий структурний документ у межах сайту, доступний за унікальним шляхом. Містить конфігураційний масив блоків. Одна зі сторінок ресурсу обов'язково призначається головною.

Товар/Product – об'єкт, що розміщується у каталозі інтернет-магазину користувача. Характеризується назвою, детальним описом, ціною, складськими залишками та галереєю зображень.

Шаблон/Template – попередньо сконфігурований набір дизайну, кольорових схем та структурних блоків. Може бути системним або персональним.

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		95

ФРАГМЕНТИ ВИХІДНОГО КОДУ ПРОГРАМНОЇ СИСТЕМИ

Лістинг Б.1. Реалізація клієнтського механізму автозбереження

(useAutoSave.js)

```

import { useState, useEffect, useRef, useCallback } from 'react';
import apiClient from '../api/api';
import { toast } from 'react-toastify';
import { debounce } from 'lodash';
export const useAutoSave = (endpoint, initialData, onSaveCallback) => {
  const [data, setData] = useState(initialData);
  const [isSaving, setIsSaving] = useState(false);
  const [lastSavedData, setLastSavedData] = useState(initialData);
  const dataRef = useRef(data);
  useEffect(() => {
    dataRef.current = data;
  }, [data]);
  const saveData = async (dataToSave) => {
    setIsSaving(true);
    try {
      await apiClient.put(endpoint, dataToSave);
      setLastSavedData(dataToSave);
      if (onSaveCallback) onSaveCallback(dataToSave);
    } catch (error) {
      console.error("Autosave error:", error);
      toast.error('Помилка автозбереження');
    } finally {
      setIsSaving(false);
    }
  };
  const debouncedSave = useCallback(
    debounce((newData) => {
      saveData(newData);
    }, 1000),
    [endpoint]
  );
  const handleChange = (field, value) => {
    setData((prevData) => {
      const newData = { ...prevData, [field]: value };
      if (JSON.stringify(newData) !== JSON.stringify(lastSavedData)) {
        setIsSaving(true);
        debouncedSave(newData);
      }
      return newData;
    });
  };
  return { data, handleChange, isSaving };
};

```

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		96

Лістинг Б.2. Ядро динамічного рендерингу компонентів (BlockRenderer.jsx)

```
import React, { Suspense, lazy } from 'react';
import { Construction, AlertTriangle, Loader2 } from 'lucide-react';
// Ліниве завантаження блоків
const blockMap = {
  hero: lazy(() => import('../blocks/Hero/HeroBlock')),
  text: lazy(() => import('../blocks/Text/TextBlock')),
  catalog: lazy(() => import('../blocks/Catalog/CatalogBlock')),
  form: lazy(() => import('../blocks/Form/FormBlock')),
  // ... інші блоки
};
const BlockRenderer = ({ blocks, isEditorPreview }) => {
  if (!Array.isArray(blocks) || blocks.length === 0) {
    return <div className="p-12 text-center">Контент відсутній</div>;
  }
  return (
    <div className="relative w-full">
      {blocks.map((block, index) => {
        const Component = blockMap[block.type];
        if (!Component) return <div key={index}>Тип блоку не знайдено: {block.type}</div>;
        return (
          <Suspense key={block.block_id || index} fallback={<Loader2 className="animate-spin" />>>
            <Component
              blockData={block.data}
              isEditorPreview={isEditorPreview}
            />
          </Suspense>
        );
      })}
    </div>
  );
};
export default BlockRenderer;
```

Лістинг Б.3. Middleware перевірки токена доступу (verifyToken.js)

```
const jwt = require('jsonwebtoken');
const db = require('../config/db');
const verifyToken = async (req, res, next) => {
  try {
    const authHeader = req.headers.authorization;
    if (!authHeader || !authHeader.startsWith('Bearer ')) {
      return res.status(401).json({ message: 'Відмовлено у доступі.' });
    }
    const token = authHeader.split(' ')[1];
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    const [users] = await db.query('SELECT token_version, status FROM users WHERE id = ?',
    [decoded.id]);
    const user = users[0];
  } catch (error) {
    return res.status(500).json({ message: 'Внутрішня помилка сервера.' });
  }
  next();
};
```

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		97

```

if (!user) return res.status(401).json({ message: 'Користувача не знайдено.' });
// Перевірка статусу акаунту
if (user.status === 'suspended') {
    return res.status(403).json({ message: 'Доступ заблоковано.' });
}
// Перевірка версії токена (Security Invalidation)
if (user.token_version !== decoded.token_version) {
    return res.status(401).json({ message: 'Сесія застаріла.' });
}
req.user = decoded;
next();
} catch (error) {
    return res.status(401).json({ message: 'Недійсний токен.' });
}
};
module.exports = verifyToken;

```

		Леус В.О.			ІПЗ.КР.Б. – 121 – 26 – ПЗ	Арк.
		Савіцький Р.С.				
Змн.	Арк.	№ докум.	Підпис	Дата		98